分类号 _____ 学校代码 10590
UDC _____ 密 级 _____

深圳大学硕士学位论文

基于分布式 Java 环境的卷积神经网络类库实现
——以深圳市高分二号卫星影像数据分类为例

学位申请人姓名 黄嘉俊

专 业 名 称 地理信息与智慧城市

学院（系、所） 土木与交通工程学院

指导教师姓名 张星 讲师

深圳大学学位论文原创性声明和使用授权说明

原创性声明

本人郑重声明：所呈交的学位论文 基于分布式 Java 环境的卷积神经网络类库实现——以深圳市高分二号卫星影像数据分类为例 是本人在导师的指导下，独立进行研究工作所取得的成果。除文中已经注明引用的内容外，本论文不含任何其他个人或集体已经发表或撰写的作品或成果。对本文的研究做出重要贡献的个人和集体，均已在文中以明确方式标明。本声明的法律后果由本人承担。

论文作者签名：  日期： 2020 年 7 月 15 日

学位论文使用授权说明

（必须装订在印刷本首页）

本学位论文作者完全了解深圳大学关于收集、保存、使用学位论文的规定，即：研究生在校攻读学位期间论文工作的知识产权单位属深圳大学。学校有权保留学位论文并向国家主管部门或其他机构送交论文的电子版和纸质版，允许论文被查阅和借阅。本人授权深圳大学可以将学位论文的全部或部分内容编入有关数据库进行检索，可以采用影印、缩印或扫描等复制手段保存、汇编学位论文。

（涉密学位论文在解密后适用本授权书）

论文作者签名：  导师签名： 

日期：2020 年 7 月 15 日 日期：2020 年 7 月 15 日

摘　要

随着时代的发展，不断涌现出了各种新技术，其中大数据、深度学习与云计算三项技术不只深刻影响我们的日常生活，也将更多的新思路带入了研究领域。在遥感研究领域，数据量的大幅增加虽然让研究有了长足的发展，但也随之涌现了两大问题，一个是海量数据的存储与管理问题，一个则是个人计算机越来越满足不了日益增长的计算需求问题。因此，越来越多的高校或实验室根据自身需求搭建了云计算平台以此保证自己的研究业务。本文即是依托所在研究组搭建的定制化混合云计算平台来展开，重点研究了基于分布式 Java 环境的卷积神经网络类库的实现。

本文在了解遥感研究人员在进行深度学习研究时出现的运行时间久、现有深度学习库较为难以配置调试分布式计算环境，以及现有卷积神经网络类库 Java 平台版本较少的现实问题后，明确了两个主要目标，可用的卷积神经网络类库以及在分布式环境下进行部署和计算。主要实现了两个模块：

（1）在较为系统的学习卷积神经网络（CNN）的前向传播与反向传播（BP）的原理以及数学模型之后，根据数学模型实现了 CNN 的前向传播代码，在 BP 的具体实现中根据梯度下降的原理设计并实现了前向式更新梯度的算法（Forward-BP），算法在进行前向传播时就记录并更新了参数的变化，并且在编码过程中更好实现。

（2）并行计算流架构下，根据 MapReduce 编程思想设计以每一次 Forward-BP 为一次子任务分发给计算线程，然后再汇总计算结果；在数据同步的架构设计上借鉴了 ConcurrentHashMap 分段加锁的结构，采用深/浅复制的搭配建立了主线程 HashMap 与子线程的数据通信，这在保证线程安全的前提下避免了锁的加入。在分布式环境中，结合 socket 技术，在主线程与计算线程中加入了通信线程保证前两者的通信与并行计算的持续进行，在分布式环境中也设计了相应的并行计算流架构与数据同步架构来理清不同线程间各自的任务以及中间的数据流向，以此为依据让编码实现。

本文最终较为完整的开发了一个可用于分布式计算和 Java 环境的卷积神经网络类库并且以深圳市高分二号卫星图像数据为案例，进行类库的测试和实验。实验结果表明，

本类库在分布式环境中，在保证了线程安全的前提下,可以对 CNN 并行计算 Forward-BP。

关键词：卷积神经网络；BP 算法；分布式计算；线程安全；遥感图像分类

Abstract

As time goes on, new technologies are emerging, including big data, deep learning and cloud computing, three of which are not only profoundly affect our daily lives and bring more new ideas into the field of research. In the field of remote sensing research, the dramatic increase in the amount of data has made great progress in research, but it has also led to two major problems: one is the storage and management of large amounts of data, and the other is that personal computers are increasingly unable to meet the growing computing needs. As a result, more universities or laboratories are building cloud computing platforms to ensure their research operations according to their own needs. The development of this paper relies on a customized hybrid cloud computing platform built by the research group and focuses on the implementation of a Convolutional Neural Network (CNN) class library based on a distributed Java environment.

This paper identifies two main goals, available CNN library and deployment and computation in a distributed environment, after understanding the realities of the long run times that arise when remote sensing researchers conduct deep learning research, the fact that existing deep learning libraries are more difficult to configure to debug distributed computing environments, and the limited Java platform versions of existing CNN class libraries. Two main modules are implemented:

After systematically studying the principle and mathematical model of CNN forward propagation and back propagation (BP), the CNN forward propagation code is implemented according to the mathematical model, and the forward upgrade gradient algorithm (Forward-BP) is designed and implemented according to the principle of gradient descent in the concrete implementation of BP. This algorithm records and updates parameter changes as it performs forward propagation, and is better implemented during coding.

The parallel computation flow architecture is designed according to the MapReduce programming idea that each Forward-BP is a sub-task distributed to the computation threads, and then the computation results are summarized. The data synchronization architecture design draws on the ConcurrentHashMap segmented locking structure, using deep/shallow replication to establish the data communication between the Master thread HashMap and the Slave threads, which avoids the addition of thread locks on the premise of ensuring thread safety. About in a distributed environment, combined with socket technology, in the Master thread and Slave threads added communication threads to ensure that the first two communication and parallel computing continues, the author designed a corresponding parallel compute flow architecture and data synchronization architecture to clarify the tasks between the different threads and the intermediate data flow, as a basis for implementing code.

This paper ultimately develops a more complete library of Convolutional Neural Network class library for use in distributed computing and Java environments, and presents the results in the form of a Shenzhen Gaofen-2 satellite image data as a case study to carry out the class library testing and experiments. The experimental results show that this class library can calculate Forward-BP on CNN in parallel in a distributed environment with the guarantee of thread safety.

Key word： Convolutional Neural Network (CNN); Back Propagation (BP) Algorithm; Distributed Computing; Thread Safety; Remote Sensing Image Classification

目　录

摘　要 I
Abstract III
第 1 章　绪论 1
1.1　课题背景及研究意义 1
1.2　相关技术研究概况 2
1.2.1　遥感分类研究概况 2
1.2.2　深度学习研究概况 3
1.2.3　深度学习库概况 6
1.2.4　云计算研究概况 7
1.3　研究内容与技术路线 9
1.3.1　研究目的 10
1.3.2　技术路线与主要研究内容 10
1.3.3　研究创新点 11
1.4　本文章节安排 12
第 2 章　相关技术及理论概述 14
2.1　深度神经网络模型概述 14
2.1.1　神经网络的诞生 14
2.1.2　从神经元到单层感知机 15
2.1.3　从单层感知机到深度神经网络 17
2.2　卷积神经网络结构概述 19
2.2.1　卷积层 20
2.2.2　激活函数 21
2.2.3　池化层 24
2.2.4　全连接层 25
2.2.5　损失函数 25
2.3 CNN 的 BP 算法 26

2.3.1 从损失函数到全连接层 26
2.3.2 从卷积层到上一隐藏层 29
2.3.3 从池化层到上一隐藏层 30
2.3.4 超参数介绍 30
2.4 并行计算框架概述 31
2.5 socket 技术概述 34
2.6 本章小结 35
第 3 章 基于 Java 的 CNN 实现与测试 36
3.1 CNN 前向传播的 Java 实现 36
3.1.1 准备工作 36
3.1.2 融合了 gdal 的数据输入层模块 37
3.1.3 卷积层模块 38
3.1.4 池化层模块 40
3.1.5 全连接层模块 42
3.1.6 输出层模块 42
3.2 CNN 的反向传播网络模型设计 43
3.2.1 Forward-BP 设计思路 43
3.2.2 Forward-BP 卷积层模块 44
3.2.3 Forward-BP 池化层模块 46
3.2.4 Forward-BP 全连接层模块 47
3.2.5 Forward-BP 更新参数模块 48
3.3 测试 50
3.3.1 实验数据与预处理 50
3.3.2 测试与结果分析 50
3.4 本章小结 52
第 4 章 基于 CPU 运算的分布式 CNN 算法设计 53
4.1 单机多线程计算环境架构设计 53
4.1.1 并行计算架构设计 53
4.1.2 数据同步架构设计 55
4.2 分布式环境下的架构设计 56

4.2.1 分布式环境的并行计算架构设计 56
4.2.2 分布式环境的数据同步架构设计 58
4.3 实现与测试 59
4.3.1 基于 Java 的实现 59
4.3.2 测试与结果分析 62
4.4 本章小节 65
第 5 章 总结与展望 66
5.1 主要工作总结 66
5.2 不足与展望 67
参 考 文 献 68
指导教师对学位论文的学术评语 73
学位论文答辩委员会决议书 74
致 谢 76
攻读硕士学位期间的研究成果 77

第1章 绪论

1.1 课题背景及研究意义

随着我国综合国力的提升，国家对地理地物的重视程度也在逐渐提高，因为这是直接关系到国家利益与人民福祉的。对海洋主权的愈发重视[1]，在 2013-2015 年全国范围内进行的地理国情普查[2]等都直接说明了明确我国地理信息的重要性。而城市空间检测则是地理国情监测中的重中之重，因为它对城市的发展与城市群的建设具有极大的现实意义[3]。在城市的建设过程中，怎么对市域内的地物进行有效管理一直是城市管理者亟需解决的难题，特别是现今我国的城镇率越来越高，城市范围的不断拓张使得地物的变化极快，面对这一复杂的现实情况，遥感（Remote Sensing）研究提供了解决方案，即利用各种遥感平台获取的数据地物进行检测，其中的关键技术即是遥感分类。

高分辨率遥感影像广泛地应用于地表分类、目标探测与国防建设等诸多领域，其中分类是高分辨率遥感影像处理和应用的一项重要内容，其最终目标是给影像中的每个像元赋以类别标识[4]。现在的遥感影像分类有多种分类方法，根据是否有监督分类来划分[5]，或根据参数和非参数来划分[6]，但不管是何分类依据，其中提及的深度学习方法是当今的一个研究热点，因为在计算机视觉领域它不仅得到了研究界的广泛认可，也在工业界被广泛应用，因此也有越来越多的中外学者将深度学习运用到遥感分类上。

遥感经过这几十年的发展，特别是我国近年来航天技术的突破，使得遥感影像数据种类越来越多，分辨率也越来越高。这给遥感的研究带来了新的机遇，数据的愈发全面带来了研究的不断突破，但遥感数据变得更广和更细也给处理分析带来了相应的挑战。比如遥感分类应用中的深度学习就是需要大量的标记数据去训练模型，训练好的模型基本只在固定模式的识别环境中才有较高的分类准确率，而地物的不断变化以及云层的干扰等难题也让这个“固定模式”难以固定。因此不管是初次应用还是后期都需要去训练参数，面对如此的海量数据，训练是非常耗费计算资源的，仅仅在个人电脑或服务器上存储与计算或许短时间能勉强支撑业务需求，但长此以往必然会极大的影响工作效率，而云计算平台就为海量数据的存储与计算提供了一种解决方案。

大数据、深度学习与云计算平台是当今互联网时代的三驾马车，不止给商业界及工业界带来无限的机遇，也让象牙塔上的研究有了更开阔的眼界，而且这三者的联系越来越紧密，可以说是相互依存的关系，大数据使得深度学习有了数据基础与应用场景，云计算平台让数据有了存放之地同时也支撑着深度学习的广泛应用。云计算平台并不如其名字上狭义的仅指计算平台，它也承担着数据库的责任，维基百科将云计算定义为是一种按需提供的计算机系统资源，特别是数据存储（云存储）和计算能力，更进一步的加州大学伯克利分校将云计算平台定义为提供服务的平台以及这些服务所依托的软硬件设施[7]。云计算在提供服务上可以分为三个层次，一个是基础设施即服务（Infrastructure as a Service，IaaS）、平台即服务（Platform as a Service，PaaS）以及软件及服务（Software-as-a-Service，SaaS），三者所提供的服务平台层网上，从硬件到平台再到软件，客户可以根据其需求选择相应的服务[8]。对于遥感海量数据的存储与运算需求，选择的服务偏向于 PaaS 这一服务类别，因为 PaaS 不仅可以有效的管理数据，也可以在 PaaS 服务上进行深度学习等计算任务。但阿里云及 AWS 这类商业公有云虽然提供了不同的实例以应对不同的需求，但灵活性依然不足，再者遥感数据大多具有保密级，而建设私有云或混合云即可解决这方面的问题。

因此，针对这一现实需求，实验室提出了建设“定制化混合云计算平台”，此平台正在搭建中，本研究所实现项目即是平台搭建中所需实现的一个子项目——针对遥感数据在分布式环境下的深度学习实现。根据实验室的硬件设施、所用数据以及研究人员的需求来定制化的实现以供遥感研究人员使用。本文根据研究目标实现了以 Java 为源码的 CNN 库，其中相对创新的根据反向传播的数学模型与原理设计并实现了前向式更新参数算法 Forward-BP，并且在设计出通用性的并发计算架构后将其运用于 Forward-BP 在分布式环境下进行并行计算的设计蓝图，以之为依据进行 Java 实现。

1.2 相关技术研究概况

1.2.1 遥感分类研究概况

针对遥感图像的分类方法按照是否有先验类别可分为非监督分类与监督分类，若按利用图像要素的不同可分为三大类：基于图像灰度值、基于图像纹理、基于多元信息融合[9]，但大体上以前两类灰度值分类与纹理分类为主，这两类以对数据是否有特定的预处理提取特征为别，其实这与计算机领域的图像分类方法是共通的。基于图像灰度值分

类即为直接从图像像元的数据信息来分类的话，有以距离的分类法[10],[11]，基于 K 均值聚类的方法[12],[13]、以降维为目的的主成分分析（Principal Ccomponent Analysis）方法[14]以及可用于夜光遥感的极大似然法分类[15]等。基于图像纹理方法的原理是利用不同地物的光谱差异进行处理后从而提取出地物的纹理信息，其中有马尔可夫随机场（Markov Random Fields）[16]、共生矩阵法（Co-occurrence Matrix）[17]、分形法[18]等，这方面的分类技术虽然相对老旧，但依然有一定的效用。随着研究的发展，一些研究人员认为上述两种分类方式存在着一个问题，即对研究单位——像元，定死了一个种类，但这往往因为空间分辨率过小而不准确，因此出现了混合像元分解法[19]、模糊分类法[20]等融合了大量数学模型的分类方法，这种方法大多存在着模型复杂、理解难的特点。同时又有一部分研究认为上述的研究对象选取不那么科学，因为以数据格式来说其研究对象是像元，而不是一片图像数据，是不符合地理学中的空间关联性概念，是不契合地理学第一定律的，因此出现了基于对象（Object-based）的分类方法[20],[21]。除了上述方法以外，还有一种近年来随着理论及硬件的爆炸式发展而重新兴起的技术——深度学习，其不仅在计算机视觉领域有着广泛的发展，在遥感领域也同样如此，并且从传统的图像预处理、基于像素的分类、目标识别等任务，到目前具有挑战性的高层语义特征提取和 RS 场景理解等任务都可以应用深度学习去解决[22]。除了二十年前广泛应用于 RS 研究的 SVM[23]外，占据现今深度学习半壁江山的卷积神经网络（Convolutional Neural Networks, CNN）更是不得不提及的技术[24]。

1.2.2 深度学习研究概况

深度学习允许由多个层组成的计算模型学习具有多个抽象层次的数据表示。近几年，这些方法极大地提高了语音识别、视觉目标识别、目标检测以及药物发现和基因组学等许多领域的技术水平，但大部分时间对于深度学习的研究是曲折缓慢的。早在 1986 年，Rumelhart、神经网络“狂热信徒”Hinton 等就在《Nature》上使用反向传播算法（Back Propagation，BP 算法）发现大型数据集中的复杂结构，以指示模型应如何更改那些从前一层输入计算后一层输出的参数[25]。深度学习的基本结构——神经元并不是一个新事物，其理论基础与数学模型的起源非常之早，早在 1943 年就已经出现了以生物神经元为依据、解决二分类问题的 MP 神经元[26]。但是由于当时相关理论发展的缓慢，一直未能解决 MP 神经元对非线性分类的有效分类，再加上同时期的诸如支持向量网络（Support-Vector Networks）能使用技巧在更高维度上解决二分类问题[27]，神经网络的研究陷入了

缓慢发展期。期间较为有名的只有 Lenet-5 就是在 20 世纪 90 年代后期以 BP 算法为基础训练出来的深度神经网络[28]。到 2006 年，Hinton 等在《Science》上相对系统的说明了深度神经网络在特征学习能力上的优秀性能，并且可以通过无监督预训练方法确定初始参数值[29]，这是“深度学习”概念的最早提出。这篇文章让计算机视觉领域研究人员第一次认识到深度学习的概念，真正拉开了深度学习的序幕，然而当时的主流人工智能界认为神经网络是“没有头脑的废话(weak-minded nonsense)”，而 Hinton 依然继续着他的神经网络研究工作。而真正掀起深度学习的研究热潮是在 2012 年的 ImageNet ILSVRC 分类挑战赛上， Hinton 和他的学生 Alex Krizhevsky 设计的 CNN——AlexNet[30]获得了冠军，比赛中需要将 120 万张高分辨图像分为 1000 个不同的类别，错误率为 15.3%，大幅领先第二名 26.2%，至此，AI 学术界与工业界正式以神经网络为研究重点，同时神经网络也成为了深度学习的代名词。往后的几年是 CNN 模型结构井喷的时期，牛津大学设计的 CNN 类型 VGG（Visual Geometry Group）[31]模型在 2014 年的 ILSVRC 分类赛中获得了亚军，虽然同年的冠军 GoogLeNet[32]错误率更低，但 VGG 模型结构更适宜于迁移学习，而 VGG 最大的缺点在于参数量太多，达到 140Mb，这在并行计算时限制了 batch 的大小，但瑕不掩瑜，其依然是一个较为成功的、应用广泛的模型。ILSVRC 2015 的冠军 ResNet[33]运用了更深的网络，虽然它的错误率比 GoogLeNet 只进步了 0.5%到 3.57%，但其最大的贡献在于利用残差模块（residual block），它建立了前面层与后面层的“短路连接”（shortcuts，skip connection），这解决了 BP 训练时的梯度弥散与梯度爆炸问题，但这其实是一个存在争议的说法，随着 RseNet 被越来越多人认识，有些人认为残差模块只是提高了训练速度，而 ResNet 中未被重点强调的 BN（Batch Normalization）才是真正解决梯度弥散与梯度爆炸的原因。在 ResNet 基础上发展出了 DenseNet[34]，它进一步扩展了网络连接，对于网络中的任意一层，该层的输入是前面所有层的输出，而该层的输出是后面所有层的输入，它进一步的使得所有层的联系更加紧密。ILSVRC 2016 的冠军由我国公安三所设计的 Trimps-Soushen 获得，其模型基础依然是 CNN，算法也基本没有什么创新，大体上只是通过不断调参获得了更高的准确率，但提升也不大，只比 ResNet 提高了 0.5%的准确率。针对 ImageNet 图像集的分类正确率提升已经不大，基本模型结构都是 CNN，各种算法也相继被提出并运用，提高正确率的方法只是花费大量时间在调参上而不是模型结构或算法上的情况下，2017 年 ILSVRC 举办最后一次竞赛后就关闭了竞赛。上述内容重点概述了 CNN 用于图像分类的发展历程，目标识别与检测是计算机视觉研究领域的另一项重点研究领域，也是 ILSVRC 的分支竞赛之一，针对

这一细分领域也涌现了诸如 R-CNN(Region-based Convolutional Neural Networks)与更进一步的 Faster R-CNN 的模型结构，以及其他如 CUImage、BDAT 等非 CNN 结构的模型，R-CNN[35]与 CNN 的最大的不同在于整体框架上加入了 SVM 分类器，但不是算法上把 SVM 加入 CNN 中，而是运用选择性搜索 (SelectiveSearch，SS)将图像划分为 2000 个独立候选区（Box）后，每个搜索区都走一遍 CNN 的流程，将输出的固定长度的特征向量放入 SVM 中去做分类，最后将全部 2000 个 Box 进行边框回归（Bouding Box Regression，BB）后输出目标的类别和位置所在，简言之三步走：region proposal(RP)->CNN->分类（SVM）+regression（BB），但这其实并不是严格的端到端的检测框架，而是 2000 个端至一个端的框架；Faster R-CNN[36]与 R-CNN 的区别在于 RP 算法从 SS 变为 RPN（Region Proposal Network），分类器也从 SVM 变为 softmax 分类，BB 也变为了多任务损失函数边框回归（multi-task loss），它真正的将 RP 和 regression 这两个模块融入 CNN 中，从而真正的实现了端到端的检测框架。除了 CNN 在计算机视觉研究领域展现了强大的统治力外，与 CNN 相似但不同的递归神经网络（Recurrent Neural Network, RNN）[37]则占据着深度学习的另外半壁江山，它与 CNN 的主要区别在于神经元计算是与多个时间输出有关的，换言之它主要顾及了样本出现的时间顺序，需要对样本的输入进行时间排列，因此它的应用场景主要为文本和语音等序列数据上。

从以上的发展历程可以看出现在 CNN 的研究已经相对成熟，并且走向了市场运用，这几年已出现了市场化项目，譬如特斯拉的无人驾驶即是使用了各种基于 CNN 的模型结构与其他视觉库对摄像头数据中的车辆或交通标志进行实时的判断，我国的图森互联在自动驾驶领域也有一定的深耕。而下一个风口又是什么呢，Hinton 在 CNN 大火的时候，也深知 CNN 有着诸多问题，比如黑箱性、高消耗、迁移能力差等被 AI 界诟病的问题，他在 2017 年提出了一种全新的深度学习方法——胶囊网络（Capsule Network）[38]，它与 CNN 最大的不同之处在于它学习的不是标量参数，而是基础特征向量，更高层次的特征由这些基础特征向量组成，这说明它的学习能力更强，使用时泛化性更好，更接近于人的认知，但这几年它并没有比 CNN 表现出更好的性能，而且训练时间也较长，也没有解决事物重叠问题，但笔者认为由于它相对 CNN 具有相当大的先天优势，在更复杂的环境或者新的领域一定会比 CNN 表现更好。但不管怎么说，CNN 依然是现今深度学习的研究主流。

综上，CNN 的强大与目共度，即使大量的 CNN 变体也实实在在的证明了它们的应用性与可靠性，但 CNN 的基础结构依然没变，因此去做一个底层 CNN 库供后续研究是

不落后于时代的。再者，CNN 为什么能发挥作用在 AI 界并没有定论，也就是说其工作机制并没有真正的被人们摸透，可用的 CNN 是更多的是依靠研究人员基于现有较为成熟的 CNN 并根据自有项目中的数据样本经过不断的网络结构设计（包括层数、通道数、kernel 的大小等）、超参数的调整或新算法的引入来获得一个合用的 CNN 结构，当然也可以从新设计一个 CNN。

这也进一步的明确了本文的研究目的，不是为了探索出一个可用于遥感分类的 CNN 结构，而是为遥感研究人员提供一个可用的、稳定的、可高性能计算的 CNN 库进行 CNN 在遥感分类的研究与引用。

1.2.3 深度学习库概况

现有的主流深度学习库虽然应用广泛，譬如 TensorFlow[39]，Caffe[40]等，但它们基本都是以 C++为源码，研究人员或工程应用大多是用它们的 Python API 或 Java API 来直接使用或实现自己的深度学习结构，但如果需要对其中的某个算法进行改进或需要对整体框架进行改变，却不能用 Python 或 Java 这类较高级的语言来进行，因为这些库虽然是开源的，但因为速度原因都是基于 C++实现，换言之，若想在较高级语言代码不变的情况下改进算法还需要在 C++来改写，其中存在最大的问题在于 C++的特性，它的精髓——指针在大型项目中的的运用非常繁杂，在不熟悉这种成熟库的庞大框架及源码架构的情况下贸然改写其中的源码极易导致不知名的错误。而以 Python 为源码的深度学习库中最出名的当属 Yoshua Bengio 创建维护的 Theano 库[41]，但其已经在 2017 年停止更新，但依然被科研人员广泛应用，除了其优秀的 API 设计外，就是因为其源码为 Python，方便研究人员进行改写，甚至现在应用非常广泛的另一个以 Python 为源码的 Keras 依然可以选择 Theano 为 backend 来构建，另一个知名度相对较小的由 Intel 维护的 neon 库也已经被放弃，刚刚所提及的 Keras 也已经被 Tensorflow 基本整合进大框架中。这一定程度上代表了不同的深度学习框架在激烈竞争后的结果，其中虽然有资金效益的原因，但也有 Python 的性能较差的原因。但除了上述两个编程语言外，Java 语言作为三大编程语言之一却好像错过了这波深度学习的浪潮，而 Java 却是现今大多数实际工程的开发语言。

Java 因为面向对象易开发的特性以及多次重大的版本更新，从而具有了功能完备的内置库与健康的社区，最重要的是 Jre 的性能已在整体上接近 C++，其实用性也与诞生

时相比不可同日而语。因此使用 Java 作为源代码的深度学习库不再是性能差的代名词，同时 Java 在实现与架构设计上相比 C++更为便捷，在兼顾实用的同时更加的适用于研究。但由于其语言定位与开发应用，以其开发的深度学习库一直没有受到足够的关注度，也搭建不起完善的社区——社区支持是新生事物发展中最关键的一点——而导致活力不足。现在以 Java 为主语言的研究人员要么去学习 Python 语言，要么调用各种以 C++语言实现的深度学习库在 Java 的接口，譬如 Tensorflow 即有 Java API，但针对 Java 的 API 远没有 Python API 功能完整，而且对 Java 的 API 维护更新相对较少，并不能很快的使用最新技术。以 Java 为源代码的深度学习库非常之少，有些只是在其他业务逻辑中开发了深度学习功能，比如 Encog 或 Spark，或者是一些大的机器学习库里只实现了少许的深度学习功能，比如新西兰大学开发维护的 Weka 库，Weka 可以说是已经在 Java 机器学习界中首屈一指的存在了，但在国内的应用并不高，另一个首屈一指的 Java 机器学习库为 Deeplearning4j 库，它虽然主要业务逻辑是深度学习，但其设计的应用场景是商用，因此在专注点上并不是在算法的更新或 API 的优化等研究相关面，而是提供 API 给商业环境下构建的项目，使得在商业应用上可以运用深度学习功能。

综上，实现一个可供研究的 Java 深度学习库不仅符合本文的业务逻辑，也有其大环境下的实际意义。而在进行深度学习中主要存在着三个问题[42]：无标签数据问题、训练速度问题以及改进方法与结构问题，因此针对训练速度问题，在实现的 CNN 库中应该能在云计算环境下进行并行计算，这不仅可以提高深度学习的训练速度，也更加符合本研究的定位。

1.2.4 云计算研究概况

云计算是什么，这个问题直到近几年才逐渐明晰，因为云计算平台的出现时间并不长，各研究机构与商业公司针对其业务环境都对其有自己的一套定义，比如根据美国国家标准与技术研究院（NIST）定义，云计算是一种可以利用互联网技术根据需求对其资源池（存储、计算、系统等）进行便捷访问的一种计算模式[43]；根据加州大学伯克利分校的定义，云计算包括互联网上各种服务形式的应用以及这些服务所依托数据中心的软硬件设施，这些应用服务为 SaaS，而数据中心的软硬件设施就是所谓的云，云计算就是 SaaS 和效用计算[7]。IBM 对云计算的定义为：“云计算是一种计算模式，在这种模式中，应用、数据和 IT 资源以及服务的方式通过网络提供给用户使用。云计算也是一种基础

架构管理的方法论，大量的计算资源组成 IT 资源池，用于动态创建高度虚拟化的资源提供用户使用”[44]。而我们国家的中国科学技术大学的陈国良院士将云计算定义为一种依托互联网来把原本存储于个人电脑、移动设备等个人设备上的大量信息集中在一起，在强大的服务器端协同工作的计算模式[45]。总的来说，针对云计算更多的被定义为计算模式，但其也可以定义为一项按需提供的服务。

现在的云计算平台已经在商业上获得巨大成功，比较有代表性的是 Amazon 的 AWS，微软的 Azure，我国的阿里云[46]，腾讯云[47]等，它们提供的实例大多相同，覆盖了通用型、存储型、内存型、CPU 计算型、GPU 型等，所提供的类型是大同小异的。也有些公司推出专项优化型来针对极端业务需求，而这些服务应该划分为 PaaS，提供给用户的是平台，用户不需考虑维护底层硬件的运行状况，只需扩展自己的业务即可。PaaS 最关键的技术有三个，MapReduce 并行计算模型、分布式文件系统（Distributed File System，DFS）与分布式数据库（Distributed Database），其中 MapReduce 负责计算任务，后两者的关系为分布式数据库建立在 DFS 之上，共同负责数据存储任务。关于这三种技术，主要介绍 Google 的相关业务，因为 Google 是这三项技术的创始者与领航者，是相关商业领域的代表者，它们最大的缺点在于闭源。而 Hadoop[48]虽然是一个开源项目，但却是依据上述的三篇论文所实现的框架，在研究与理论上没有什么创新点，且效率远低于 Google 的，但它在研究机构及中小公司中都有着极为广泛的使用，因为它是开源免费的。

Google 将其自有的 DFS 命名为 GFS（Google File System）[49]，这是一种保证了 Google 内部程序业务逻辑高可用性、高可靠性、高可伸缩性的 DFS，它的存储单元是块。Bigtable[50]是建立在 GFS 上的大规模数据库管理系统，它主要是为了解决存储在 GFS 上的数据不能很好被更高层次的功能或组件调用所存在的问题，它将 GFS 中的数据块逻辑封装成了格式化及半格式化数据形式以便调用人员理解，并且提供了 API 方便程序员去调用。MapReduce 其实是一种编程范式或者说是编程思想，它第一次被系统的提出是 2003 年 Google 发表的那篇著名文章《MapReduce: Simplified Data Processing on Large Clusters》[51]，这是一篇在超大规模数据处理技术领域具有划时代意义的作品，其中介绍了如何从纷繁复杂的业务逻辑中抽象出 Map（映射）和 Reduce（规约）这样的编程模型，它让业务逻辑非底层的编程人员只需要编写一个 Map 函数和一个 Reduce 函数就可以使用分布式计算框架。但值得一提的是，现在 Google 内部已经不再使用 MapReduce，主要原因在于高建设成本及高维护成本。一个较大型的业务基本由多个 MapReduce 所组成，子 MapReduce 和业务逻辑如何紧密耦合让大部分程序员焦头烂额，而且业务往往

不是一层不变的，而如果要动其中一个子 MapReduce 就可能造成整个业务的耦合性出现动荡，使得业务之外需要花费大量的精力去不断的调试；除此之外也有性能的原因，这体现在 Google 在对 MapReduce 进行了十年的优化后让他们发觉重建一个并行计算模型反而成本更低，因此就有了 Google Flume/Apache Beam。但 MapReduce 作为一种编程模型依然在被使用，即使是替代 MapReduce 的 Apache Beam 也是从诸多项目中抽象出来的，这其中就包含了 MapReduce，可以说 MapReduce 作为一种编程思想并不是一种过时的东西。这二十年来，我国一些有实力的高校也在自建云计算平台，比较成功的有清华大学张尧学教授在 2004 年提出的“透明计算平台”[52]，其实本质上与云计算平台大同小异，定位偏向于 PaaS；也有东南大学为了暗物质研究而建的“AMS-02 应用平台”[53]，其为了一个业务需求提供了 IaaS、PaaS 和 SaaS 的由低到上的全套服务，可以说是国内较为成熟的科研云计算平台。

而深圳大学广东省城市空间信息工程重点实验室也在筹划一个定制化的混合云计算平台，混合在于数据的多样，定制化即是根据具体业务来实现。这个平台不止是为了迎合云计算时代的大趋势，也有其实际的业务需要，它是一个根据实验室的数据与研究需求而提出的一个云计算构想。“定制”是这个云计算平台的一大特色，它可以根据具体研究或业务来设计相应的分布式计算架构，本文的研究即为此平台关于“科学计算与人工智能计算支撑服务块”的子项目，是对大规模遥感图像数据分类的深度学习研究的定制化实现。

1.3 研究内容与技术路线

本文的研究内容主要是解决两大问题，一个是 CNN 库的实现，一个是在分布式环境下进行计算。前者的问题主要难点是在“学习”上面，也就是怎么进行参数训练，具体的就是 BP 算法的实现，这里作者设计了 Forward-BP 算法来解决，这个算法在 CNN 进行前向传播时就记录各参数的梯度变化。而在解决分布式环境下的并行计算问题上，作者主要通过设计计算流与数据同步这两个框架来解决，计算流框架明确了不同计算任务的特点及关系，数据同步框架则主要通过 HashMap 与深浅复制的搭配解决了在异步计算环境下的数据同步问题。

1.3.1 研究目的

在科研过程中，经常使用已有的现成库，让笔者通常只知其然而不知其所以然，因此在定制化混合云计算平台的遥感数据存储与处理这一子项目下，笔者出于学习底层技术的主要目的下，期望能在明晰深度学习的基本理论与分布式计算的流程后能独立自主的设计一个能在分布式环境下进行深度学习的框架，并且在实现过程中尽量的做到易读、易拓展、易用的三个目的。同时也可以根据实际项目，即遥感研究提供一些定制化的帮助，比如读取遥感数据以及批量切割图像等任务。

1.3.2 技术路线与主要研究内容

本文的技术路线采用 “探索-设计-实现”三步走的研究模式来推进，重点以两大模块进行。

第一个模块主要是实现一个基于 Java 的 CNN 库。“探索”阶段集中在于明晰 CNN 的网络结构以及网络的运作方式，同时明晰不同类型层的概念、特点以及相应的数学模型，在这中间也要掌握各自的参数以及超参数，以此来作为 CNN 库顶层设计的基础，第 2 章的 2.2 节即为 CNN 前向传播的“探索”，2.3 节即为 BP 算法的“探索”；在“设计”阶段，根据以上所探索的内容对不同类型层的功能进行设计以及明确其变量，这将直接体现在后续的 Java“实现”阶段，CNN 前向传播的设计在本文中并不明确，因为相对传统并没有更多的创新点，BP 算法在文本中设计了一种前向式 BP，Forward-BP，其设计思路在 3.2.1 节中；在“实现”阶段，原则上，对所涉及的功能实现上应该以“探索”阶段的所定义的数学模型为基础，但可以在不影响整体逻辑结构的基础上进行有限程度上的改变，CNN 前向传播的“实现”整体在 3.1 节中进行描述， 3.2.2 节到 3.2.5 节描述了则描述了 Forward-BP 的不同层的结构及重要函数的流程。最后在 3.3 节则测试了所实现的 Forwrad-BP 的功能，即是否可以进行梯度下降。最后所实现的 CNN 单线程库在运行逻辑没有第三方库加入的情况下可以进行梯度下降，实现了“机器学习”的功能。

第二个模块的主题是关于分布式计算框架，一样根据“探索-设计-实现”的框架设计流程来进行，主要在第 4 章详细描述。这一模块虽然是以提升批量 Forward-BP 的运算速度来展现，但其在设计时是尽量普世的，不只局限于本研究的，因此 Forward-BP 在其中只以一个模块存在。在“探索”阶段，深入了解了 MapReduce 编程范式在 Hadoop

的经典实现明晰了并行计算的组织及 Java util 包里的 ConcurrentHashMap 的结构如何保证数据同步，同时也较为深入的了解了在分布式环境下的通信技术 socket 的流程，这一步为后续的设计与实现提供了基础，在 2.4 节及 2.5 节有详细描述。在“设计”阶段，主要从两个方面设计并行计算，一个是计算流，一个是数据同步，前者主要以 MapReduce 为基础进行设计，明确了不同线程间的计算流的流向，后者则借鉴了 ConcurrentHashMap 的结构，但不分段加锁，而是每个键值对都独立，并且结合了深浅复制的特性，这使得 HashMap 从线程不安全变为了线程安全，而且还避免了锁的加入，4.1 节为针对单机多线程环境下的设计，4.2 节则为多机分布式环境下的设计。4.3.1 节则为 4.2 节的实现部分，其中根据业务具体引入了 socket 技术保证了主机间的通信，主要实现了主线程类、通信线程类、计算线程类这三种类。4.3.2 节则为具体的测试以及结果分析，测试了在分布式环境下所设计实现的架构在进行并行运算的同时也能保证线程安全，并且分析了不同硬件间的性能差距，得出了在组建云计算平台时应软硬件一起并重的结论。

总体技术路线可归纳为图 1-1。

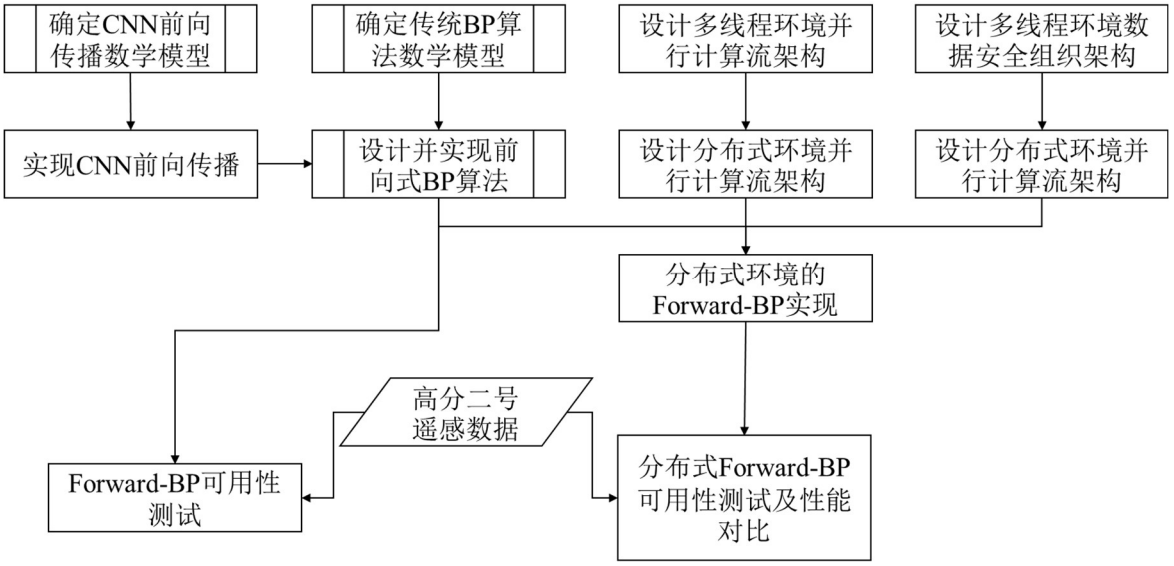


图 1-1 本文技术路线图

1.3.3 研究创新点

创新点主要有两大方面：

（1）创建了一个基于 Java 语言的 CNN 类库:1)对比现在的主流深度学习库基本以 C++为源码，以 Java 为源代码，且技术框架与逻辑完全以 CNN 的数学模型与逻辑流程进行构建，便于 Java 从业或研究人员快速上手，而且因为源码中在运行逻辑中几乎没有

第三方库，因此在后续的实际开发中可以根据具体业务在源码上做功能性的拓展或代码的改写 2)针对遥感研究做了相应的优化，融合了 gdal 遥感专业库来读取遥感图像，重点解决了如 TensorFlow/Keras 的等主流库不易读取多通道数据的痛点 3)根据 BP 算法的数学模型设计并实现了前向 BP 算法（Forward-BP），降低了 BP 算法代码实现的逻辑难度与编写难度。

（2）分别对在单机环境与多机环境下进行了架构设计: 1)在单机多线程环境下，借鉴了 MapReduce 编程思想，设计了并行计算流，它的结构由 Master/Slave 模式组成，Master 线程分发任务，Slave 线程根据任务计算；借鉴了 ConcurrentHashMap 的结构，使用 HashMap 与深/浅复制的特性设计了数据同步架构，在保证线程安全性的同时也避免了线程锁的加入 2)在多机分布式环境下，引入了 Socket 技术设计并实现了分布式分布式 Forward-BP 运行框架，这个框架比之单机多线程环境在 Master 与 Slave 线程之间加入了通信线程，使得两类线程能一直保持通信，确保并行计算在分布式环境下的运行。

1.4 本文章节安排

本研究的文章组织架构分为以下五章层层递进：

第一章，绪论部分。介绍了本研究的项目背景与依据来源，依托在建的“定制化云计算平台”项目展开，进行对遥感图像数据深度学习的定制化构建研究。在研究过程中所涉及的图像分类、深度学习研究现状、并行计算等相关技术现状做了一个大体介绍。最后在理清了整个研究的组织架构与流程脉络的前提下，明确了本文的研究目的为实现一个可在分布式环境下进行深度学习计算的 CNN 库。

第二章，理论部分。首先针对 CNN，在明确各层级结构的同时也对其做出了数学定义，尽量以元素法作定义，这可以让后续的实现更有参考性；然后详细介绍了 CNN 反向传播训练参数的数学推导，明确了其以梯度下降的实现原理，也为后续的 Forward-BP 提供理论依据；而后探索了 Hadoop 中的 MapReduce 的流程与逻辑结构以及线程安全容器 ConcurrentHashMap 的结构与实现思路，这为第四章的分布式计算环境的搭建提供了理论基础；最后也对分布式计算的必要组件 Socket 技术进行了概述。

第三章，CNN 实现部分。根据第二章所定义及推导的数学模型实现了 CNN 的前向传播，并且以实际情况出发，提出了 Forward-BP 算法让 CNN 得以进行学习并更新参数。最后测试了前向与 Forward-BP 这两个模块的可行性，都获得了成功，证明了 Forward-

BP 的可用性。

第四章，分布式计算实现部分。针对第三章 Forward-BP 所存在的运算效率问题，同时也为了能充分利用云计算平台的计算服务，根据 MapReduce 编程思想设计了并行计算流架构，根据 ConcurrentHashMap 分段加锁的实现思路设计了并行计算环境下的数据架构。以设计的架构为理论依据，简单清晰的实现了 Forwad-BP 在分布式环境下的并行计算，并且在后续的测试中根据测试结果明确了硬件配置与算法设计同等重要的结论。

第五章，总结与展望部分。本章总结了全文所做的工作，归纳了本研究所实现的 CNN 类库的主要功能以及如何实现分布式下的并行计算，并且从功能缺陷上展望了本类库的未来研究方向，也指出了本研究中的分布式计算的优化方向。

第2章 相关技术及理论概述

本章重点讲述基本卷积神经网络（Convolutional Neural Network，CNN）的层级结构与其数学模型，以此为后续的 Java 实现提供理论基础。本章也将介绍用于并行计算的 MapReduce 编程思想以及在多线程并发环境下保证线程安全的 ConcurrentHashMap 这一并发安全容器的结构以及实现思路，最后介绍了用于分布式数据通信的 Socket 技术。

2.1 深度神经网络模型概述

2.1.1 神经网络的诞生

在了解深度神经网络之前，需要大体知道它的前世今生。神经网络的发展是漫长曲折的，严格来说它并不是一个新兴的事物，其最远可追溯至 1943 年由神经学家 McCulloch 和数理逻辑家 Pitts 在神经科学的第一种计算方法——MP 模型[26]，其 MP 即取自两位科学家名字的首字母 M-P，其具体数学模型如式(2.1)，从生物神经元到 MP 模型的参数关系如表 2-1，其示意图如图 2-1，这个数学模型代表了一个神经元需要与它相连的神经元给予足够的总刺激自身才有反应，而后在向后续神经元传递的信号是以自身接受的信号量为参数而输出的一个信号量。这简单的数学模型就是现代计算机逻辑设计以及人工智能的支柱，结合现在神经网络百花齐放的状况不禁感叹万物始于一，再繁复的研究也有其最基本的理论。

$$y_j = \sum_{i=1}^{n} w_{ij}\, x_i - \theta_j \qquad (2.1)$$

表 2-1 生物神经元和 MP 神经元模型对应关系表

生物神经元	MP 神经元
神经元	j
输入信号	x_i
权值	w_{ij}
输出信号	y_j
膜电位	$w_{ij}x_i$
阈值	θ_j

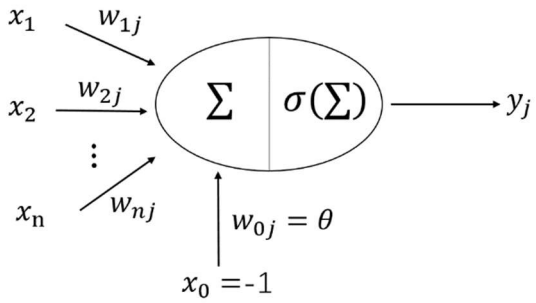


图 2-1 MP 模型示意图

2.1.2 从神经元到单层感知机

在 1958 年，基于 MP 神经元模型，Rosenblatt 首次定义了一个单层计算单元的结构，并称之为感知机（Perceptron），这是为了解决二分类问题而出现的，本质上是将 MP 模型设定了分类应用场景[54]。假设有一组不限维数据二元可分，即在有 m 组 n 维数据里都有一个相应的输出，可以数学表达为$(x_1^{(0)}x_2^{(0)},\dots x_n^{(0)},y_0)$ ，$(x_1^{(1)}x_2^{(1)},\dots x_n^{(1)},y_1)$ … $(x_1^{(m)}x_2^{(m)},\dots x_n^{(m)},y_m)$。以图 2-1 的形式，若要对每一组数据需要分类，我们需要找到一个超平面在两类中间，即超平面可表达为$\mathrm{w}_1x_1+\mathrm{w}_2x_2+\cdots \mathrm{w}_nx_n+\theta=0$,需要提及，从其式子可知，如若其存在，则其肯定并非唯一解。而为了简化其表达，在超平面式中的偏置函数θ加一$x_0=1$，则可变形为累加式$\sum_{i=0}^{n}w_ix_i=0$，进一步的，以向量表示，W表示w的向量，X表示x的向量，则上述累加式可为$\mathrm{W}\bullet\mathrm{X}=0$，其中•表示内积，为了理解，做出一维数据二维平面如图 2-2 所示，而在图 2-1 中 MP 模型的输出信号y是要经过激活函数f的。以上，即可构造出一组数据的单层感知机的数学模型，如式(2.2)：

$$z=\mathrm{W}\bullet\mathrm{X}$$
$$y=\sigma(z)=\begin{cases}-1 & z<0\\ 1 & z\geq 0\end{cases} \qquad (2.2)$$

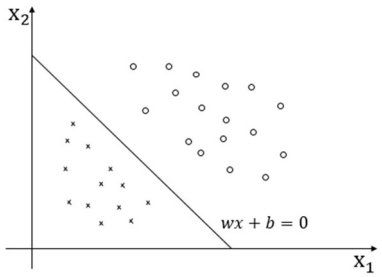


图 2-2 感知机二维空间示意图

为了找到一个最好的超平面，即找到一个 m 组数据到这个超平面是最短距离的，为

了求这个最短距离不以传统的最小二乘法来找，因为这是有训练集与测试集之分的，如果训练集由最小二乘法求出的超平面最优，不一定在测试集也是如此，因此采用另一种思路，用梯度下降法来求，因此需要计算错误类点的损失函数，由式(2.2)知，求$y\mathrm{W}\bullet\mathrm{X}$即可获悉是否为错误类，当且仅当$y\mathrm{W}\bullet\mathrm{X}<0$时这样本为错误类，因此对于错误类的某样本$i$，其到超平面的距离为$-y^{(i)}W\bullet\mathrm{X}^{(i)}/||W||_2$，其中$||W||_2$为 L2 范数，因此所有错误类的点的集合 M 到超平面的距离之和为式(2.3):

$$\mathrm{J}(\mathrm{W})=-\sum_{x_i\epsilon M}y^{(i)}W\bullet\mathrm{X}^{(i)}/||W||_2 \qquad (2.3)$$

由上式可知这是一个W为因变量的损失函数。又因为分母$||W||_2$这个 L2 范数与分子直接成比例，因此J(W)可以简化为式(2.4):

$$\mathrm{J}(\mathrm{W})=-\sum_{x_i\epsilon M}y^{(i)}W\bullet\mathrm{X}^{(i)} \qquad (2.4)$$

有了损失函数就可以使用梯度下降法，而且这是单层感知机模型，因此直接对W求导即可得出，但要注意的是这其中有三种梯度下降方法，一个是批量梯度下降法（Batch Gradient Descent, BGD），一个是随机梯度下降法（Stochastic Gradient Descent, SGD），小批量梯度下降法（Mini-batch Gradient Descent, MBGD），区别在于参与梯度下降更新时的样本数，BGD 是全样本，SGD 是一个样本，MBGD 是一部分样本，训练速度 BGD<MBGD<SGD，BGD 最大的缺点是如果样本量大更新速度慢，SGD 虽然速度快但每次迭代的样本对象一般不同，会造成梯度下降方向变化很大，因此不能很快达到局部（全局）最优解。在单层感知机中因为样本并不全是错误类，因此选用 SGD 或 MBGD，一般采用 SGD，虽然是 Stochastic 但样本肯定确定是错误类样本，只不过仅针对一个样本而已，由此，J变为j，代表为单个样本更新，假设采用第i个样本，对任意参数w求导得式(2.5)(2.6)：

$$\frac{\partial}{\partial w}j(\mathrm{W})=-y^{(i)}\bullet X^{(i)} \qquad (2.5)$$

确定步长α，则任意参数w的梯度更新公式为式(2.6)：

$$\mathrm{w}=\mathrm{w}-\alpha(-y^{(i)}\bullet\mathrm{w}^{(i)}) \qquad (2.6)$$

由此，式(2.6)即为单层二分类感知机的单步训练结果。

单层感知机有一个最大的问题是不能对二分类以外的问题，譬如经典的异或（XOR）问题，其描述了一种二元分类，样本为二维数据，如果数据两个元素相同则为正，如果

不同则为负，但光使用超平面不能解决此类问题，换句话说就是数据是线性不可分的，图 2-3 即为示意图，不管怎么画线都不可能将绿点为一类，蓝点为一类。数据如果线性不可分则满足不了使用单层感知机的先验条件，但同时期的支持向量机（SVM）却可以通过技巧将数据在高位可分，因此单层感知机退出了研究舞台。

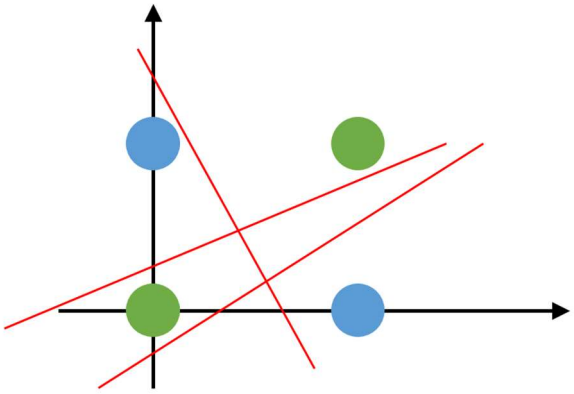


图 2-3 XOR 问题示意图

2.1.3 从单层感知机到深度神经网络

上一节介绍了单层感知机的使用条件有一个很大的缺陷即是需要分类数据是线性可分的，而为了解决这个问题，多层感知机（Multilayer Perceptron, MLP）就出现了，或者称其为人工神经网络（Artificial Neural Network, ANN）或深度神经网络（Deep Neural Networks, DNN）,其实都是一类事物，其余单层感知机的最大区别是增加了在输入层与输出层中间增加了隐藏层（Hidden Layer），其结构可拓展为多层神经网络，如图 2-4，而每一个神经元的数学模型依然是感知机模型，即为类似式(2.2)的形式，但在层与层的传递间不再只是非连续函数的形式。这使得最后的数学模型不再是超平面的，或者说不再是线性的，而是非线性的，而且也能观察到 DNN 不仅可以解决二分类问题，由于层数的增加也可以解决多分类问题了。

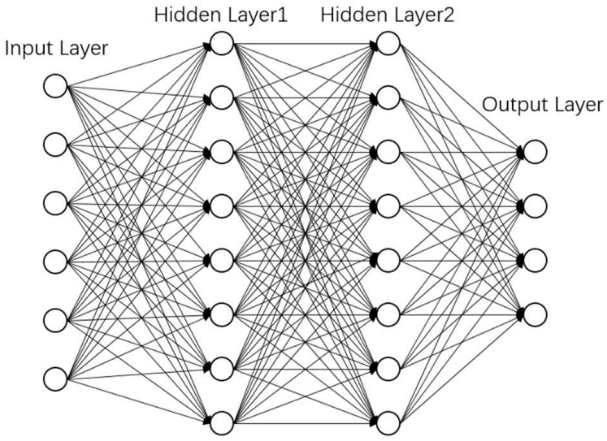


图 2-4 DNN 结构示意图

而具体的数学模型定义如下，将图 2-4 缩略至如图 2-5 所示：

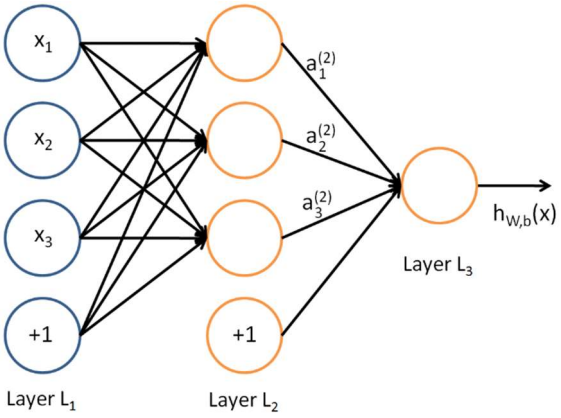


图 2-5 三层 DNN 示意图

这是一个三层 DNN，如单层感知机一样，需要定义一个激活函数$\sigma(z)$，其目的在于使得整个神经网络是非线性的，不然再多的层数都可以参数累乘为一个多元一次方程，因此$\sigma(z)$必须是非线性函数，现在一般选用 Relu 函数来顾及速度，Sigmoid 函数也是一个选择，则由图知第一层的输出为x_1, x_2, x_3,对于第二层的输出a_1^2，a_2^2，a_3^2分别有式(2.7) 式(2.8) 式(2.9):

$$a_1^2 = \sigma(z_1^2) = \sigma\left(\sum_{k=1}^{3} w_{1k}^2 x_k + b^2\right) \quad (2.7)$$

$$a_2^2 = \sigma(z_2^2) = \sigma\left(\sum_{k=1}^{3} w_{2k}^2 x_k + b^2\right) \quad (2.8)$$

$$a_3^2 = \sigma(z_3^2) = \sigma\left(\sum_{k=1}^{3} w_{3k}^2 x_k + b^2\right) \quad (2.9)$$

对于第三层的输出a_1^3有式(2.10)：

$$a_1^3 = \sigma(z_1^3) = \sigma\left(\sum_{k=1}^{3} w_{1k}^2 x_k + b^3\right) \quad (2.10)$$

则归纳上述两层的输出形式，有式(2.11)：

$$a_j^l = \sigma\left(z_j^l\right) = \sigma\left(\sum_{k=1}^{m} w_{jk}^l a_k^{l-1} + b^l\right) \quad (2.11)$$

其中l表示输出层数，j表示神经元位置，k表示上一层神经元的位置，进一步的，假设第$l-1$层有m个神经元，第l层有n个神经元，则w_{jk}^l可表示为一个$n \times m$的矩阵W^l，说明了在 DNN 中每一个连接都有一个系数，a^{l-1}表示一组$m \times 1$的向量，b^l表示为一组$n \times 1$全为相同值的向量，z^l为一组$n \times 1$的中间向量，a^l表示一组$n \times 1$的第l层的输出，则可以式(2.11)可简洁的用矩阵法表示为式(2.12)：

$$a^l = \sigma(z^l) = \sigma(W^l a^{l-1} + b^l) \quad (2.12)$$

至此，DNN 的模型结构与其数学模型表达完成。

2.2 卷积神经网络结构概述

从 DNN 到 CNN，有什么联系与区别呢，按笔者的理解，CNN 的出现在于根据图像的特性，相近的像元蕴含的信息相近这一前提提出了 CNN 这一网络结构，它减少了参数量与训练时的连接数，而从数学模型来剖析两者并没有具体区别。

其网络结构如图 2-6 所示，譬如在图中最左边的图像就是我们熟悉的影像图，其有三个通道 RGB，每个通道是一个二维矩阵，二维矩阵中的每个元素即为影像中对应位置在这一通道的数值。在图中经过了一次卷积运算及 ReLu 激活函数后变为了一个有三个 feature map——可以通俗的理解为二维矩阵——的卷积层，卷积运算出来的值类似于式(2.8)算出的中间值z^l，只不过卷积运算时的W^l并不如 DNN 中一样每一个连接就有一个系数，而是有一个卷积核（kernel）的概念来压缩W^l的大小，而后的激活函数跟 DNN 一样是针对每一个元素都要进行的；卷积层池化（Pooling）后输出的是池化层，这是一种将 feature map 压缩的手段；在 CNN 中的最后一般都要经过全连接层让整个网络结构随着层数的深入让相似特征的变量更加紧凑以筛选出各项特征，最后根据一个输出函数就可以算出所处类别的概率。下面将大致介绍各层概念。

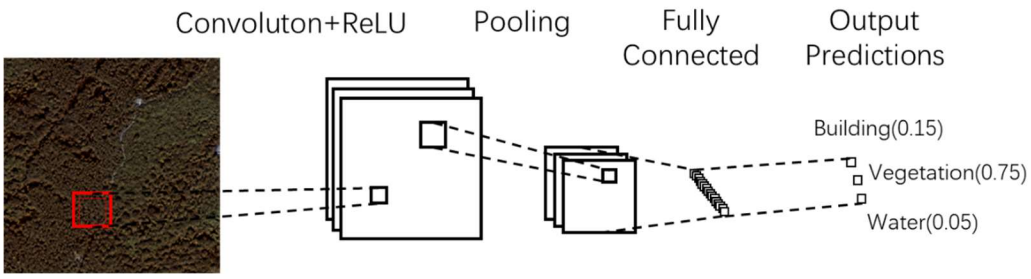


图 2-6 CNN 结构示意图

2.2.1 卷积层

卷积层的数学模型定义应该结合流程来确定，如图 2-7[1]，这张图出自斯坦福大学李飞飞的 CS231n（全称：面向视觉识别的卷积神经网络），可以说是 CNN 的入门图，其清晰直观的展示了 CNN 的卷积过程。图中左边为 Input Volume，是一个三通道的影像图，它的维度是 (7×7) ×3，在此定义张量（tensor）这个概念，它即为每一个二维矩阵，为了表达将其提出，因此 Input Volume 为一个 3 维张量数据；中间红色代表了 kernel，有两个 kernel，每个 kernel 的维度是(3×3)×3，套用上述张量的概念，则每一个 kernel 是一个 3 维张量，张量的大小是 3×3，可以看到每一个 kernel 的张量数与 Input Volume 的张量数相等；最右边是 Output Volume，其维度为(3×3)×2，则它是一个二维张量，每一个张量的大小是 3×3，可以看到 Output Volume 的张量数与 kernel 的个数相等。需要注意的是，这本来是一个动图，截图的这一刻运算到了 Output Volume 第二幅 feature map 的 3 行 2 列位置，整个卷积运算的步长（stride）为两格。kernel 是 CNN 的精髓所在，它的理论依据在于在一副图像里，相近的元素所蕴含的特征或信息是相近的，而不同组 kernel 结合激活函数将会提取出不同的信息，当然这些信息基本不能为人所识别，但随着 CNN 的深度或广度的增加这些信息将会越来越清晰。

[1] 图片来源于 https://img-blog.csdn.net/20160707204048899

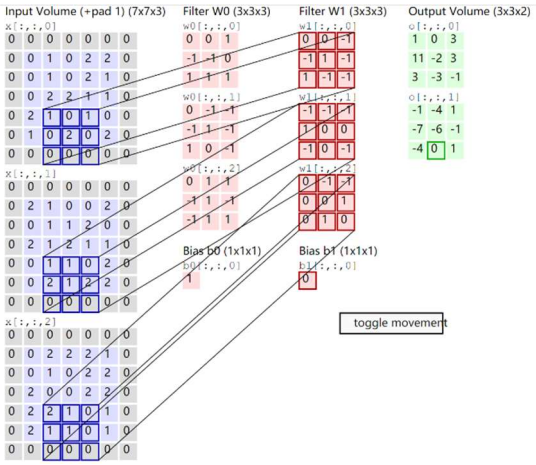


图 2-7 CNN 卷积过程示意图

根据上面的描述和图 2-7，再结合式(2.12)的 DNN 数学模型定义过程，则对中间变量有式(2.13)：

$$z_{dij}^{l} = \sum_{d',i',j'} w_{dd'i'j'}^{l} a_{d'(i*s+i')(j*s+j')}^{l-1} + b_{d}^{l} \tag{2.13}$$

其中l指输出层数，d指输出层数的张量维度，i指矩阵的行数，j 指矩阵的列数，d'、i'、j'指 kernel 的张量维度、张量中的矩阵行数、张量中的矩阵列数，s 指 stride 步长，w 指 kernel，a 指上一层的输出，b 指偏置项。可以看到式(2.13)的表达稍显复杂，但其是第三章的 Java 实现的数学基础，当然也可将式(2.13)以矩阵法表示有式(2.14)：

$$z_{dij}^{l} = w_{d}^{l} \odot a_{d}^{l-1} + b_{d}^{l} \tag{2.14}$$

⊙表示 Hadamard 积，向量中等同于点积，需要注意的是式(2.14)隐含了a^{l-1}确定点积位置这一信息，进一步的，可以将元素位置信息隐含，则式(2.14)可定义为式(2.15)：

$$z^{l} = w^{l} * a^{l-1} + b^{l} \tag{2.15}$$

同样的，式(2.15)隐藏了点积位置的信息，这样表达简洁虽不明了，但可为后续的数学模型继续定义。

2.2.2 激活函数

激励函数有时也被称作激活层，其主要任务是保证整个网络结构的非线性映射。算出了中间变量z^{l}后要经过激活函数来让整个网络结构所成的数学模型结构是非线性的，具体来说，激活函数应该具有以下六个特性：1)非线性，线性激活层对于神经网络没有作用，因为其作用以后整个网络依然是各种线性变换 2)连续可微，这是为了进行 BP 时训练时可求导数 3)非饱和性，因为在饱和或近饱和区域的梯度为 0，BP 计算的时候参数几乎不更新 4)单调性，主要在于求梯度的时候不会出现正负号震荡导致参数不能更

新 5)接近恒等变换，即导数值为 1，这样可以使得前向传播时输出值出现明显增加或突然归于 0，并且 BP 传播时也更好计算 6)计算应该简单，这样不管是前向传播还是 BP 的时候都能节省计算时间。

但现在没有一个激活函数完全符合上述条件，但特性 1)是必要的，选取合适的激活函数一个是经验与不断调试，一个是在效率与准确率之间抉择。以下介绍几种常用的激活函数，以σ表示激活函数：

1.Sigmoid 函数

$$\sigma(x) = 1/(1 + e^{-x}) \tag{2.16}$$

其函数图与导数图如图 2-8，Sigmoid 在很长一段时间都被研究人员认为它是最接近生物神经元的传导情况，优点在于其连续可微，并且值域范围在(0,1)，这与最后输出的概率暗合，但其最大的问题是饱和性问题，可从导数图观察到在大于 6 或小于-6 时导数的变化几乎为 0，这就导致了梯度弥散的问题；其次它的值域恒大于 0，使得靠近输出层的几层参数在迭代训练时很大可能只朝着一个方向更新参数，这可能造成整个 CNN 变得过拟合，这被称为非 zero-centered 问题。同时幂运算也对计算性能有消耗，对于有庞大数据集的训练来说会造成时间的消耗。

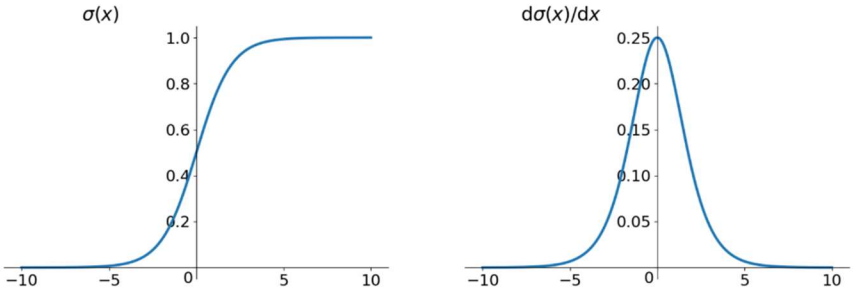


图 2-8 Sigmoid 函数图与导数图

2. tanh 函数

$$\sigma(x) = (e^{x} - e^{-x})/(e^{x} + e^{-x}) \tag{2.17}$$

其函数图与导数图如图 2-9，它与 Sigmoid 函数形式相近，主要解决上述的值域只大于 0 的问题，但除了解决非 zero-centered 问题外，最大的梯度弥散问题依然没有解决，但在实际调试 CNN 时依然是一个备选方案可供选择。

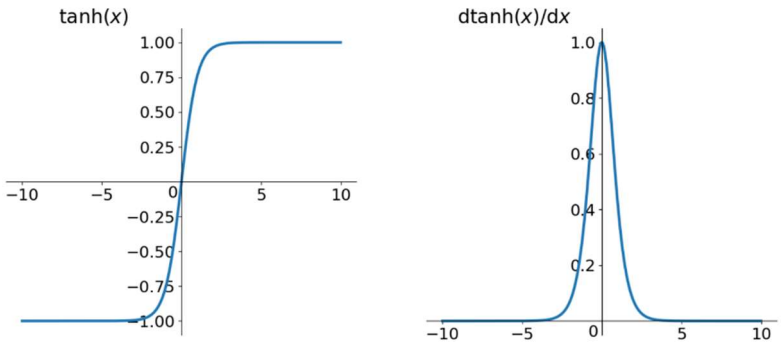


图 2-9 tanh 函数图与导数图

3. ReLU 函数

$$\sigma(x) = \max(0, x) \tag{2.18}$$

其函数图与导数图如图 2-10，ReLU 是现在 CNN 最常用的激活函数，它最早是在 AlexNet 中所使用，它可以说是 CNN 发挥作用的最大功臣之一，它主要在正区间解决了上述的梯度弥散问题，因其导数值在正区间恒为 1；而且计算速度非常快，在前向传播与 BP 时只需判断输入是否大于 0；在实际运用中它的收敛速度远快于 sigmoid 和 tanh。但它同样存在非 zero-centered 问题；同时也有 Dead ReLU Problem，指的是输入小于 0 的情况下参数永远不会被更新，这一般是因为参数初始化或者训练时的学习率设置太高导致。但不管怎么样，它依然是目前最常用最广泛有效的激活函数。其余的激活函数如 Leaky ReLU、ELU 等都为 ReLU 的变型或应用极少等原因，故不再列出。

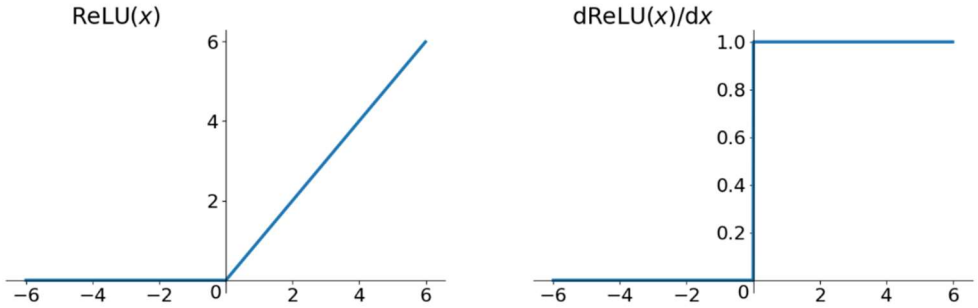


图 2-10 ReLU 函数图与导数图

确定了激活函数后，即可承接卷积层算出的中间值 z，若以矩阵元素为单位，则可表示为式(2.19)：

$$a^l_{dij} = \sigma(z^l_{dij}) = \sigma(\sum_{d',i',j'} w^l_{dd'i'j'} a^{l-1}_{d'(i*s+i')(j*s+j')} + b^l_d) \tag{2.19}$$

若以矩阵法，则为式(2.20)：

$$a^l = \sigma(z^l) = \sigma(w^l * a^{l-1} + b^l) \tag{2.20}$$

至此，卷积层前向传播数学模型定义完毕。

2.2.3 池化层

池化层一般在进行了激活函数计算以后，并不是一个必要层，它运算的结果是减小了张量的尺寸，但它的作用主要有两个，一个是控制了后续计算的运算量，一个是为了控制过拟合，因为它让一些图像信息消失了。池化层根据池化视野（filter）来对输入的张量进行矩阵值的计算，有两种池化方式，一种是 mean-pooling，取池化视野中所有值的平均值，一种是 max-pooling，取池化视野中的最大值，如图 2-11 展示了两种池化方式。

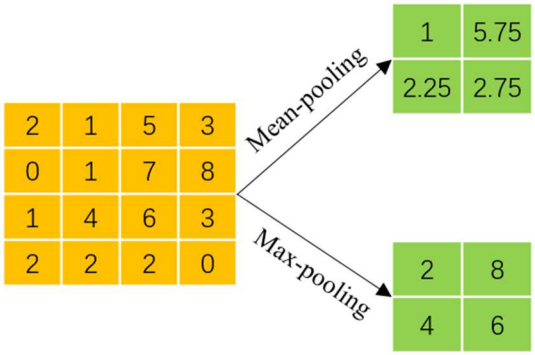


图 2-11 池化的两种方式示意图

可以看到池化的逻辑简单，而根据池化定义，若以元素为单位可以定义 mean-pooling 数学模型为式(2.21)：

$$a_{dij}^{l} = pooling(a^{l-1}) = (\sum_{i'=1}^{s}\sum_{j'=1}^{s} a_{d(i*s+i')(j*s+j')}^{l-1})/s\text{^}2 \qquad (2.21)$$

其中 s 代表池化视野的大小，同时，若a^{l-1}张量大小为 m，则$i,j = 1,\dots,m/s$；需要注意的是a^{l-1}为整个输入张量，$pool(a^{l-1})$会根据位置信息输出a_{dij}^{l}，因此，max-pooling 数学模型为式(2.23)：

$$a_{dij}^{l} = pooling(a^{l-1}) = max_{i',j'} a_{d(i*s+i')(j*s+j')}^{l-1} \qquad (2.22)$$

若隐含全部位置信息，可用矩阵法将两者直接表示为式(2.24)：

$$a^{l} = pooling(a^{l-1}) \qquad (2.23)$$

有些结构对 pooling 后的a^{l}做形如$y = wx + b$的变换，主要是因为加入一个偏置项以实现非线性转换，但没有明确证据这是关键因素，故此在这里不做数学模型定义，后续第三章实现也没有这一模块功能。

2.2.4 全连接层

全连接层在 CNN 中承担的责任在于将分散的特征进行合并，减少特征信息的丢失。它在 CNN 整个结构中可以理解为特殊卷积层，特殊在于两个方面，一个是 kernel 的尺寸即为输入张量的尺寸，一个是输出张量虽然是一维的，但也可以将其认定为一个 1×1 的二维矩阵。根据上述的特殊，可以将其数学模型定义为式(2.24)：

$$z_d^l = z_{d11}^l = \sum_{d',i',j'} w_{dd'i'j'}^l a_{d'i'j'}^{l-1} + b_d^l \qquad (2.24)$$

上式中z_d^l是一般表达，从全连接层开始不再有矩阵表达与元素表达之分，全连接层算出 z 中间变量后依然要使用激活函数，因此，有式(2.25)(2.26)：

$$a_d^l = \sigma(z_d^l) = a_{d11}^l = \sigma(z_{d11}^l) = \sigma(\sum_{d',i',j'} w_{dd'i'j'}^l a_{d'i'j'}^{l-1} + b_d^l) \qquad (2.25)$$

上式为卷积层经过卷积后变为全连接层的数学表达，而全连接层与全连接层直接相连的数学模型与式(2.25)相近，如式(2.26)：

$$a_d^l = \sigma(z_d^l) = a_{d11}^l = \sigma(z_{d11}^l) = \sigma(\sum_{d'} w_{dd'}^l a_{d'11}^{l-1} + b_d^l) \qquad (2.26)$$

而一般最后的输出层也是一种特殊形式的全连接层，只不过其激活函数可以根据具体业务需求选择第 2.2.2 节中所列的激活函数，譬如二分类的时候 Sigmoid 也可作为输出层的激活函数使用，或者没有激活函数也是可以的，但一般不这么做，也有一个专门的激活函数可供选择——softmax 函数，如式(2.27)。

$$a_d^l = \sigma(z_d^l) = a_{d11}^l = \sigma(z_{d11}^l) = \frac{e\hat{}z_{d11}^l}{\sum_d e\hat{}z_{d11}^l} \qquad (2.27)$$

上式将z_d^l的计算过程简略，以简洁表示。

2.2.5 损失函数

损失函数的作用是定量的评估 CNN 最后的输出值与真实值的偏差，以此判定 CNN 的有效性，并且为了后续的 BP 算法作为反推依据。以下主要介绍两种代表性损失函数，均方误差（Mean Square Error，MSE）以及交叉熵损失函数（Cross Entropy Error Function，CEE），两者应用场景不同，MSE 更适用于回归问题，CEE 更适用于分类问题，这是由它们的数学定义决定的。

假设现有 CNN 输出值 $a_1, a_2, \ldots a_n$ 所组成的向量 a，有真实值 $y_1, y_2, \ldots y_n$ 所组成的向

量 y，则 MSE 的数学定义为式(2.28)：

$$J(a)=\frac{1}{2}*\frac{1}{n}\sum_{i=1}^{n}(y_i-a_i) \tag{2.28}$$

可以看到 MSE 主要计算的是预测值和真实值之间的距离，a 向量里的每一项都影响着 $J(a)$最后的结果，式中的$\frac{1}{2}$可乘可不乘，乘$\frac{1}{2}$是为了后面 BP 运算二次方求导时可以抵消。

CEE 的损失函数定义为式(2.29)：

$$J(a)=-\sum_{i=1}^{n}y_i lna_i \tag{2.29}$$

交叉熵的损失函数非常清晰明了，指的一提它的原理，求熵是一个非常常见的度量模型所蕴含的信息多寡的方式，它起源于信息论，是信息学之父香农开创性的借用物理学上的熵概念来定量的描述信息的多寡，它应用的领域非常之广，比如城市交通中用熵值来衡量交通数学模型车辆分配情况的合理性[55]，熵值越大说明了蕴含的信息越多，其实也可以用其他函数形式来替代式(2.29)的表现形式，这里不展开描述，但熵确实是一个神奇且应用广泛的数学模型。再有，可以从式(2.29)中观察得知，如果这是一个分类 CNN 的话，y 向量必定有 0 值，因此并不是所有 CNN 的预测值 a 都可以参与 CEE 的计算，这就是其与 MSE 最大的不同，因此在 BP 更新参数时 CEE 对正确分类变量更敏感。

2.3 CNN 的 BP 算法

深度学习的深度指的是网络结构，而学习恰恰就是指 BP 算法，BP 算法让 CNN 得以应用，它的算法流程一般在算出损失函数后采用梯度下降法来更新网络结构中的参数，使得训练后的 CNN 比之未训练时的损失函数值更小。

2.3.1 从损失函数到全连接层

在 2.2.5 节中介绍了两种损失函数，这一节主要描述在 BP 算法这两种函数如何更新全连接层的系数 w 与偏执项 b。

先介绍 MSE 的 BP 过程。CNN 的最后几步都是全连接层，即使是输出层亦是如此，因此先明确数学模型，即为式(2.12)，而损失函数也用向量表示，同时结合式(2.12)，则将式(2.28)改写为式(2.30)：

$$J(\sigma(z),y)=J(W,b,x,y)=\frac{1}{2}||a^l-y||_2^2=\frac{1}{2}||\sigma(W^la^l+b^l)-y||_2^2 \quad (2.30)$$

在式(2.30)中特意写成欧几里得范数（L2 范数）以隐含 CNN 输出值与真实值的距离这一信息，同时不以元素而以矩阵法表示更加清晰明了，推理简单；有了式(2.30)，则求系数矩阵 W 与偏置向量 b 的导数如式(2.31)及式(2.32)。

$$\frac{\partial J(W,b,x,y)}{\partial W^l}=[(a^l-y)\odot\sigma'(z^l)](a^{l-1})^T \quad (2.31)$$

$$\frac{\partial J(W,b,x,y)}{\partial b^l}=(a^l-y)\odot\sigma'(z^l) \quad (2.32)$$

其中⊙表示 Hadamard 积，而且机器学习的资料里一般不会明确的显示出矩阵向量的求导布局方式，一般根据科研人员自行决定，但在此需要明确：进一步的明确矩阵大小，假设a^{l-1}为以 m×1 的向量，a^l为一 n×1 的向量，则式(2.31)中若为分子布局的雅克比 (Jacobian)矩阵的结果为一个 n×m 的矩阵，若为分母布局的梯度矩阵则与W^l的形式一致，但为了后续推导方便，在此依然使用雅克比矩阵，特此说明。同时，观察到式(2.31)与式(2.32)有一个共同项$(a^l-y)\odot\sigma'(z^l)$，它是中间依赖部分$\frac{\partial J(W,B,X,Y)}{\partial z^l}$的计算结果，因此把共同项提出来并定义为δ，同时应该注意到上面求的是激活函数 MSE 对全连接层的 W 与 b 的导数，因此将输出层定位 L，则δ^L为式(2.33)：

$$\delta^L=\frac{\partial J(W,b,x,y)}{\partial z^L}=(a^L-y)\odot\sigma'(z^L) \quad (2.33)$$

从而，可以进一步的根据链式法则求前面几层的全连接层与全连接层之间的导数，可用δ^l表示式(2.34)：

$$\delta^l=\frac{\partial J(W,b,x,y)}{\partial z^l}=(\frac{\partial z^L}{\partial z^{L-1}}\frac{\partial z^{L-1}}{\partial z^{L-2}}\cdots\frac{\partial z^{l+1}}{\partial z^l})^T\frac{\partial J(W,b,x,y)}{\partial z^L} \quad (2.34)$$

根据式(2.34)可以方便的求出W^l与b^l的导数分别为式(2.35)与式(2.36)：

$$\frac{\partial J(W,b,x,y)}{\partial W^l}=\delta^l(a^{l-1})^T \quad (2.35)$$

$$\frac{\partial J(W,b,x,y)}{\partial b^l}=\delta^l \quad (2.36)$$

因此求出δ^l是问题的关键。输出层的δ^L已经求出的情况下，这里运用数学归纳法，从δ^{L-1}开始都是全连接层之间的关系，进而假设第 l+1 层的δ^{l+1}也已经求出，根据式(2.34)可

求得式(2.37)：

$$\delta^l = \frac{\partial J(W,b,x,y)}{\partial z^l} = (\frac{\partial z^{l+1}}{\partial z^l})^T \frac{\partial J(W,b,x,y)}{\partial z^{l+1}} = (\frac{\partial z^{l+1}}{\partial z^l})^T \delta^{l+1} \quad (2.37)$$

从而关键点在于求$\frac{\partial z^{l+1}}{\partial z^l}$，而根据两者的关系，式(2.12)可改写为式(2.38)：

$$\mathrm{z}^{l+1} = W^{l+1}\sigma(z^l) + \mathrm{b}^{l+1} \quad (2.38)$$

将式(2.38)代入$\frac{\partial z^{l+1}}{\partial z^l}$可以求出式(2.39)：

$$\frac{\partial z^{l+1}}{\partial z^l} = W^{l+1} diag(\sigma'(z^l)) \quad (2.39)$$

进而将式(2.39)代入式(2.37)得式(2.40)：

$$\begin{aligned}\delta^l = (\frac{\partial z^{l+1}}{\partial z^l})^T \frac{\partial J(W,b,x,y)}{\partial z^{l+1}} &= diag\Big(\sigma'(z^l)\Big)(W^{l+1})^T \delta^{l+1} \\ &= (W^{l+1})^T \delta^{l+1} \odot \sigma'(z^l)\end{aligned} \quad (2.40)$$

式(2.40)即为从损失函数到任一全连接函数的 BP 算法过程，当然这个 CNN 结构中全连接层只承接卷积层，而不会出现卷积层在全连接层后面这类变体 CNN。

而回到最初的问题，CEE 的 BP 算法仍需明确，而从上述推导过程知道只需算出δ^L即可。在此应该首先明确 CEE 的应用场景主要为分类问题，其搭配 softmax 算出的总和为 1 的概率分布是相得益彰的，因此以下只根据 CEE 搭配 softmax 的情况做 BP 算法的数学模型定义。

softmax 函数在式(2.27)已给出定义，但其更加注重元素的计算过程，这里用矩阵法重新定义为式(2.41)：

$$a_i^L = \sigma(z_i^L) = \frac{e\text{^}z_i^L}{\sum_j e\text{^}z_j^L} \quad (2.41)$$

首先求$\delta^L = \frac{\partial J(a^L)}{\partial z_i^L}$，如式(2.42):

$$\frac{\partial J(a^L)}{\partial z_i^L} = \sum_j (\frac{\partial J(a_j^L)}{\partial a_j^L}\frac{\partial a_j^L}{\partial z_i^L}) \quad (2.42)$$

直接从式(2.42)开始推导不方便，因为 CEE 算出的$J(a^L)$与全部a^L有关，因此在累加里需要分情况，一个是 *j=i* 的情况，一个是 *j≠i* 的情况。首先算$\frac{\partial J(a_j^L)}{\partial a_j^L}$可有式(2.43)：

$$\frac{\partial J(a_j^L)}{\partial a_j^L}=\frac{\partial(y_j ln a_j^L)}{\partial a_j^L}=-y_j\frac{1}{a_j^L} \tag{2.43}$$

可以看到式(2.43)中没有变量与 *i* 项有关，因此不用分情况，但后面$\frac{\partial a_j^L}{\partial z_i^L}$这需要分情况，因为分子的$z_i^L$是固定的，不是随着不同项而变的 ，首先推导 *j=i* 的情况如式(2.44)：

$$\frac{\partial a_i^L}{\partial z_i^L}=\frac{\partial\frac{e^{z_i^L}}{\sum_j e^{z_j^L}}}{\partial z_i^L}=\frac{\sum_j e^{z_j^L}e^{z_i^L}-(e^{z_i^L})^2}{(\sum_j e^{z_j^L})^2}=\left(\frac{e^{z_i^L}}{\sum_j e^{z_j^L}}\right)\left(1-\frac{e^{z_i^L}}{\sum_j e^{z_j^L}}\right)=a_i^L(1-a_i^L) \tag{2.44}$$

当 $j\neq i$ 时，有式(2.45)：

$$\frac{\partial a_j^L}{\partial z_i^L}=\frac{\partial\frac{e^{z_j^L}}{\sum_j e^{z_j^L}}}{\partial z_i^L}=\frac{0\cdot\sum_j e^{z_j^L}-e^{z_i^L}e^{z_j^L}}{(\sum_j e^{z_j^L})^2}=-a_i^L a_j^L \tag{2.45}$$

将式(2.43)、式(2.44)与式(2.45)代入式(2.42)，得式(2.46)：

$$\begin{aligned}\delta_i^L=\frac{\partial J(a^L)}{\partial z_i^L}&=\sum_{j\neq i}\left(\frac{\partial J(a_j^L)}{\partial a_j^L}\frac{\partial a_j^L}{\partial z_i^L}\right)+\sum_{j=i}\left(\frac{\partial J(a_j^L)}{\partial a_j^L}\frac{\partial a_j^L}{\partial z_i^L}\right)\\&=\sum_{j\neq i}-y_j\frac{1}{a_j^L}\left(-a_i^L a_j^L\right)+\left(-y_i\frac{1}{a_i^L}\right)a_i^L(1-a_i^L)\\&=\sum_{j\neq i}a_i^L y_j^L+a_i^L y_i-y_i=a_i^L\sum_j y_j-y_i\end{aligned} \tag{2.46}$$

式(2.46)就是 CEE 与 softmax 结合后损失函数对z^L的求导求出了中间依赖部分δ^L。

至此，第三章所用的两个损失函数的 BP 算法推导至全连接层的数学模型定义完毕。

2.3.2 从卷积层到上一隐藏层

在 CNN 中，全连接层在 2.2.4 节分析过其实可以视为特殊的卷积层，因此在本节里直接探究从卷积层到上一隐藏层的 BP 算法计算导数的数学模型。

在 CNN 中依然如此，DNN 中的向量可以理解为 CNN 输出矩阵的 n×1 的特殊形式，BP 的时候则从向量求导到多维张量求导，因此从式(2.37)中可知在 DNN 中δ^l和δ^{l-1}的递推关系，因此在 CNN 中要获悉δ^l和δ^{l-1}的递推关系结合式(2.40)有式(2.47)：

$$\delta^{l-1}=\left(\frac{\partial z^l}{\partial z^{l-1}}\right)^T\delta^l=\delta^l * rot180(W^l)\odot\sigma'(z^{l-1}) \tag{2.47}$$

可以看到式(2.47)与式(2.40)类似，只在于卷积核被旋转了 180°，这使得卷积核上下

与左右各翻转一次，这个结果的不同就是因为矩阵运算时直接推演的参数过多，这里不详细推导，只需理解其原理并为第三章实现时提供参考即可。

因此损失函数对W^l张量与偏置向量b^{l-1}则为式(2.48)及式(2.49)：

$$\frac{\partial J(W^{l-1},b^{l-1})}{\partial W^{l-1}}=\delta^{l-1}(a^{l-2})^T \tag{2.48}$$

$$\frac{\partial J(W^{l-1},b^{l-1})}{\partial b^{l-1}}=\delta^{l} \tag{2.49}$$

与全连接层的表达一样，只是这里是多维张量。

2.3.3 从池化层到上一隐藏层

在 2.2.3 节，介绍了图 2-11 是 CNN 前向传播时的两种池化方式，池化以后的矩阵大小变化，不能直接求导，必须恢复到 *l*-1 层的矩阵大小，这个过程称为 upsample，池化矩阵先恢复前一矩阵的大小，然后根据池化类别做相应的变化，如图 2-12，unsample(a^l)的结果即为a^{l-1}，因此由图 2-12 可直接得$\frac{\partial J(W^{l-1},b^{l-1})}{\partial a^{l-1}}$，则$\delta^{l-1}$即为式(2.50)：

$$\delta^{l-1}=(\frac{\partial a^{l-1}}{\partial z^{l-1}})^T\frac{\partial J(W^{l-1},b^{l-1})}{\partial a^{l-1}}=\text{unsample}(\delta^l)\odot\sigma'(z^{l-1}) \tag{2.50}$$

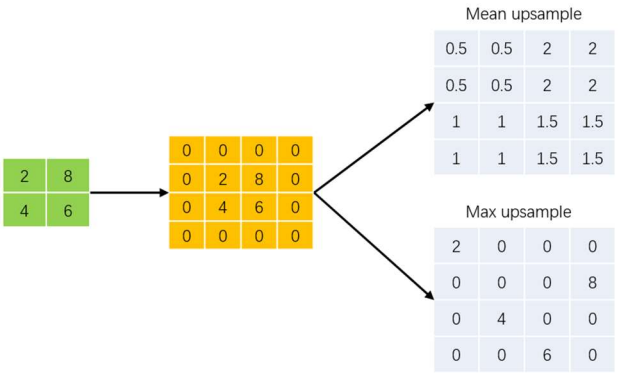


图 2-12 upsample(a^l)示意图

2.3.4 超参数介绍

本节介绍 CNN 进行 BP 训练学习时所能用到的常规的超参数，超参数的意思是相对 CNN 里面的W及b定义的，因为 BP 的时候就是更新这两个参数，而超参数即为不能学习更新，是人为定义的，而是为了调参让 BP 能发挥作用的参数叫超参数，并且这将

给第三章设置 API 时提供依据。

最重要的超参数可以说是学习率（步长，Learning Rate），它表明了每次训练时更新网络权重的大小，α即为更新率，其更新形式如下式(2.51)及式(2.52)：

$$\mathrm{W}^{l} = \mathrm{W}^{l} - \alpha\delta^{l}(a^{l-1})^{T} \qquad (2.51)$$

$$\mathrm{b}^{l} = \mathrm{b}^{l} - \alpha\delta^{l} \qquad (2.52)$$

迭代次数（Epoch）指的是训练集在被循环训练的次数，与之相关且需要区分的有两个重要超参数，一个是 Batch，一个是 Iteration，前者指的是并行运算的时候的训练样本数，后者指的是 Batch 循环的次数。三者所述不一定严谨，但逻辑确定。其他超参数还有激活函数的选择、整个 CNN 的层数、kernel 的尺寸以及 kernel 的组数等不经过 BP 训练的参数都属于可变参数。

2.4 并行计算框架概述

本节主要介绍 MapReduce 编程思想以及 Hadoop 的 MapReduce 组件的运行流程，最后粗略介绍 Java 内置库 util 包里的 ConcurrentHashMap 的组织架构和实现理念。

MapReduce 在 1.2.4 节中介绍了它其实是一种编程模型或编程思想，不应该等同于 Google 或者 Hadoop 的 MapReduce 分布式计算模块。MapReduce 的名字就是其精髓所在，它将可并行计算的任务通过键值对将其分为可独立运算的子任务分发到各运算节点中，这一过程即是 Map；运算完毕以后根据键值对归约回主任务，这就是 Reduce。Google 和 Hadoop 做的事情是将这一编程模型实现成 API 可供调用，让调用者专注于 Map 函数和 Reduce 函数，而 MapReduce 模块则负责保持节点间的通信与具体分配计算任务至节点等底层工作，这就让程序员更关注于数据的逻辑结构，而不用花精力到更底层的硬件运行与逻辑协调上。

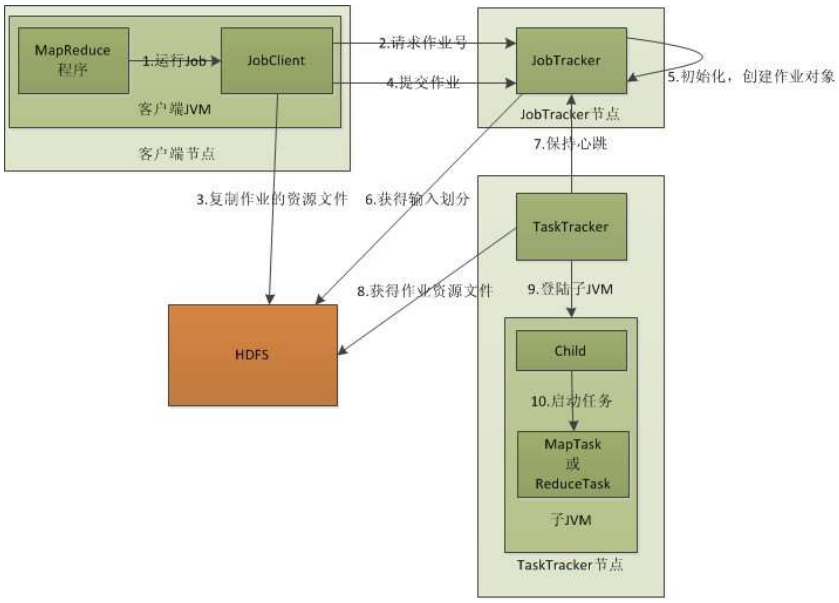


图 2-13 Hadoop MapReduce 1.X 运行流程图

图 2-13 [2]表示 Hadoop MapReduce 版本号为 1.X 的运行流程示意图，自从 2017 年 Hadoop 推出 2.X 版本后其节点间的逻辑有所改变，但 1.X 能多少获悉其庞大的、顾及鲁棒性的且复杂的 MapReduce（后文称 MR）流程。图中四个大的方块表示了 MR 运行时主要模块间的关系，HDFS 模块主要承载存放数据的任务，它让其他三个模块都通过它来读取数据；编写好 Map 函数和 Reduce 函数后就在客户端节点启动 JVM，它先把需要并行计算的数据存进 HDFS 内，并且通知 JobTracker（后文称 JT）节点有 MR 任务；JT 节点充当 MR 过程中的协调领导者，它接收任务后根据各 TaskTracker（后文称 TT）节点的 CPU 数量以及内存的大小来分发并行任务给各 TT 节点，并且在 TT 节点计算的时候时刻监听 TT 的运行状况；TT 节点接收 JT 节点分发的任务后从 HDFS 里获取需要计算的数据，在计算的同时也向 JT 汇报自己的运行状况，在计算完成后告知 JT 节点。以上只是 MR 的大体流程就可窥见其内部的复杂程度，各模块的通信协调，任务分发，保证安全健壮的措施都顾及到了。值得一提的是因为 MR 在设计之初假设的场景是大规模数据，即在运算时内存是必定饱和的，因此用 HDFS 来承载在 MR 流程中数据的读写，而且也保证了大规模数据的可读写性，因此 MR 中 JT 给 TT 发送的任务是不包括数据的，而是 JAR 包，这是大规模数据处理的核心思路，数据本地化（Data-Local）。但这也极大的造成了 I/O 时间的浪费，因此针对这一情况出现了 Spark，这也是一种 MR 编程模型的实现，与图 2-13 的运行逻辑类似，与 MR 大体内部结构相似，只是 HDFS

[2] 图片来源于 https://blog.csdn.net/yangwenxue_admin/article/details/46547519

模块变为了内存，Spark 在科研中运用的比 Hadoop MR 更多，因为科研数据量的规模更适合 Spark 的运行场景。

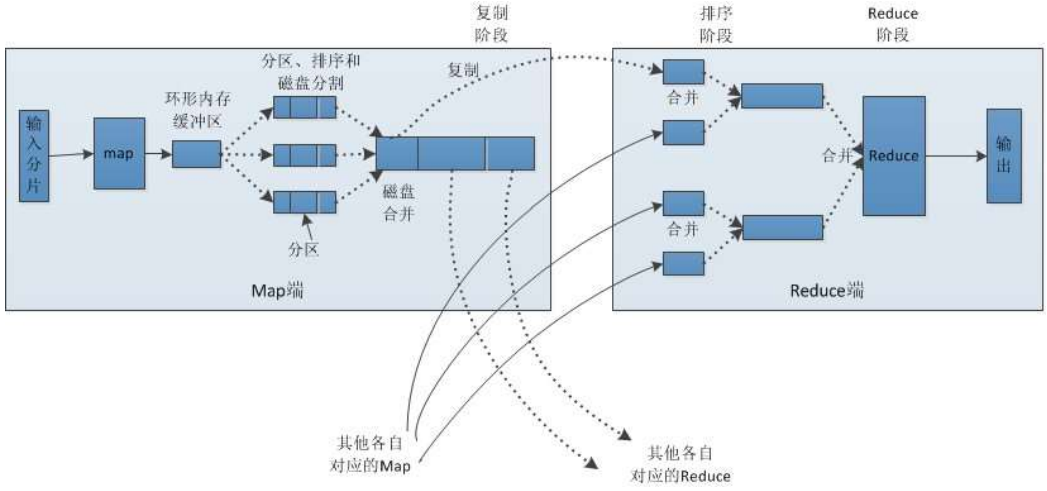


图 2-14 Map 及 Reduce 流程示意图

而具体的 Map 及 Reduce 如何组织数据则如图 2-14[3]所示，图中最大的信息是每一个 Map 端在进行完 Map 任务后会将结果合并，然后再根据键的信息分配到不同的 Reduce 端，而 Map 端与 Reduce 端则是 TT 节点不同时期的不同角色，不过主要是能直观的学习获知其内部流程，以之为构建本研究的并行计算逻辑所参考。

MR 编程范式给并行计算提供了解决方案，而 ConcurrentHashMap[56]（后文称 CHM）则提供了并行计算数据时数据同步问题的解决思路。CHM 是并发编程大师、对 Java 影响最大的人——Doug Lea 亲自操刀为 Java 开发设计的并发编程容器，是 Java 的内置类。CHM 主要是为了针对在并发编程中的两个容器 HashMap 和 HashTable 所存在的问题而设计；HashMap 在并发环境中进行 put 操作创建元素时可能会造成环形链表，如果后面进行 get 操作则会引起死循环，导致线程不安全问题；而 HashTable 虽然是线程安全，但因为它是使用 synchronized 关键字来保证线程安全，换言之一个 HashTable 只有一个锁，因此同一时间多线程进行数据同步时会造成线程间的竞争，导致连 get 这种不需数据同步的方法都会进入阻塞或轮询状态，从而效率大幅降低。CHM 就应运而生了，它的结构示意图如图 2-15，CHM 最大的特点在于采用了分段锁技术，不在同一段的数据互相不影响，多个线程对多个不同段的操作是不互相影响的，因为每个段使用一把锁，这使得并发访问率有效提升。CHM 的多锁秘诀在于它是由一个 Segment 数组和多个 HashEntry 数组组成；Segement 是一种可重入锁，Segement 的结构和 Hashmap 类似，都

[3] 图片来源于 https://blog.csdn.net/yangwenxue_admin/article/details/46547519

是一种数组和链表的结构。Segment 数组里的每个 Segment 守护着一个 HashEntry 数组，每个 HashEntry 数组是一个链表结构的元素，里面存储着键值对，如果要对 HashEntry 数组里的元素进行修改，必须先获得它对应的 Segment 锁。

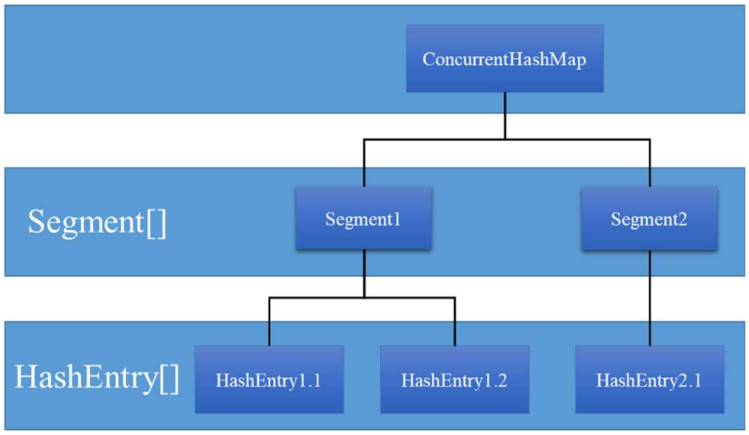


图 2-15 ConcurrentHashMap 结构图

综上，Hadoop 的 MR 是一个针对业务逻辑庞大的处理模块，它的主要运行场景是在大规模数据上，因此是不适合本研究的，而且其本身“体型”就太过庞大，如果要部署 Hadoop 每一台电脑都需要装载它，不能做到“即插即用”的轻量级应用，而且其中的大多数功能在本类库里是不需要的，因此建立在其上是完全没必要的。但根据其 Map 和 Reduce 的流程可以借鉴其在并行计算时抽象的独立出自计算单位以及计算流随着 MR 的进行而出生消亡的架构。在 CHM 中，可以看到其在并发编程环境解决数据同步效率问题是将数据分隔开的思想。

2.5 socket 技术概述

TCP/IP 协议（Transmission Control Protocol/Internet Protocol）是指在多个不同网络间信息传输的协议族，它的封装和应用就是 socket。因此 socket 本身并不是某种协议，而是一门技术，或者对程序员来说它是对 TCP/IP 协议的抽象，从而形成了一个 API（调用接口）可供调用。这里不对诸如 TCP 的三次握手或四次挥手这些 TCP/IP 协议与 socket 如何封装这些协议进行详解，主要对 socket 的 API 应用逻辑进行概述。

socket 的中文翻译为“套接字”，主要指 IP 地址与端口的“套接”使得 A 进程可以在网络间找到指定的 B 进程，socket 不止可以运用在类似 QQ 聊天窗口这类功能上，在分布式系统中依然是一项关键技术[57]。socket 的通信流程如图 2-16 所示，可以看到 socket 的通信的两个进程是分 C/S 模式的，server 进程创建了 socket 以后需要监听绑定

的端口号以后，client 进程的 socket 才能与之建立连接并且通信。socket 在 Java 的 net 库中已经提供了完整的接口，在本研究中 socket 承担了分布式计算时的数据通信责任。

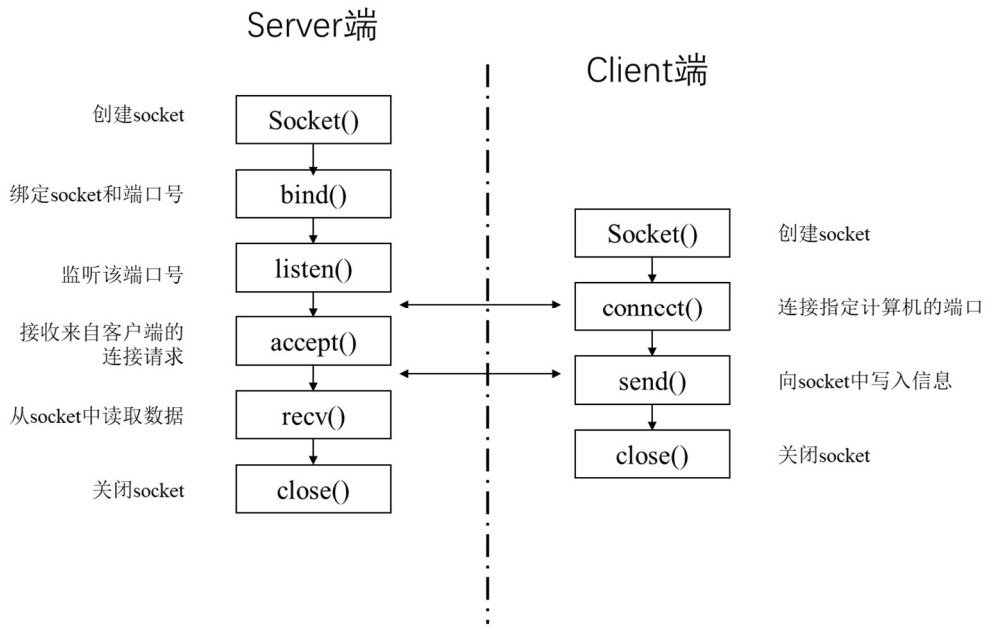


图 2-16 socket 通信流程

2.6 本章小结

这一章重点讲述了深度学习的起源以及数学模型的定义以及它的应用场景、CNN 的前向传播、BP 算法的实现与并行编程的理念以及技术。CNN 的前向传播流程给出了每一步的数学模型，重点是在矩阵法表述的同时给出了明确的每一个元素的变化过程，这为第三章的 CNN 前向传播算法的 Java 实现提供了理论基础。在 BP 算法中给出了具体的推导过程，需要提出的是这是矩阵法的表示，并不是元素法的表述，但却是 BP 的理论基础，并且逻辑清晰明了使人明晰算法结构与流程，在第三章中的 BP 实现是以元素法为数学基础来 Java 实现的，这是需要明确的。本章还介绍了 MapReduce 编程思想以及 Hadoop MapReduce 的流程，这为第四章的并行计算提供了计算流的组织架构思路，ConcurrentHashMap 则为第四章的并行计算时为解决数据的同步问题提供了思路。最后还大致概述了多机通信的 socket 技术与运用其 API 的逻辑。

第3章 基于 Java 的 CNN 实现与测试

本章基于第 2 章的理论来实现 CNN，尤其是 2.2 节的 CNN 前向传播数学模型定义以及 2.3 节 BP 算法的逻辑运算与各类推导，本章的主要任务就是结合第二节的数学模型将其以 Java 实现，且在运行逻辑上不用第三方库的目标来做，以达到“定制化”、轻量级与可拓展性的目的。

3.1 CNN 前向传播的 Java 实现

3.1.1 准备工作

工欲善其事必先利其器，本节主要描述了两个方面，一个是介绍实现的基础环境，一个是在实现 CNN 前向传播时所应考虑的细节。

本研究所用 jdk 版本号为 1.8，并未使用更新的版本，因为 jdk1.8 是现今开发人员所用的版本，以此为基础编写的代码移植性相对较好，不用考虑版本容错问题，同时本研究所写的库并未用太多很新潮的技术，jdk1.8 的内置库已经够用。IDE 为 IntelliJ IDEA，这是一款受到 Java 程序员广泛赞赏的 IDE，同时采用 maven 作为项目管理工具，它让程序员更好的管理第三方包而且能方便的调用，虽然本类库尽量不用第三方包，但还是依然使用了 gdal 来读取遥感数据，用序列化在分布式计算时承载传输的数据。gdal 的选取主要顾及两方面：1.Java 没有内置库或方法去直接读 tiff 数据，即使是一些第三方库也是用 Jni 来去调用 C++来读 tiff，说明 tiff 本来就不能直接在 Java 中打开 2.遥感影像 tiff 蕴含位置信息，原则上是不能损失的。而为了将本类库能更加的针对遥感从业人员，库中也实现了诸如读取遥感 tiff 数据信息的 API、指定大小切割 tiff 的 API 以及将 tiff 转换为 csv 文件的 API，这可以在 Python 方便的读取 csv 文件中的遥感多通道数据，从而能简便的使用 Keras 或 Tensorflow 而不用更改配置文件或使用第三方库。

第二个准备工作涉及实现部分。本研究中的所有浮点数都用 float 型，因为其是 double 型所占内存的一半，而且经过实测运算速度有所增加。所有矩阵都采用 Array 来存储数据，在创建时就确定了其使用大小，而不用 List 拖慢效率，因此，在存储每一层

的输出张量时为一个三维数组，float[][][]。由图 2-7 CNN 卷积过程示意图中可以看到 CNN 在卷积的时候是输入多维张量与卷积核（kernel）的乘积，kernel 这一关键参数在不同的深度学习库有不同的组织方式，本类库中的 kernel 作为一个对象存在，但它只会在层对象中被实现。kernel 在本类库中被命名为 Array_Kernel，图 3-1 列出了重要函数及变量，其中橙色框表示函数，黄色框表示变量；第一个构造函数接收一个三维数组然后直接将其填入 kernel 容器中，后两个构造函数是随机初始化时的构造函数，填入信息以此为本层通道数，上层通道数以及 kernel 大小，为了鲁棒性，设置了两种构造函数；实例变量 AK 即为 array of kernel 的意思，其为 kernel 参数的实际容器，也即式(2.13)的所有$w^{l}_{add'i'j'}$都存储在其中，AK 的第一维是所属层的通道数，第二维是上一层的通道数，第三维第四维则为 kernel 的行数和列数。

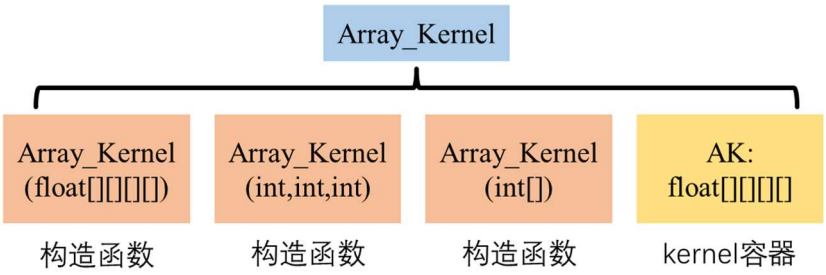


图 3-1 Array_Kernel 结构

3.1.2 融合了 gdal 的数据输入层模块

输入层在本类库里被命名为 InputLayer 类，其结构如图 3-2，实例变量 mat 为储存本层 Feature Map 的容器，三维数组从第一维到第三维分别为通道数，行数，列数，存储了读取的影像数据的像元值。dotsize()方法是将 mat 的值域控制在[0,1]中间，将 mat 除以 tiff 每个像元的位即可，本例中的像元大小为 2 字节、十进制为 512。

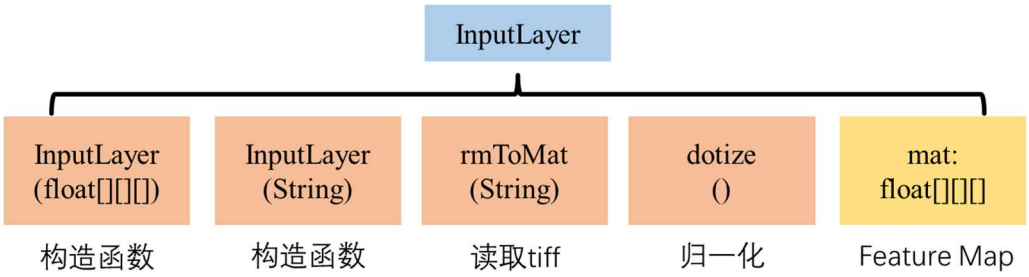


图 3-2 InputLayer 结构

读取 tiff 遥感影像的函数是 rmToMat，其实现流程如图 3-3，输入遥感 tiff 所在硬盘位置，输出三维数组，它的运行流程是先使用 gdal 打开目标文件，然后根据文件的波段

与高宽的信息循环读取像素值并将其存储进三维数组。当然 InputLayer 也可以直接接收三维 float[][][]作为类里的 mat。

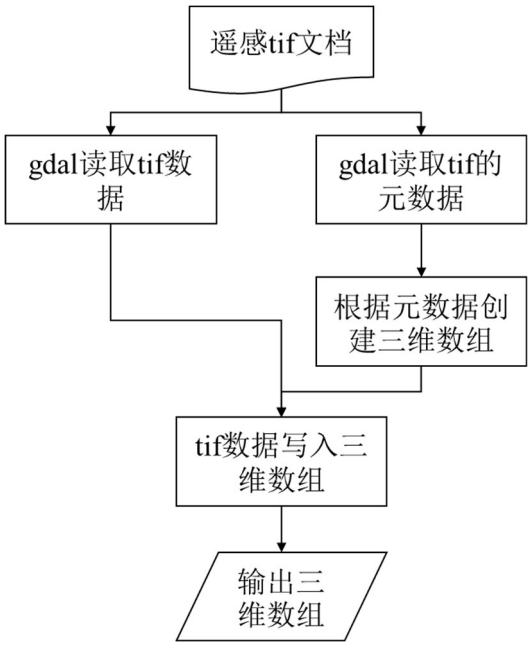


图 3-3 读取 tiff 数据流程图

3.1.3 卷积层模块

在本类库里，卷积层的实现为 HiddenLayer 类，命名不是特别规范，但保证基本功能，同时如有需要也可更改其类名，其类结构如图 3-4，可以看到同时里面的构造函数是为了鲁棒性而写的重载方法，因为卷积层的上一层可以有输入层、卷积层或池化层，因此参数列表用(…)表示；实例变量里依然有 Fature Map，储存了本层的图像，Array_Kernel 是一个类，但为了表述清楚写成一个四维数组，其即为 Array_Kernel 类，bias 以一个一维数组保存。

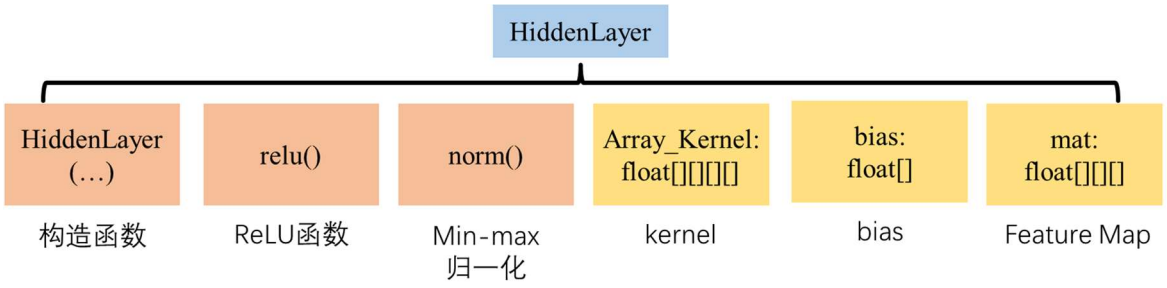


图 3-4 HiddenLayer 结构

卷积层的主要运算是式(2.13)卷积运算与激活函数运算。针对卷积运算，

HiddenLayer 的运行逻辑是在实例化类时就已经进行，即在构造函数里实现式(2.13)。同时式(2.13)已经用元素法清晰的定义了卷积运算的过程，图 3-5 即为卷积流程，图中的预设处理包含了位置判断流程，这个流程以输入的两组数据的位置判断经过卷积运算后的数据输出到新的三维数组的位置，具体实现可在 GitHub 源码中查看。

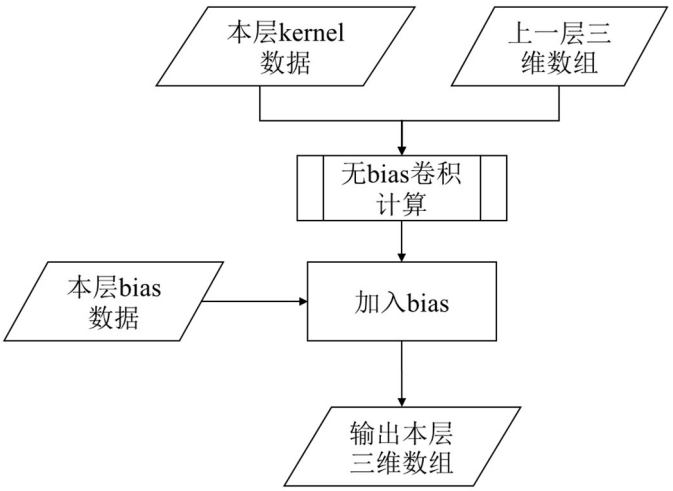


图 3-5 卷积运算流程图

而激活函数在本类库里除了输出层有 softmax 与 ReLU 外，其它层只有最广泛的 ReLU 函数，根据式(2.18)其实现流程如图 3-6。

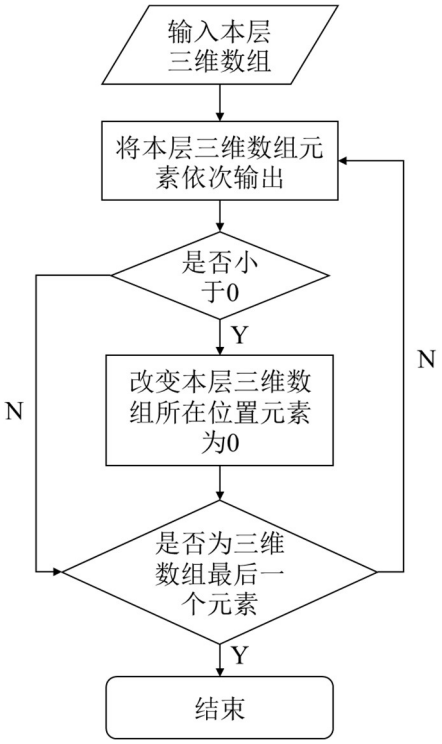


图 3-6 ReLU 函数流程图

其中还有一个 norm 函数，这是一个 min-max 标准化（Min-max normalization）函数，是 mat 里的每一通道的矩阵在调用后进行 min-max 标准化，目的是为了元素值域在[0,1]

内其数学定义为式(3.1)，代码实现流程图为图 3-7，其中的预设处理流程为先遍历矩阵的全部元素找出最大最小值。

$$x' = \frac{x - min}{max - min} \tag{3.1}$$

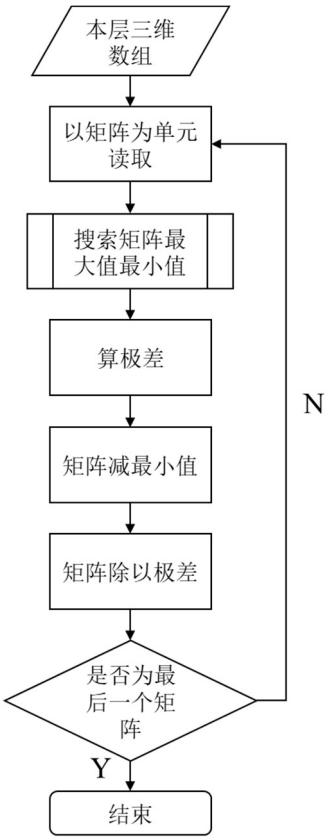


图 3-7 min-max 标准化流程图

3.1.4 池化层模块

在本类库里，池化层模块被命名为 PoolMat 类，其结构如图 3-8，与输入层隐藏层的实例变量类似，其池化流程也是在构造函数时进行，根据构造函数接受所需池化的 HiddenLayer，池化视野的大小以及池化种类，根据池化种类调用 pm_max 与 pm_mean 函数分别进行 max-pooling 与 mean-pooling 的操作。因为在式(2.21)与式(2.22)中已经以元素法表示，因此直接根据数学模型定义实现 mean-pooling 与 maxpooling，分别如图 3-9 及图 3-10。在流程图中，以池化视野为范围来进行描述，因为在整个矩阵中每个池化视野所做的动作都是一样的，因此只讨论池化视野内的流程即可。流程图清晰明了，在 mean-pooling 时最后输出池化视野内元素的均值，在 max-pooling 时最后输出池化视

野的最大值。

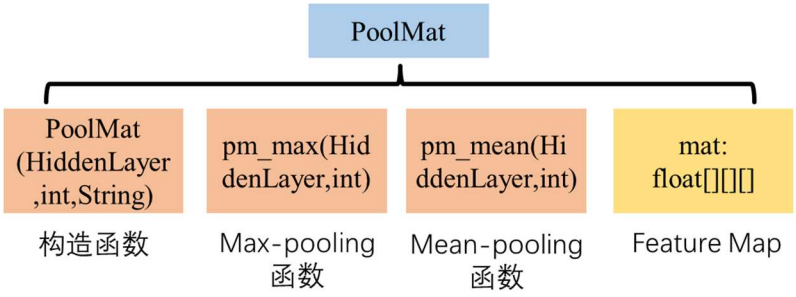


图 3-8 PoolMat 结构

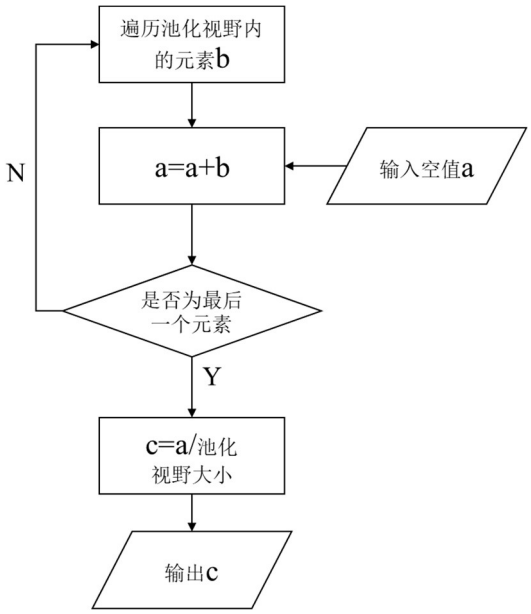


图 3-9 mean-pooling 流程图

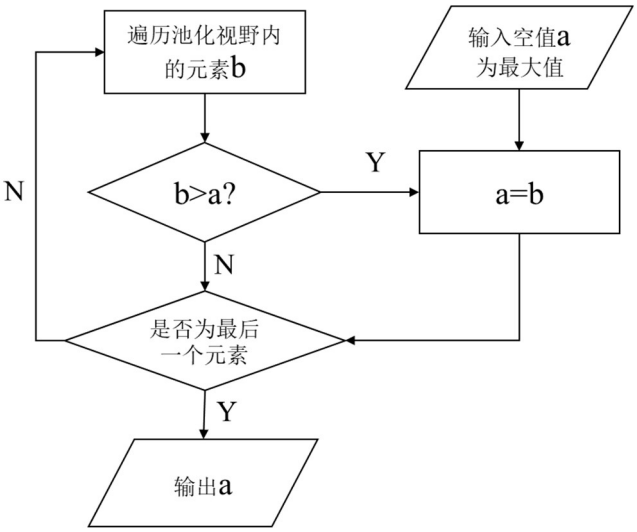


图 3-10 max-pooling 流程图

3.1.5 全连接层模块

在本类库里，全连接层模块被命名为 FC 类，其结构如图 3-11 所示，它的 Feature Map 有所改变，不再以三维数组作为容器，而是以一维数组 vector 来存储元素。并且，CNN 一般要连接全连接层的时候都是先经过池化，因此 FC 类构造函数只承接 PoolMat 类与 FC 类，根据式(2.24)可有池化层到全连接层的卷积过程，在此过程中 kernel 的行列数即为输入层 Feature Map 的行列数（全连接层的行列数设为 1），因此相比卷积层的卷积计算，输出从三维数组变为了二维数组，或者可以认为是一个最后一维只有 1 个元素的三维数组，因此在实现时同样以图 3-5 为流程，激活函数 ReLU 及 max-min 标准化也同样与图 3-6 及图 3-7 相同。

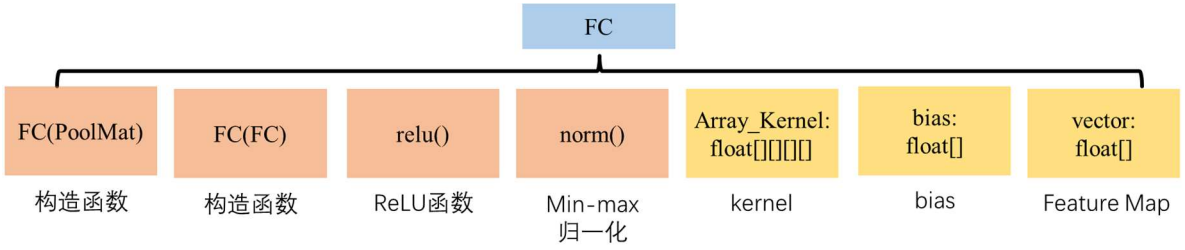


图 3-11 FC 结构

3.1.6 输出层模块

在本类库里，全连接层模块被命名为 OutPutLayer 类，它是继承 FC 类的，因此大体没有区别，其结构如图 3-12 所示。主要区别在于其中的两个函数，一个是 softmax()，一个是 percentify()。softmax 函数即是严格根据式(2.27)实现的。而其中还有一个 percentify，其所依据的式子为式(3.2)，它表达了数组中的每一个元素在整个数组中所占的比例，它是一个逻辑简单的输出函数，它在进行后续的 Forward-BP 时只需乘以一个倍数即可，便于实现。

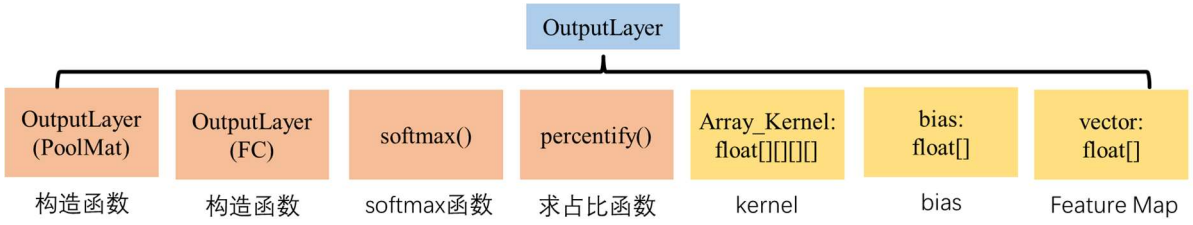


图 3-12 OutPutLayer 结构

$$x_i' = \frac{x_i}{\sum_j x_j} \tag{3.2}$$

3.2 CNN 的反向传播网络模型设计

3.2.1 Forward-BP 设计思路

在 2.3 节中，用严谨的数学推导确定了 CNN 的 BP 算法，但这主要是为了明晰 BP 的工作流程及求导过程，因为在 2.3 节中用的都是矩阵法表示，而在本类库里是使用多维数据来存储数据和进行数据的运算，因此虽然可以使用第三方包来对矩阵进行运算，但本类库的设计理念就是能尽量少用第三方库就不用，而且用自己的代码实现也有两个好处，一个是能非常明确的将数学模型以代码的方式实现，完全把控运行逻辑，一个是在要进行算法升级和调整的时候可以直接改写，而不用担心第三方库无法支持的情况。但自己实现有一个很大的问题即是 CNN 的 BP 过程都是矩阵运算，应该如何的进行代码实现呢。这里应该首先明晰 BP 的理论基础，它使用的是梯度下降法，针对复合函数的求导使用的是链式法则，在 2.3.1 节中定义的中间依赖部分δ即为链式法则的“链”，它让求导能从损失函数传递到所求层的δ，从而求出损失函数对 kernel 和 bias 的导数。而“链”的本质是导数，是每一函数的导数，导数即为函数值的变化幅度。如果是复合函数，最外层的函数对最内层的函数某个变量求导是使用链式法则，如式(3.3)列出了三层复合函数的求导链式法则，但换个角度来想，复合函数求导是获悉复合函数里某个变量对整个复合函数的变化影响，或称变化梯度，那如果从一开始记录这个变量的变化直到复合函数最外层的结果也是可以的，如式(3.4)，这貌似与式(3.3)没有变化，但却是不同的求导思路，为什么式(3.4)不是主流的原因在于它要在 x 的每个输出值在进入新的函数时就记录一次，这不是普世的，但在 BP 中却恰恰有其存在意义，因为为了计算梯度，CNN 在前向传播时是要记录参数位置与结果位置的关系，因为即使是矩阵运算求得了δ，也要进行位置判断才能更新参数。虽然可以根据矩阵的布局额外设置算法来推算连接关系，但在此只与传统的连接作对比。

$$(f\left(g(z(x))\right))'_x = \frac{df\left(g(z(x))\right)}{dx} = f'(g(z(x))g'(z(x))z'(x)$$

$$= \frac{df\left(g(z(x))\right)}{dg(z(x))}\frac{dg(z(x))}{dz(x)}\frac{dz(x)}{dx} \quad (3.3)$$

$$(f\left(g(z(x))\right))'_x = z'(x)g'(z(x))f'(g(z(x))) \quad (3.4)$$

综上，作者根据式(3.4)，结合 BP 的逻辑过程以及特定的适用环境，提出了 Forward-BP 算法，它求导的对象是以元素而不是矩阵为单位，并且取代 CNN 中记录连接关系的容器（在本类库里的前向 CNN 并没有实现这一容器），而出现一个新的 Forward-BP 来记录每个参数对后续层的影响。而这个容器就如 Array_Kernel 与 bias 一样存储在除了输入层的每个层中，它在本类库里被命名为 bpmat 与 bpbias，bpmat 是存储 kernel 的导数的，是 7 维数组，bpbias 是存储 bias 导数的，是 4 维数组，bpmat 命名中的 mat 取自 kernel 也是高维矩阵，但还是不那么规范。bpmat 的 7 维数组是因为在 Array_Kernel 中存储 kernel 值是 4 维数组加上层里的存储元素的 3 维数组，bpmat 的前四项与 Array_Kernel 的 AK 的结构对应，即这一层通道数、上一层通道数、行、高，后三项与每一层的存储结构对应，即通道数、行、高，bpbias 也是如此，4 维的第 1 维即为 bias 的存储结构。而在全连接层与输出层，其形式各减少了两个维度，bpmat 变为 5 维数组，bpbias 变为 2 维数组，因为它们本身所在的全连接层的 Feature Map 的维度就比卷积层及池化层少 2 维，所以存储导数容器也少 2 维。

除了设置 bpmat 以及 bpbias 这样的存储导数容器，还应有存储整个 CNN 的存储容器，它们是以 HashMap 来作为存储 kernel 和 bias 导数的容器，key 为它们的卷积层或全连接层（输出层属于全连接层），value 则是每一层的 bpmat 与 bpbias。

Forward-BP 与普通的前向传播最大的不同之处在于任何一个涉及计算的过程都要重写，以同步更新导数容器，以下进行详细描述。

3.2.2 Forward-BP 卷积层模块

由 3.2.1 确定了 Forward-BP 的定位和数学理论基础，本节开始介绍在不同层 Forward-BP 是如何具体实现的。本节讲述了 Forward-BP 如何在卷积计算及激活函数时的实现方式，实现还是以 bpmat 与 bpbias 两个容器为基础来设计。

Forward-BP 卷积层模块在本类库里被命名为 bpHiddenLayer，其结构如图 3-13，它

继承于 HiddenLayer 类的，因为在大的框架里它还是前向传播的，只是在细节中会加入记录导数的功能，比如每一层都有一个 bpmat 与 bpbias，而这些细节的不同直接体现在 HiddenLayer 的卷积函数与激活函数的运算过程中。

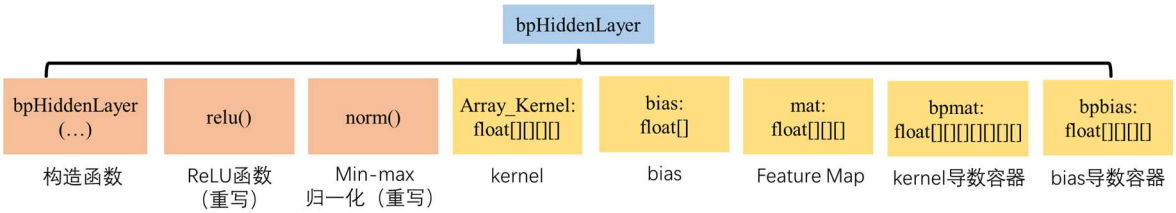


图 3-13 bpHiddenLayer 结构

虽然 bpHiddenLayer 是继承自 HiddenLayer 的，但不能直接用后者的构造函数 super，因为 bpHiddenLayer 多了 bpmat 与 bpbias，因此其卷积运算流程是与图 3-5 不同的，具体流程如如图 3-14，其中涉及本层 bpmat 与 bpbias 的初始化以及前面层的 bpmat 与 bpbias 的更新。bpmat 的元素的初始化是每个元素都有着当前层 mat 的结构，但只储存前一层 mat 中与其运算的元素，数学基础源于式(2.48)，而从式(2.49)中也能看出 z 对本层 b 的导数为 1，因此记录为 1，初始化在构造函数里进行，也即卷积运算之前。而卷积过程中，计算的时候就将前面所输入的导数容器里的元素一并计算了，因为导数容器随着神经网络的运算进行，它也同样要进行更新以最后得出对损失函数的影响，也即梯度。

$$z(w,b)'_w = (wx+b)'_w = x \tag{3.5}$$

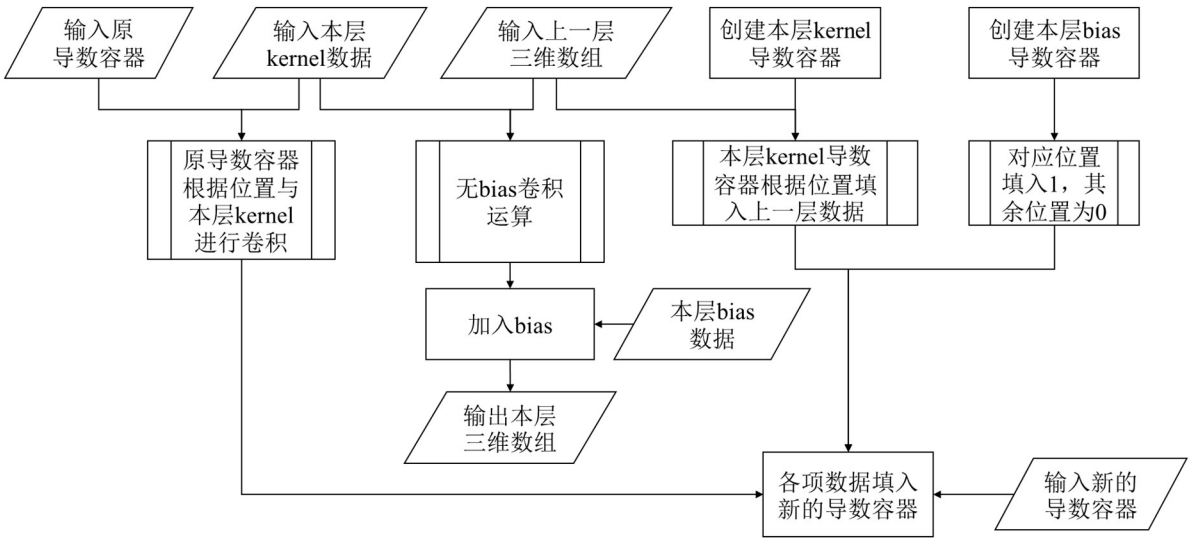


图 3-14 Forward-BP 卷积流程图

在 Forward-BP 中的 ReLU 与传统前向传播的流程不同之处很小，如图 3-15 对比图 3-6，仅仅在三维数组的元素进行最大值 0 的判断时同步更新导数容器里相应位置的元

素，因为导数容器的后三维与三维数组的位置相对应，因此能很好的将其实现。

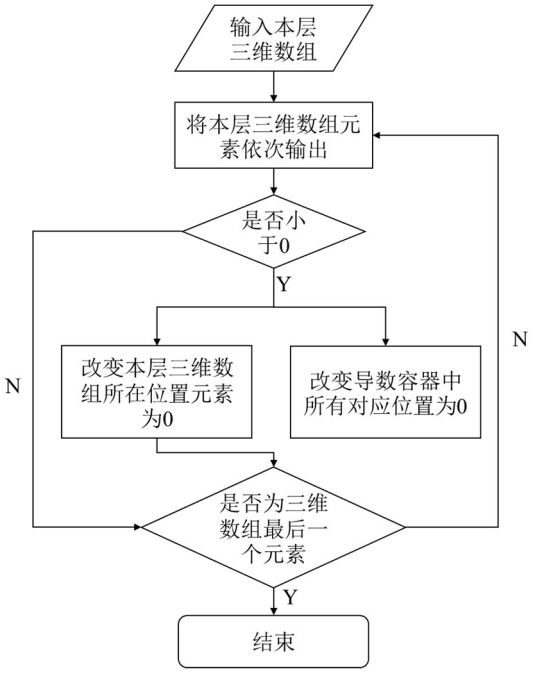


图 3-15 Forward-BP 卷积层 ReLU 流程图

3.2.3 Forward-BP 池化层模块

Forward-BP 的实现池化是在 bpPoolMat 里，其结构如图 3-16，它的父类是 PoolMat，因此为了实现导数的前向传播，重写 pm_max 方法与 pm-mean 方法，让 Forward-BP 内置进池化过程中。

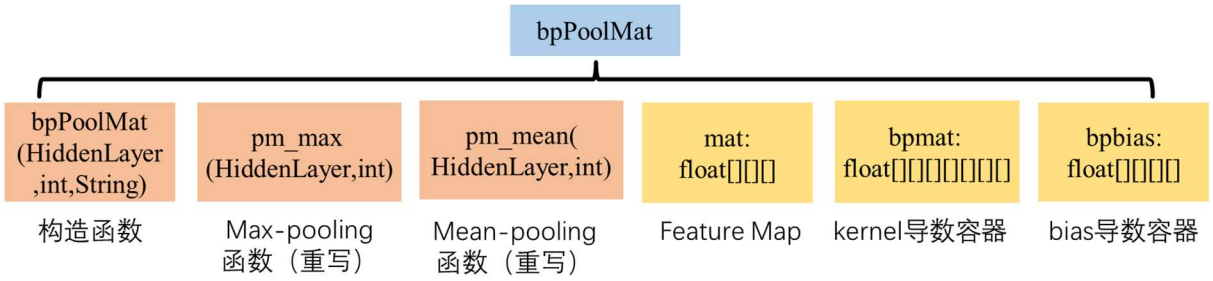


图 3-16 bpPoolMat 结构

如图 3-17 为 Forward-BP 中的 mean-pooling 工作流程，它与普通前向传播时的 mean-pooling 逻辑完全相同，对导数容器内相应的池化视野范围内的元素做累加后进行平均，虽然在流程图中以平行的逻辑关系展现，但在具体实现时是合二为一的，是一起计算的。图 3-18 为 max-pooling 在 Forward-BP 的工作流程，它还是依托前向传播的逻辑，对比图 3-10 只是在最后根据 a 所在的位置将对应的原导数容器内的值填入新容器即可。

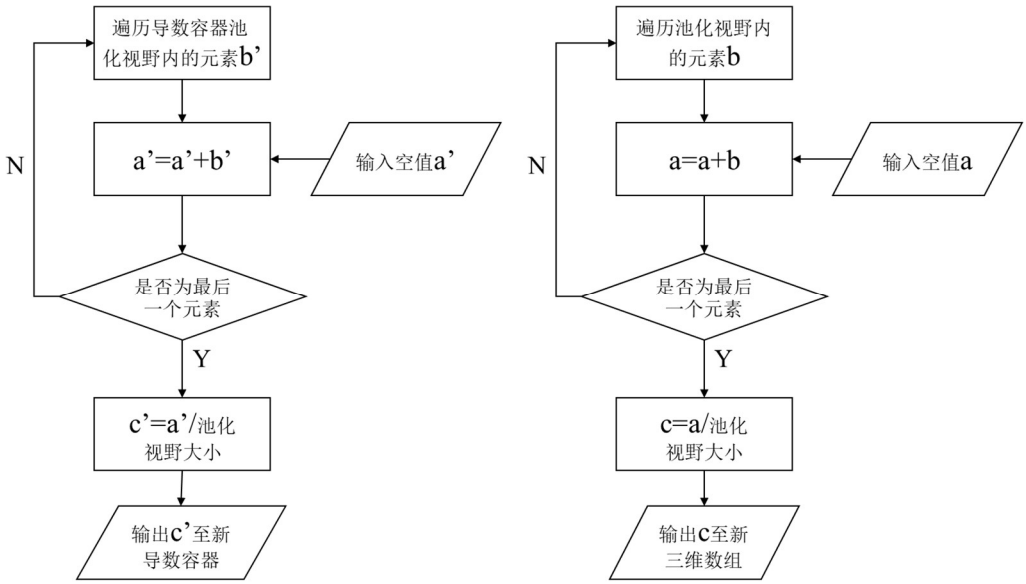


图 3-17 Foward-BP 的 mean-pooling 流程图

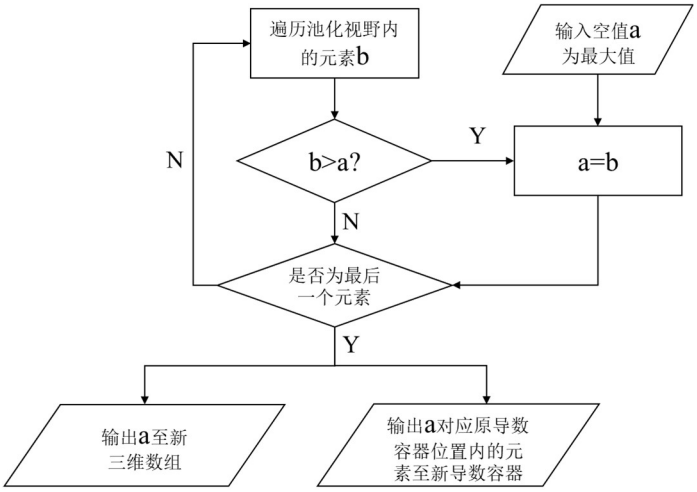


图 3-18 Foward-BP 的 max-pooling 流程图

3.2.4 Forward-BP 全连接层模块

Forward -BP 全连接层模块在本类库中被命名为 bpFC 类，结构如图 3-19，它是 FC 类的子类，它的卷积与 ReLU 函数的运行与 bpHiddenLayer 中的相应动作实现代码类似，即为图 3-14，因此不单独列出各项计算的流程图。值得一提的是因为其从三维数组变为了一维向量，因此其 bpmat 与 bpbias 的后三维变为了一维，bpmat 变为 float[][][][][]五维数组与 bpbias 变为 float[][]二维数组。而同时，它属于 FC 类的子类，所以应该对其构造函数以及 relu 函数与 norm 函数都进行重写。

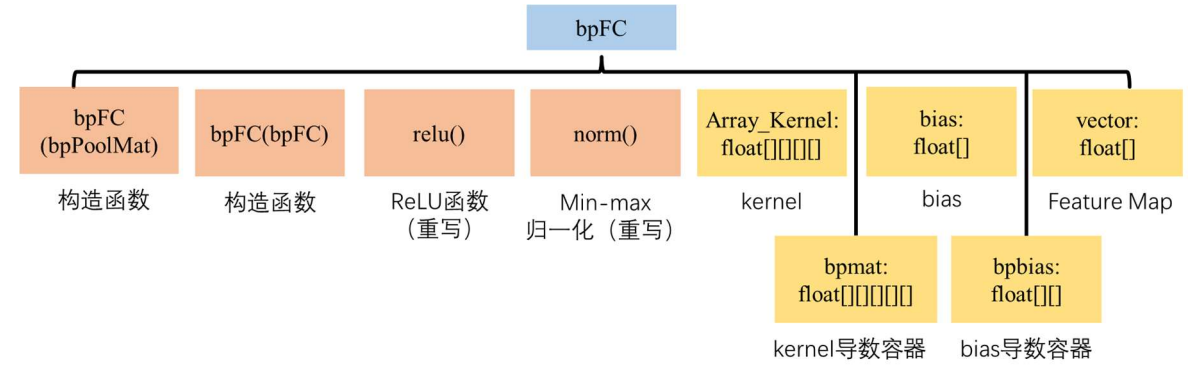


图 3-19 bpFC 结构

在本类库中因为输出层与全连接层的关系非常密切，因此将输出层设为全连接层的子类，其结构如图 3-20，它只增加了在 3.2.4 中所提到的 percentify 与 softmax 两个激活函数，它们的作用都是让一维向量 vector 中的元素和为 1，以概率的形式表达，在 Forward-BP 中 percentify 进行导数容器的更新，但 softmax 却不用，因为在本类库中它只与 CEE 搭配使用，所以有其特殊式，具体如下节。

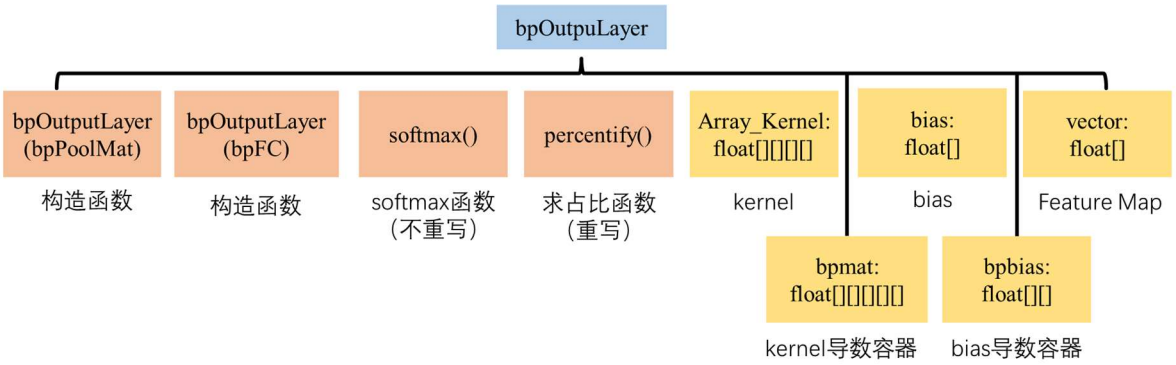


图 3-20 bpOutputLayer 结构

3.2.5 Forward-BP 更新参数模块

Forward-BP 在更新参数时不必再与传统 BP 一样要计算参数，因为在走完 CNN 整个流程并计算完损失函数以后，各参数对损失函数值的变化，也就是梯度已经算出，随后就可直接进行参数更新，这最后的模块被称为 Forward -BP 更新参数模块，在本类库里被命名为 bpFinal，其结构如图 3-21。

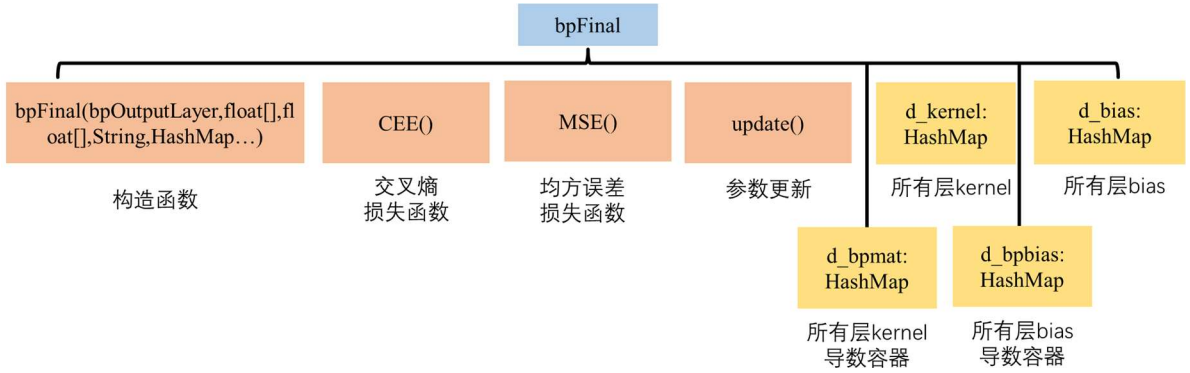


图 3-21 bpFinal 结构

bpFinal 在实例化的时候就要指定其损失函数的种类，在本类库中有 MSE 均方误差与 CEE 交叉熵两种损失函数可选，其中 CEE 只与 softmax 捆绑；MSE 按式(2.28)实现，因为这是一次计算过程，因此要 Forward-BP 要进行更新导数容器，则以元素法表示为式(3.6)，因为式(2.28)求导后即为一元一次函数，同样的，对于 CEE 来说，因为其只与 softmax 捆绑，所以有其特殊式(2.46)，所以在输出层进行 softmax 的时候不进行导数容器的更新，而在进行到损失函数时一并更新，同时，如果一副图像只给一个类别，则可将式(2.46)进一步变为式(3.7)，这表明了更新导数时只需判断本类别的真实值是否为 1，如果为 1 则减去 1 即可，如果不是则保持原值，这就是 CEE 与 softmax 配合进行分类时体现在算法上的优势。

$$a_i = y_i - a_i \tag{3.6}$$

$$\delta_i^L = \begin{cases} a_i^L - 1 & y_i = 1 \\ a_i^L & y_i \neq 1 \end{cases} \tag{3.7}$$

最后，在已经确定了学习率α之后，用于更新所实现的函数为 update 函数，这个函数严格按照式(2.51)及式(2.52)的数学表达来实现，因为在最后，导数容器已经完成了它的最后任务，bpmat 也从五维数组变为了四维数组，bpbias 则从二维数组变为一维数组，与参数所在位置一一对应，因此即可直接乘以学习率进行参数的更新。

至此，Forward-BP 的运行流程与根据数学模型定义所实现的代码介绍完毕。

3.3 测试

3.3.1 实验数据与预处理

实验数据采用高分二号卫星影像数据，数据来源于实验室，已经过初步处理，数据的具体情况如下：采用 2017 年 1 月至 12 月时间跨度的 17 景 L1A 级已经过辐射定标的 GF-2 遥感影像，通过正射校正模块对覆盖整个深圳市的 17 景高分辨率影像经过 Rational Polynomial Coefficient（RPC）正射校正、Pansharpening 算法图像融合、图像镶嵌、模板匀色处理等遥感影像预处理步骤，以上过程均在 PIE-Ortho 软件 5.0 完成。经过多光谱影像与全色融合后，不仅有多光谱 4 个波段丰富的影像信息，而且在融合处理后数据的空间分辨率提升为全色波段的 1m。处理完后的图像大小为 90698*46444，每个像元 2bytes 大小，因此图像位深 16bit，十进制为 512，四个通道总共有 32.9Gb 大小，如图 3-22 为处理后的示意图。而在实验中，并不是整个深圳是遥感影像数据放进本类库运行，而是事先选取一定范围内的 50*50 的分辨率保留位置信息与通道数的裁剪。

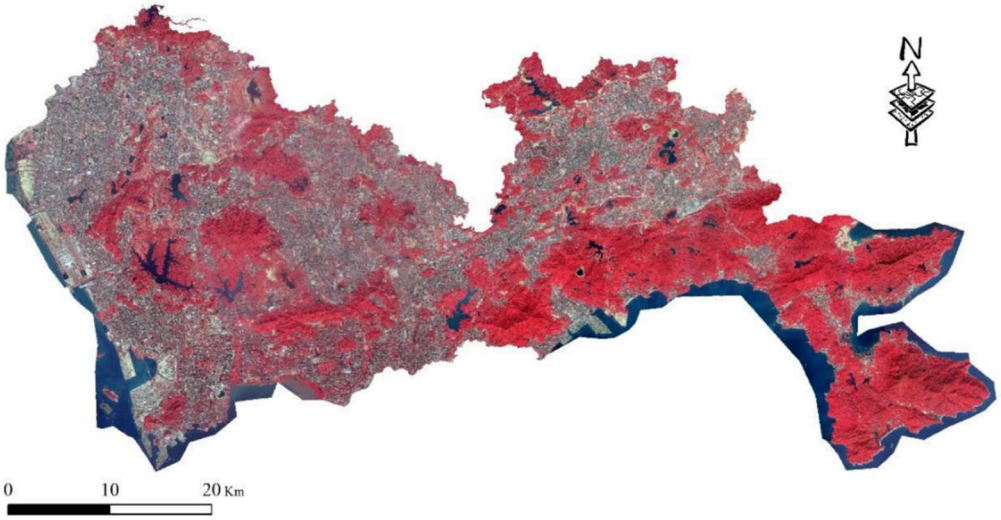


图 3-22 高分二号遥感影像

3.3.2 测试与结果分析

为了验证本类库所实现的 CNN 库可行，现测试其可行性，验证 CNN 前向可否走通，验证 Forward-BP 在本类库中是否真的能更新参数。

为了验证前向传播，设置如图 3-23 两种结构，左右两个结构深度一样，但左边的结构 1 广度更小，而且使用 percentify 作为最后的激活函数，现对两个结构分别运行 30

次，其运行时间结果如图 3-24，可以看到刚开始的时候两个结构都出现了高位运算时间，说明内存及 CPU 在刚开始运行 CNN 前向时反应较慢。而且结构 1 相对更稳定，而且虽然两个结构的参数量差 6 倍左右，但结构 2 运行时间最少是结构 1 的 20 倍，说明 CNN 结构的参数量增加以后运算量比线性增长更快。

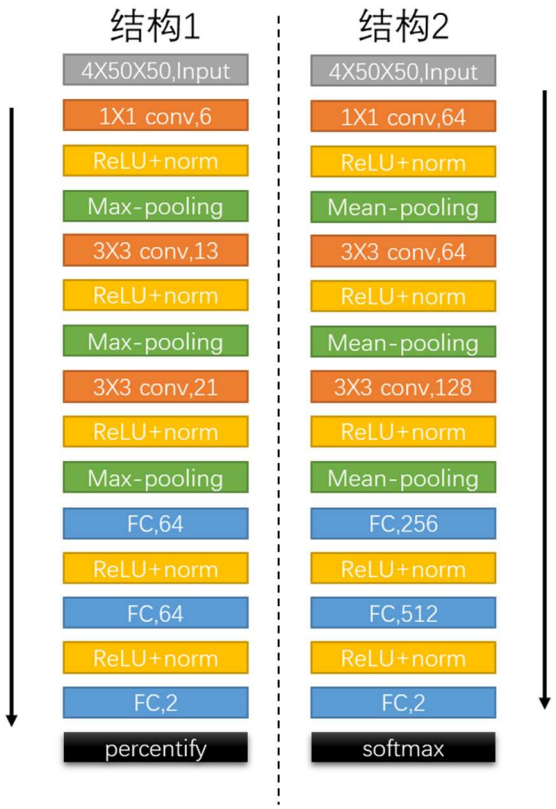


图 3-23 前向传播测试 CNN 结构

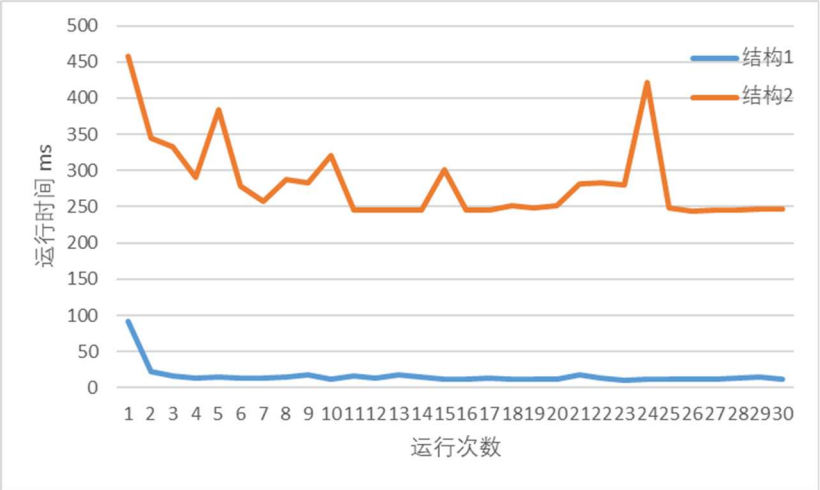


图 3-24 CNN 前向传播运行时间图

在测试 Forward-BP，我们只用结构 1 来做测试，只对同一张图像做学习，循环 30

次，但为了做对比，结构 1 的损失函数分别用 MSE 与 CEE 进行，学习率都为 0.1。运行结果如图 3-25，实线代表运行时间，单位为 ms，可以看到两者的运行时间差距不大，浮动的原因更多的应该来源于硬件，运行时间都在 20s 左右，而经过 Forward-BP 学习以后的准确率却相差有点大，虽然准确率都在增长，但 CEE 基本在第五次就已经达到了 90%准确率，而后增长缓慢，而 MSE 的准确率的上升更接近线性，这也符合其数学模型的推导。

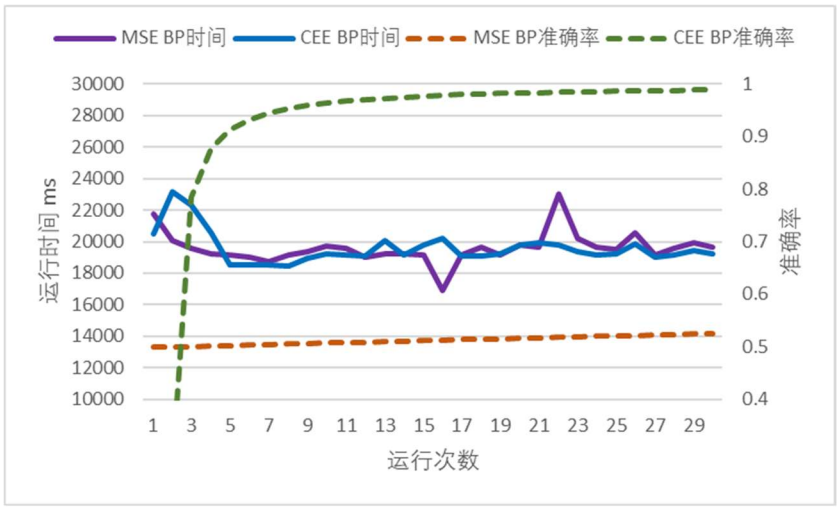


图 3-25 Forward-BP 运行时间图

经过上述的前向传播与 Forward-BP 训练以后的准确率变化，运行情况符合预期，说明 CNN 前向传播可以正常运行，Forward-BP 更是完成了可以深度学习的目标。本类库的两个重要目标已达成。

3.4 本章小结

本章基于第 2 章的数学模型定义与推导在 Java 上实现，先是严格根据 2.2 节的 CNN 前向传播的数学模型来编写本类库的前向传播模块。然后在 BP 算法上，本研究提供了一个 Forward-BP 的内附在前向传播的求导思路，并且根据 2.3 节的 BP 求导逻辑来实现。最后经过测试本类库的两个功能都已经实现，但效率确实不高，除了硬件以及 Jre 的先天缺陷外，主要还是因为代码没优化以及一些 BP 算法里的一些优化器没有实现的原因。值得一提的是，本类库里有一些能读取遥感 tiff 文件至 csv 文件的 API，这让遥感研究人员（大部分使用 Python）不用再花心思去找第三方库或在 Tensorflow 调整配置文件才能读取多通道 tiff 文件。

第4章 基于 CPU 运算的分布式 CNN 算法设计

第 3 章实现的 Forward-BP 存在一定的效率问题，而且本研究的一个重点是定制化云计算平台，因此能运用云计算平台是一个重点，因此要在本类库里实现并行计算甚至是分布式计算。同时第三章实现的 Forward-BP 在本章中只作为一个运算模块存在。在确定了基本目标后，本章的技术支撑以 MapReduce 编程模型实现并行计算流的组织，分段加锁结构来实现线程安全，socket 技术来保证不同主机间的通信。不采用 Hadoop 最大的原因在于其 I/O 是通过硬盘的，这极大的影响了运算速度。不采用 Spark 则为其运算逻辑仍是 MapReduce，但没有针对 CNN 定制化实现，需要编程人员去具体把控 Map 和 Reduce 两个流程。同时，不管是现有哪种成熟的分布式框架或接口，如 Flink 或 MPI 等，都需要在每台电脑进行安装调试，而在本研究中，只在 Master 线程所在的主机需要引入 gdal 第三方库来读取遥感数据外，Slave 线程所在的主机并不需要其余的第三方库，其运算逻辑在本类库源码中已经充分实现完成，这充分体现了“定制化”的优势。

4.1 单机多线程计算环境架构设计

在 2.4 节中阐述了并行计算中 MapReduce（后称 MR）编程模型与 Hadoop MapReduce 的架构流程，以及 ConcurrentHashMap 如何在保证效率的同时实现了线程安全，因此本研究的架构设计一样以计算任务与线程安全两个角度去设计架构，通过架构的设计去满足两个功能并且以给后文实现提供蓝图。

4.1.1 并行计算架构设计

并行计算的架构还是打算使用 MR 编程思想，即使它在商业里已经被淘汰或在被逐渐淘汰的路上，这是因为其面对较大业务时多个子业务之间的耦合性极差，而在本研究中，业务单一，而且不是使用 Hadoop 那套庞大的 MapReduce 框架，所以使用 MR 的思想而不用现有庞大的库是针对本研究的，是具体问题具体分析的。

在确定了 MR 作为本研究并行计算实现的思想后，就应该确定 Map 什么，怎么 Map

的问题。在 BP 算法里，有两种可以并行处理的对象，一个是计算时的矩阵，一个是 BP 计算。前者的实现在于用 GPU 实现，GPU 的结构是由很多 Streaming Multiprocessor（SM），一个 SM 又包含着很多的运算单元，但一般以 SM 为基本单元，它对矩阵的并行运算在于矩阵的每个元素都可以使用一个 SM 来与相应的 w 与 b 进行运算，而不像本类库运用 CPU 运算时是遍历元素，因此 GPU 的并行在于此。而它也是可以运用 MR 思想去再 Map 矩阵元素给 SM 计算后再 Reduce 回来的，但本研究以框架设计为主，加上现有资源——Java 没有直接调用 GPU 的库——也不支持使用 GPU 来进行 MR 的研究，因此本研究以第二种可以并行的对象来研究并行算法的实现。而针对 BP 计算可以使用并行计算的理论依据在于梯度下降，现有的 CNN 训练有多重梯度下降法，一种是批量梯度下降法（Batch Gradient Descent，BGD），一种是随机梯度下降法（Stochastic Gradient Descent，SGD），一种是小批量梯度下降法（Mini-batch Gradient Descent，MBGD），除了 SGD，都可以运用于并行计算的，结合式(2.51)式(2.52)用数学表达为式(4.1)与式(4.2)，其中的$\sum_{j=1}^{m}\delta_j^l(a^{l-1})^T/m$即表达了在 BGD 或 MBGD 中参数的更新可以多个样本计算了梯度以后求平均更新。同时在两个式子中也能看到每次训练时可以串行也可以是并行的，直到都算出来以后进行平均即可。

$$\mathrm{W}^l = \mathrm{W}^l - \alpha\frac{1}{m}\sum_{j=1}^{m}\delta_j^l(a^{l-1})^T \tag{4.1}$$

$$\mathrm{b}^l = \mathrm{b}^l - \alpha\frac{1}{m}\sum_{j=1}^{m}\delta_j^l \tag{4.2}$$

因此，MR 的对象是一批输入为训练样本的 BP 过程，即在本研究中是一批在同一时间需要并行运算的 Forward-BP 对象，由此出发，Map 的对象是$\sum_{j=1}^{m}\delta_j$，Map 的结果是每一个δ_j，而 Map 是需要键值对结构进行数据关联的，在这里以 HashMap 来构建键值对关系，每一个 Forward-BP 对象为 value，同时给一个身份标签 Integer 作为 key，即 Map 后的存储结构如 HashMap=$\{\mathrm{Interger}_j:\delta_j\}$，确认了 Map 以后的结构，即为 Reduce，这里直接根据定义的 HashMap 作为依据计算$\sum_{j=1}^{m}\delta_j/m$，MR 并行计算流的图如图 4-1。

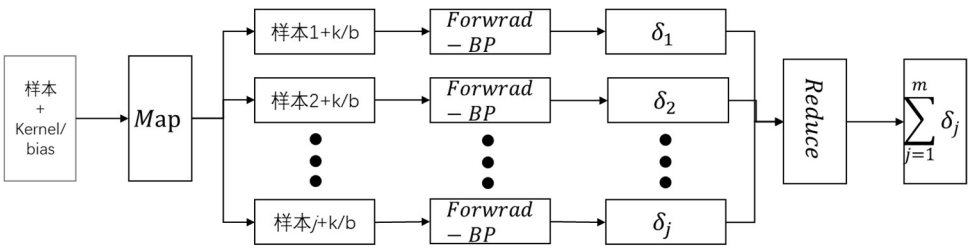


图 4-1 MapReduce 计算流图

4.1.2 数据同步架构设计

在确定了使用 MR 编程思想实现并行计算以及明确计算流的流程后，在异步计算时如何同步的问题就随之出现，在此打算借用 ConcurrentHashMap（后称 CCH）的分段加锁实现思路来实现数据的安全性。在 CCH 中是给 HashMap 动态加锁，初始锁的数量为 16 个，它是给某段数据加锁，说明如果在某个时间段要频繁访问读写这个段的数据依然会造成堵塞，但在 CCH 里也不会给每个 key-value 都加一个锁，因为这反而又会频繁的创建销毁锁引起的性能损耗，而在本研究中，在上文确定了 MR 的计算流以后，每次的 Forward-BP 都是一个线程的任务，而且每次 Map 以后的任务都是独立任务，并没有与其他的线程产生交互直到 Reduce，因此本研究在主线程依然使用两个 HashMap 来做数据的存储，一个是 kernel 与 bias 的存储，其形式为：HashMap h_kb={“kernel”：kernel，“bias”：bias}，一个是会根据线程数来做存储，key 为线程编号，value 内容为 h_kb，但会是深复制，以此保证数据的安全性，即 HashMap h_d={Interger:HashMap h_kb′}，在内存堆区域根据线程数开辟新的空间存储 h_kb 的深复制对象。因此与线程打交道的是 h_d，每一个 key-value 只与一个线程打交道，用 CCH 数据分段的思想与借用了一定的乐观锁的概念来构建，以此提高数据的吞吐效率，其数据的组织流程如图 4-2，其中直角方框代表数据容器，圆角方框代表运算模块，单向箭头表示数据流向，双向箭头表示浅复制，实现了数据共享，即不同变量名指向堆里的同一个数据。

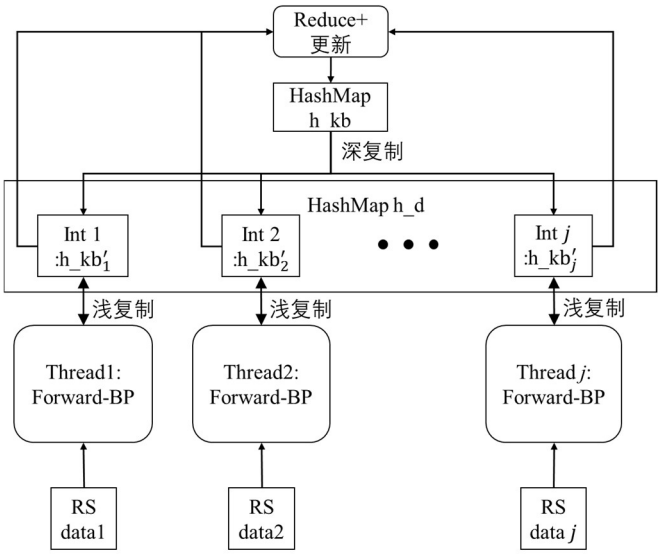


图 4-2 并行计算数据组织图

从图 4-2 就可以清晰明了的了解在本类库里并行计算组织数据的结构，在 HashMap

h_d 里实现了数据的分段而保证独立，因此即使多个线程对一个 HashMap 修改数据都不会引起脏数据的问题，在此需要明确为什么 HashMap 在一般情况下不可以用在多线程并发的情况，并对比本研究的数据组织，以此说明本数据组织的正当性。HashMap 不能在多线程使用一般会引起两个问题，一个是脏数据，即为多个线程对 HashMap 的同一个 value 做修改，即对堆里的同一个对象做修改，这是多线程并发环境下经常出现的问题，为了解决这个问题，采用了两个 HashMap，h_kb 变为 h_d，以保证每个线程都只有一个独立的 HashMap 对象予其读取与改写；HashMap 的另一个并发环境会出现线程不安全的情况是会出现闭循环（线程利用率 100%），这个问题出现只存在于多线程对 HashMap 同时 put 了几组数据，这可能会超出 HashMap 的容量，从而在多线程同时扩容转移元素的时候产生了闭环，如果再有 get 操作到形成闭环的地方就会造成线程利用率 100%，但在本数据结构里不会出现这种情况，一个是因为主线程的 HashMap h_d 在跟线程产生交互前就已经定好 HashMap 的大小，就算出现线程特别多 HashMap 要扩容的情况也是已经在单线程内完成，而后续只不过是读写 HashMap 中的 key 所指向的对象而已，因此多线程可以不受限制的读取 HashMap 里的数据，这也借鉴了乐观锁的思维。因此，通过架构设计，将数据组织架构结合 MR 的计算流架构，将线程不安全的 HashMap 变为线程安全的 HashMap，也免去了 CCH 或 HashTable 在结构里加锁的“沉重感”。

4.2 分布式环境下的架构设计

4.2.1 分布式环境的并行计算架构设计

以上的计算框架与数组组织架构都是针对单机（单 CPU）多线程的并行环境的，而在多机多 CPU 的并行环境，亦或称之为分布式计算环境下，与单机情况有多少有些不同。不同之处主要在于怎么将单机多线程的 MR 计算流与数据组织架构在联机间重现。

不管是实现分布式并行计算还是数据组织，都需要在不同机子间通信，这里采用 2.5 节介绍的 socket 技术，因为它在实时通信以及稳定性上都受到广泛认可。因此，问题转换为如何将 socket 加进图 4-1 与图 4-2 所示的并行计算流与数据组织架构中了。

首先针对并行计算的 MR 流架构重构，需要明确的是在单机多线程环境下有个主线程一直在进行 MR 的流程，它掌控者着主 kernel 容器与主 bias 容器的持有权并在 Map

的时候将各复制对象分发给各线程以让子线程进行 Forward-BP，最后的 Reduce 以及后续的计算也是在主线程进行的。遵循这个逻辑架构，分布式环境下也需要一个主线程来保证数据 kernel 与 bias 数据的完整性并且能分发任务给子线程，而子线程也即承担 Map 以后的 Forward-BP 计算任务的线程的逻辑是要结合 socket 技术分析的。socket 作为一个通信技术，封装了 TCP/IP 协议提供 API 以供程序员使用，只需提供 IP 地址与接口号即可两机通信，如果像在单机多线程环境下主线程不断的与其他机子的线程建立了通信传输了数据以后就让它们进行 Forward-BP 的计算是不现实的，因为这只满足 MR 计算流的流程，数据虽然可以如图 4-2 根据架构设计同步，但这样的计算流架构的缺陷在于一个线程建立了很多对 socket 以后的建立用于所有 socket 通信实例的 BufferedWriter 及 BufferedReader 是串行的，特别是实例化了 BufferedReader 以后是要等待对方发送了整个运行才会往下走，而在多线程环境下这是会造成性能损耗，并且这样就丧失了数据同步架构设计的优势了，只需设计成串行即可。因此，本研究提出了在主线程之下建立多个专门用于通信与传输数据的线程以发送需运算的数据及接收结果以后写进 h_d 中的相应位置。为了区分，这里借用 Hadoop 的主从模式（Master/Slave）来区分，这以上所描述的可以称为 Master 的运算逻辑。

而实际运算的 Slave 运算逻辑本质上与单机多线程计算环境类似，给了计算任务就开始计算，但在单机环境下是主线程不断的唤起线程来进行计算，计算完以后线程就消亡了，而在多机环境下不可能 slave 计算完一轮以后就消亡，然后再手动的开启，因此结合 socket 技术，将 slave 设计成一直开启的状态，因此在 2.5 节所述的 socket 技术 C/S（Client/Server）模式中 slave 充当 Serversocket 的角色一直是开启的并且等待着数据以及计算任务开启的指令，而在 Master 主线程下用通信的线程在 socket 里则是 Client 角色，这是需要明确的，逻辑上是清晰的，但描述上或许有所混淆。综合以上的架构设计，现做出分布式计算环境下的 Forward-BP 的计算架构图 4-3，图中的点线方框为主机，圆角方框为线程，直角方框为运算，单向实线表示计算流的流向，虚线也是计算流的流向，粗虚线则为迭代计算的分界点。在 Master 线程所在主机里一直存在着 Master 线程，它承担着读取 RS 数据以后进行 Map 以后唤起相应数量的通信线程，让通信线程将 RS 数据连同 kernel 和 bias 传输给各个 Slave，每个 Slave 线程接到了数据和计算任务的指示以后就开始了 Forward-BP 的计算，Slave 线程并不会消亡，一直就存在着，计算任务完成并将结果返回给 Master 所在主机的通信线程后即等待下一个计算任务的到来，而且一个主机内可以根据需要唤起多个 Slave 线程。通信线程接受到 Slave 线程计算完成传

输过来的结果后将数据写入主线程所持有的 h_d，然后进行更新 kernel 与 bias 的参数，然后进入下一轮循环。

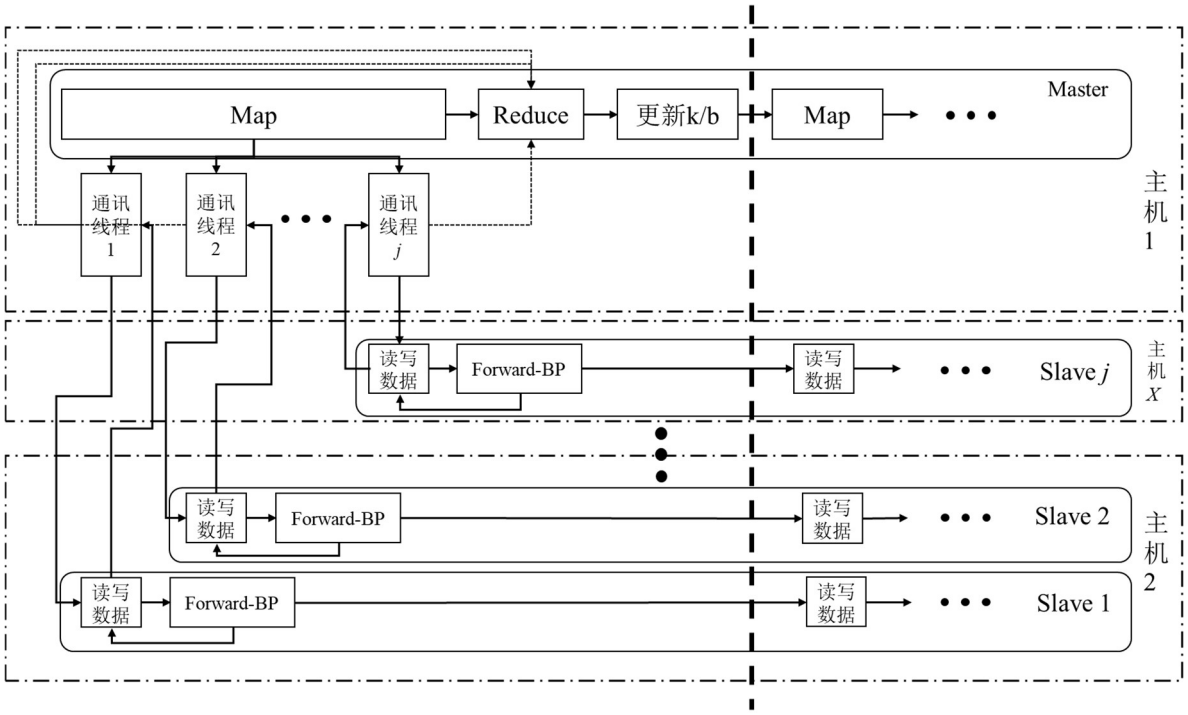


图 4-3 分布式计算环境下的 MapReduce 计算流架构图

4.2.2 分布式环境的数据同步架构设计

上节介绍完了分布式环境下的并行计算架构，本节将详细的介绍本研究的分布式计算环境下的数据组织架构。分布式计算环境下数据组织结构因为计算架构的变化而出现变化，但依然保持 4.1.2 节的数据组织整体结构，即有一个 h_d 来存储需要被多线程读写的 kernel 与 bias，但由于需要多机通信，因此在通信中使用 Java 内置的序列化来将 h_d 的键值对与需要传输的数据序列化后在通讯线程与 Slave 线程中传递。序列化承担的任务具体有两个，一个是承担通信线程发送给 Slave 线程的样本数据与 kernel 与 bias，一个是承担 Slave 线程经过 Forward-BP 计算以后将更新好的 kernel 与 bias 打包好传回给对应的通信线程。其具体的数据架构为图 4-4。图中的四直角方框表示 Java 内置数据，四圆角方框线程，两斜角方框为序列化后的数据，两圆角两直角方框为运行程序，单向箭头为数据流向，双向箭头表示浅复制数据共享。值得注意的是在每个 Slave 线程里有一个 h_kb，这个 h_kb 是实例变量，与 Master 线程的 h_kb 并没有直接关系，但结构与初始值与 Master 线程的 h_kb 相同，而是从序列化对象里反序列化后解析出来的。从图 4-4 可以看到三种线程之间的数据流向与数据关系，通过这个数据架构保证了数据在异

步计算时保持安全以及健壮性。

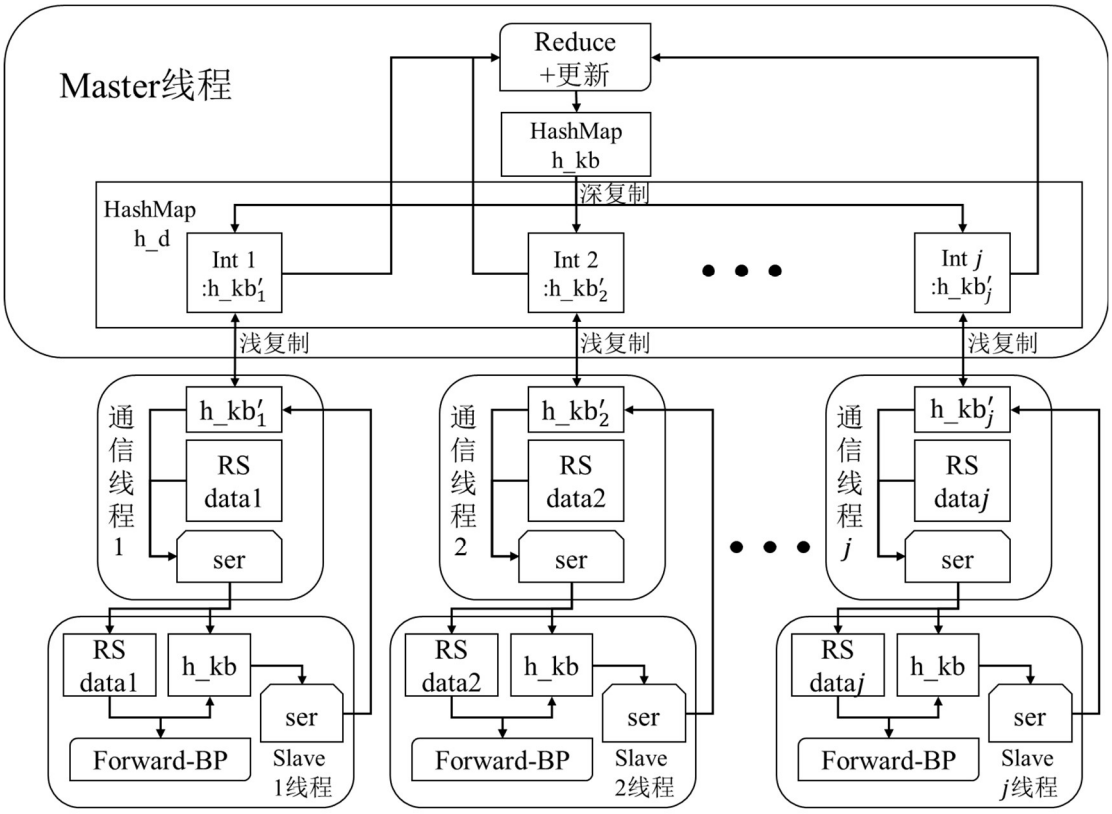


图 4-4 分布式计算环境数据同步架构图

4.3 实现与测试

4.3.1 基于 Java 的实现

在单机（单 CPU）多线程的环境下进行并发计算直接根据 4.1 节列出的 MR 计算流与数据组织结构在 main 函数即可实现，因此在本小节重点描述在分布式环境下的并发计算的实现，这更直接的关联在云计算平台的可行性。

在 4.2 节中明确了为了保证数据安全以及分布式环境下的运算，有三种线程情况，因此这里在本类库中也相应的实现了三个类，在本类库分别命名为 chuanshu_master，master_thread 与 chuanshu_slave 三个类，分别对应 4.2 节描述的 Master 线程、通信线程与 Slave 线程。所实现代码的工作流程图如图 4-5。

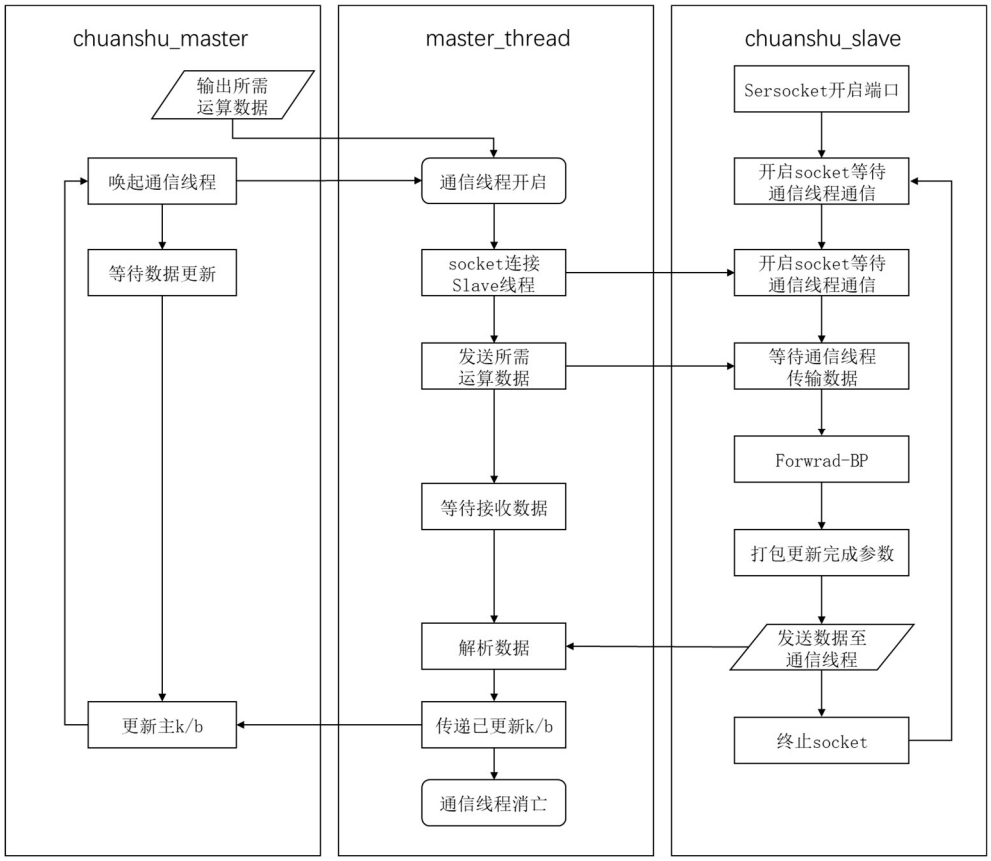


图 4-5 程序运行时的三种线程运行流程图

图 4-6 为 chuanshu_master 类的结构，注意到其除了 main 函数外只有两个函数两个变量，而且两个变量是类变量，说明 chuanshu_master 类的定位很明确，就是在主函数里不断的开启通信线程来唤起 Slave 线程的计算流程，d_kernel 与 d_bias 即为存储 kernel 与 bias 的 HashMap，它们在每次传输的时候都会深复制给一个具有 Slave 线程数大小的 HashMap h_d，以此来做为线程安全的 HashMap，在多线程计算完成以后 h_d 即进入 update_kernel_bias 中进行参数的更新，update_kernel_bias 即根据式(4.1)与式(4.2)实现，更新结果直接写到两个类变量中。write_txt 则是将 kernel 与 bias 从内存中以 txt 格式写入硬盘中。

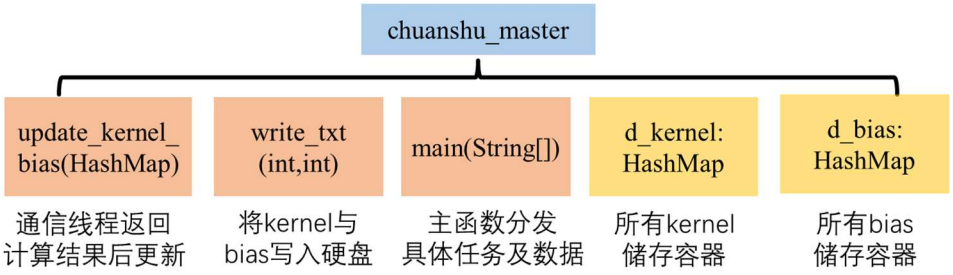


图 4-6 chuanshu_master 结构

图 4-7 为 master_thread 结构，也即通信线程的结构，实例变量 IP 与 port 即为 IP 地

址与端口号，是 Master 线程提供的，kernel_bias 表达的即为图 4-4 中的通信线程的 h_kb'，但实现时稍有偏差，在 Master 给通信线程时是个空的 HashMap，但两者所指的对象依然是同样的，这是为了减少运行开销所做的优化，在 Slave 线程传回序列化对象后通过将变量名指向反序列化解析出的 kernel 对象与 bias 对象再在 kernel_bias 中建立键值对。zhibei_mat 为已序列化并存储在硬盘中的以三维数组存储的 RS 数据，chuanshu_master 给予 master_thread 的只是其在硬盘中的位置，在构造函数中就将其读取进 mat 中，realsocre 则为 RS 数据的类别，最后这些训练相关的参数连通 chuanshu_master 的 d_kernel 与 d_bias 都被一并序列化后传输给 chuanshu_slave，这里有个关键点是序列化后的对象与原对象不相关，因此 chuanshu_master 的 d_kernel 与 d_bias 可以直接序列化。在图 4-5 的中间大框 Master_thread 中即为其实现后的工作具体流程，可以看到在程序运行时通信线程虽然是与 Master 线程直接沟通的，但在 socket 的 C/S 模式中为 Client 角色。

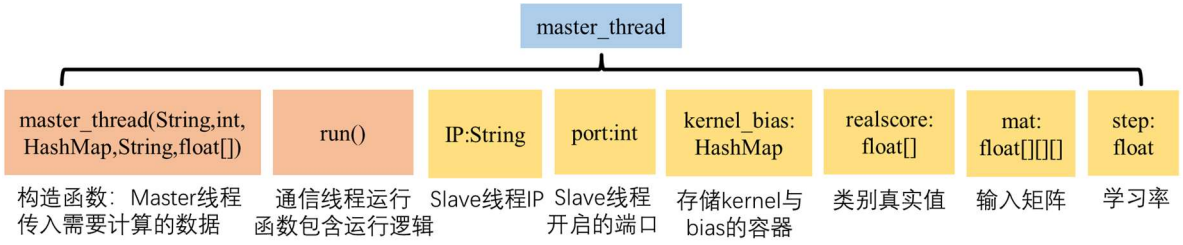


图 4-7 master_thread 结构

图 4-8 为 chuanshu_slave 结构，其中的 port 为 Slave 线程作为 Serversocket 开放的端口号，IL_mat、d_kernel、d_bias、real_score 则为从接收的序列化对象中解析出来的，而后在 run 函数中进行 Forwar-BP 的计算。在图 4-5 中，可以看到，与 master_thread 对应，chuanshu_slave 在 socket 的 C/S 模式中充当 Server 角色，并且它是一直开启的，在计算完本轮 Forward-BP 后并没有关闭，而是继续开启 socket 等待下一轮数据的到来。

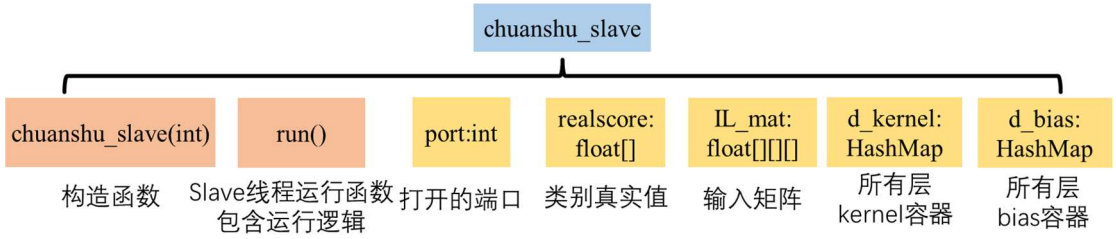


图 4-8 chuanshu_slave 结构

因此，在实际运用中运行 Slave 线程的主机只需有 chuanshu_slave 类以及除输入层类（InputLayer）以外的所有 CNN 相关的类即可（大小仅为 200kb），不需要再带入 gdal 库，因此尽可能的实现了即插即用的思想，即直接复制粘贴到需要运行 Slave 线程的机子上就可快速运行。

4.3.2 测试与结果分析

在 Java 实现了以后，本次测试集中在分布式计算环境下的并行计算能否实现，因为分布式都能成功，单 CPU 也是肯定可以成功的，因此只验证分布式环境下的情况，并且在验证了分布式计算的可行性以后，对比了在不同分布式环境下的计算效率。

在验证可行性的测试上，我们不去多次测试来验证其是否能成功，而是运用反证法思维，即如果本类库的分布式并行计算 Forward-BP 不成功，那会出现两个情况，一个是运行失败，一个是数据组织问题，前者很好判断，直接报错即为运行失败，后者则可以设计这样一个测试。给所有的 Slave 线程一样的 RS 数据与 kernel 及 bias，每次进行完 Forward-BP 后输出其更新结果，然后再观察 Master 线程经过更新前后的结果，因为 Master 的 update_kernel_bias 根据的数学模型为式(4.1)与式(4.2)，其中的累加后除个数说明了如果是相同的参数则结果与单线程运算的 Slave 的 Forward-BP 后的结果是一样的，如果不一样则说明失败。因此只用一次实验即可验证分布式环境下的并行计算 Forward-BP 是否能成功。

由于疫情原因，不能返校，因此本次分布式环境测试的平台以笔者家里的计算机为依据，一台是正好与学校实验室所用的计算机相同配置的 DELL 品牌机，一台是组装的 AMD 平台的主机，其配置对比如表 4-1，其中 AMD 平台的内存基础频率其实为 2133MHz，但运用了 XMP 技术超频至频率 3466MHz。

表 4-1 测试平台配置列表

配置项	DELL（Intel 平台）	AMD 平台
CPU/虚拟核数	I7-7700/8	Ryzen5 2600/12
主板	DELL 办公用主板	MSI B450M MORTAR
内存大小/内存频率	16G/2400MHz	16G/3466MHz(XMP)
电源	260W	SAMA 24K 750W

设计一组 Intel 平台与 AMD 平台各一个 Slave 线程与 Master 线程的运行情况，在测试中给两个平台同样的数据，得出图 4-9 所显示的三种线程的运行情况，其中绿框为第一次更新，蓝框为第二次更新，黄框为第三次更新，三者每次运行时间都是 Master 最慢完成，说明在 Slave 线程运行完以后 Master 才汇总，而且可以看到三者每次更新后的准确率都是一样的，这是符合预期的，因为两个 Slave 线程都是运算一样的值，而且参数也是一样的，因此结果肯定也是一样的，在传回数据的过程中也没有损失数据，证明了

线程安全，保证了数据的完整性。



图 4-9 线程运行情况示意图

在验证了在分布式计算环境下能正常进行计算以及保证数据的完整性与安全性后，进一步的验证其效率提升性。为了简化实验步骤，不再使用图 3-23 的结构一，减少一层卷积层以及全连接层，这是为了减少在多线程时候的因为内存及网络拥堵而造成的不相关条件的损耗时间，具体结构如图 4-10，在计算环境这一列，将 AMD 平台记为 A，Inetl 平台记为 I，后面紧跟线程数，在两台主机的不同计算环境下其性能，初始准确率都为 40.9%，学习率为 0.1，输入的都是同一副图像，从以上的可用性测试中获知运行的准确率只与运行次数有关，而不是样本数相关，则最后有表 4-3。因为实验条件有限，加上最初是在 Intel 平台上进行开发的，因此 Master 线程与通信线程是运行在 Intel 平台上的，但根据图 4-3 分布式计算环境下的 MapReduce 计算流架构图所示，在 Slave 线程进行 Forward-BP 计算时对 Master 线程进行最后 Reduce 计算的影响并不大。

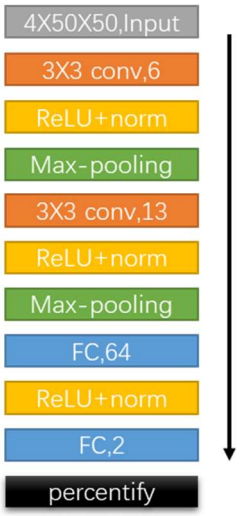


图 4-10 分布式环境测试用 CNN 结构图

表 4-2 循环次数与准确率关系表

循环次数	准确率(%)
2	88.85
4	93.62
8	96.52
16	98.17

表 4-3 不同分布式计算环境下的性能对比表

计算环境	循环总次数	训练样本数	单次最小/最大时间(ms)	总时间(ms)	AMD 平台最小/最大时间(ms)	Intel 平台最小/最大时间(ms)	最终准确率(%)
A1	16	16	6302/9514	110918	5198/8133	-	98.17
I1	16	16	6890/9422	116173	-	6656/9110	98.17
A2	8	16	7625/9703	66991	5437/6844	-	96.52
I2	8	16	7781/11668	71405	-	7404/11481	96.52
A1+I1	8	16	6844/11984	64047	5272/8573	6672/9484	96.52
A2+I2	4	16	8046/12142	37095	4832/7860	8953/11766	93.62
A4+I4	2	16	11533/15501	27049	5524/7912	9938/15126	88.85
A4+I4	4	32	11532/14656	50124	5616/6855	9938/14312	93.62
A4+I4	8	64	11203/13608	94507	5969/8317	10031/11765	96.52
A8+I8	4	64	22056/33395	102915	6620/9524	20228/32897	93.62
A10+I6	4	64	18236/22202	77173	8813/14673	12094/21796	93.62
A6+I10	4	64	26203/35419	118521	6516/9016	20069/34942	93.62
A12+I4	4	64	19135/22326	85121	7359/11859	10049/15215	93.62
A14+I2	4	64	24876/31727	107619	8094/17703	7199/11367	93.62

注：A1 表示 AMD 平台 1 个线程，A2+I2 表示 AMD 平台 2 个线程与 Intel 平台 2 个线程，以此类推

结合表 4-2 与表 4-3 可以观察到以下几个信息：1)准确率只与次数相关，与样本数无关 2)AMD 平台效率普遍比 Intel 平台高 3)在线程都不高的情况下，两个平台的效率差距不大 4)当线程数增加的时候，Intel 平台的效率降低很大 5)并行计算的开启实实在在的提高了整个环境的运行效率 6)当总线程足够大时，Master 线程的效率下降明显。对应的有以下几个原因：1) 这是符合预期的，因为投入的都是同样的数据，运行时的计算步骤一样，结果因此一样，在传回 Master 线程时没有数据丢失经过累加平均即为每一个

Slave 线程运算出的结果，因此在本测试中并不以数据多寡决定其准确率，而以次数决定其最终准确率 2)虽然两个平台的 CPU 在理论数值上差不多，但因为 AMD 平台的内存使用了超频，使得内存与 CPU 寄存器之间的转换效率更大 3)因为在数据量相对较小时，CPU 寄存器与内存的交换频率并不高，CPU 的运算单元能很快的读取需要运算的数据 4)同样这是内存频率直接导致的结果，而且加上在主板设计与电源上，B450 的供电更足使得 CPU 可以有足够的供电去运算，主板北桥的设计也使得内存与 CPU 寄存器之间的通道更大，这都会较为直接的影响整个环境下的运算速度 5)就比如 A10+I6 对比 I1 或 A1 的线程部署情况，数据量前者虽为后者的四倍，但所用时间却减少了 1/3，效率比值达到 6 倍之多，这就是并行计算的优势 6)这是因为 Master 线程是部署在 Intel 平台的，而且随着数据量的增加，即使全部 Slave 线程能很快的返回结果，但 Master 也要进行累加平均计算，数据量的增加也使得 Master 线程的 CPU 与内存的数据交换频次增加，降低了整体效率。

通过本小节，可得出两个主要结论：1)本次的架构设计达到了目标要求，可以在分布式计算环境下运行，并且在测试中也没有报错，实现了可用性 2)分布式计算环境相对复杂，如果要最大程度的发挥性能，不止是要在架构设计上下功夫，线程数量的应用也应根据硬件情况来合理调用，而且在组建云平台时硬件的挑选与合理应用也直接影响着整个分布式计算平台的效能。

4.4 本章小节

本章主要做了两个大的工作，一个是在 MapReduce 编程思想指导下完成了计算流在并行计算环境的架构设计，一个是借用了 ConcurrentHashMap 的分段加锁来实现线程安全的方式来对数据做了架构设计，明确了数据在不同的线程是如何流动的。而且也详细描述了在单 CPU 多线程环境下的架构图与分布式多 CPU 环境下的架构设计，同时也详细描述了如何使用 socket 技术进行分布式环境下并行计算的通信问题。在明确了两个架构的前提下，根据架构完成了 Java 的分布式环境下的并行计算的代码实现，并且进行了可行性测试与不同计算场景的探究，得出了高性能计算是软硬件相配合的结果，只偏重任一个都是发挥不出整个计算平台的全部计算效能的结论。

第5章 总结与展望

5.1 主要工作总结

本论文的主要研究内容是基于 Java 的遥感图像分布式 CNN 计算研究，该研究主要包含了两大研究目的，一个是针对遥感数据的 CNN 的前向传播与反向传播的算法实现，一个是如何在分布式环境下进行并行计算。两项主要研究目的基本完成，代码都已经在本类库里实现，并且都通过了相应的测试，具体研究成果如下：

（1）在明确了 CNN 前向传播的原理以及数学模型后用 Java 实现了其基本功能，有卷积层、池化层、全连接层与输出层，并且基本的激活函数与损失函数都已实现，是一个结构完整的 CNN 库，其可供 Java 研究人员进一步探索。

（2）在进行 CNN 反向传播 BP 训练的代码编写时，根据数学定义及 CNN 的梯度下降法提出了前向式 BP 算法（Forward-BP），这个算法以梯度为研究对象，重写 CNN 前向传播的各种运算过程，将梯度计算内置进前向传播运行过程中，动态更新直到最后的损失函数，这减轻了使用人员的理解难度，只要如 CNN 前向传播一样编写网络结构即可，而且 Forward-BP 使得代码实现的难度降低。它也通过了测试，证明了其可以进行以 MSE 或 CEE 为损失函数的参数更新。

（3）在完成 CNN 库以后，将其中耗时的 Forward-BP 当作一个模块进行下一步的并行计算研究。以设计架构为切入点，以 MapReduce 编程思想为并行计算流的指导思想，理清并行计算的流程，同时借鉴 ConcurrentHashMap 分段加锁技术为解决数据异步计算下如何保证数据同步问题的思路，以架构设计的优势，结合具体项目，将线程不安全的 HashMap 变为线程安全的 HashMap。并且根据架构图以 Java 实现了代码，并且通过了测试，并且在不同线程的分布式环境下，通过合理的配置可使效率比达到了 6 倍，这为在分布式计算环境下如何配置利用硬件提供了参考与思路。

同时，本文虽然以遥感分类为研究切入点，却并未完整的探索出可用于遥感分类的 CNN 结构，但这并不是本文的研究关键点，关键点在于为遥感研究人员提供可在分布式环境下进行计算的 CNN 库，这方面的工作是基本达到要求的，可对遥感数据直接进行

读取进 CNN 库中并运行，而且也以遥感数据为测试样本在分布式环境下进行了运算测试，证明了其可用性。

总的来说，本文的主要研究目的已经达到，实现了在分布式环境下以遥感数据为样本的卷积神经网络并行计算，对深度学习与云计算这两个研究热点都有一定程度的探究与实现，而且相较 GitHub 中个人完成的以 Java 为源码的 CNN 库相比完成度较高，结构更完整，功能也实现了分布式计算。同时在文章之上，第三章的 Java 版 CNN 也不仅限于遥感数据，任何数据都可使用，并且第四章设计的计算流架构与数据架构也并不仅限于本研究，可以继续运用于任何云计算平台的定制化项目中。因此，这是一篇以海量遥感数据为切入点的针对分布式深度学习的研究论文。本类库的具体代码实现可在 https://github.com/CHJERS/cnn4rs 中查看源码。

5.2 不足与展望

本文的研究庞杂，并不以一个目标为主要目标，再加上个人能力不足，在理论探索上多少会有遗漏。同时，实现的技术虽然具备可用性及完整性，但仍有很多工作未完成。针对 CNN 库，有两大方面，一个是功能的健全：1)激活函数与损失函数的补充 2)网络结构的灵活连接性，诸如实现 ResNet 的“短路连接”3)训练时的 batch normalization 或 Adam 等能提高训练效率的功能实现；一个则是代码性能的优化，现在已完成的 CNN 在计算 BP 的效率低下，只相当于几年前的主流性能，虽然可作为研究用，但却不能推广普及。而且因为时间的原因也未进行遥感分类具体项目的验证，这也是未来需要努力的方向。而针对分布式环境计算的设计及实现完成度虽然较高，但在鲁棒性上面有所欠缺，对计算环境的扰动有可能造成整个计算进程的报错终止，而且也未真正的运用于完整的云计算平台，在大规模高性能计算环境下的表现如何尚未可知。

参 考 文 献

[1] 司玉琢, 曹兴国. 海洋强国战略下中国海事司法的职能[J]. 中国海商法研究, 2014, 025(003):9-15.

[2] 陈俊勇. 关于地理国情普查的思考[J]. 地理空间信息, 2014(02):7+13-15.

[3] 余永欣, 杨伯钢, 李立军, 等. 面向地级以上城市空间变化的地理国情监测与应用[J]. 北京测绘, 2020, 34(4):501-506.

[4] 杜培军, 夏俊士, 薛朝辉,等. 高光谱遥感影像分类研究进展[J]. 遥感学报, 2016, 020(002):236-256.

[5] 赵春霞, 钱乐祥. 遥感影像监督分类与非监督分类的比较[J]. 河南大学学报(自然科学版), 2004, 034(003):90-93.

[6] 贾坤, 李强子, 田亦陈,等. 遥感影像分类方法研究进展[J]. 光谱学与光谱分析, 2011, 31(10).

[7] ARMBRUST M,FOX A,GRIFFITH R, et al. Above the clouds: a Berkeley view of cloud computing[R]. Berkeley:University of California Berkeley,2009:1-23.

[8] Villegas D, Bobroff N, et al. Cloud federation in a layered service model[J]. Journal Of Computer And System Sciences, 2011, 78(5): 1330-1344.

[9] 曾生根, 王小敏, 范瑞彬,等. 基于独立分量分析的遥感图像分类技术[J]. 遥感学报, 2004(02):55-62.

[10] N. M. Nasrabadi T. D. Tran Y. Chen. Hyperspectral Image Classification via Kernel Sparse Representation. [J]. IEEE Transactions on Geoscience and Remote Sensing, 2013, 51(1):217-231.

[11] Yuxiang, Zhang, Bo, et al. Joint Sparse Representation and Multitask Learning for Hyperspectral Target Detection[J]. IEEE Transactions on Geoscience & Remote Sensing, 2016, 55(2):894-906.

[12] Celik T . Unsupervised Change Detection in Satellite Images Using Principal Component Analysis and k-Means Clustering[J]. Geoscience and Remote Sensing Letters, IEEE, 2009, 6(4):772-776.

[13] 陈华, 陈书海, 张平,等. K-means 算法在遥感分类中的应用[J]. 红外与激光工程, 2000(02):28-32.

[14] María González-Audícana, José Luis Saleta, Catalan R G , et al. Fusion of multispectral and

panchromatic images using improved IHS and PCA mergers based on wavelet decomposition[J]. IEEE Transactions on Geoence and Remote Sensing, 2004, 42(6):1291-1299.

[15] Small C , Elvidge C D , Balk D , et al. Spatial scaling of stable night lights[J]. remote sensing of environment, 2011, 115(2):269-280.

[16] Bruzzone L , Prieto D F . Automatic analysis of the difference image for unsupervised change detection[J]. IEEE Transactions on Geoscience and Remote Sensing, 2000, 38(3):1171-1182.

[17] 武文波, 陈静. 基于 ETM+的遥感影像信息提取研究[J]. 甘肃农业大学学报, 2008, 43(5):142-146.

[18] Yu Q , Gong P , Clinton N , et al. Object-based Detailed Vegetation Classification with Airborne High Spatial Resolution Remote Sensing Imagery[J]. Photogrammetric Engineering & Remote Sensing, 2006, 72(7):799-811.

[19] Miao L , Qi H . Endmember Extraction From Highly Mixed Data Using Minimum Volume Constrained Nonnegative Matrix Factorization[J]. IEEE Transactions on Geoscience and Remote Sensing, 2007, 45:765-777.

[20] Benz U C , Hofmann P , Willhauck G , et al. Multi-resolution, object-oriented fuzzy analysis of remote sensing data for GIS-ready information[J]. Isprs Journal of Photogrammetry & Remote Sensing, 2004, 58(3-4):239-258.

[21] Fallatah A , Jones S , Mitchell D . Object-based random forest classification for informal settlements identification in the Middle East: Jeddah a case study[J]. International Journal of Remote Sensing, 2020, 41(11):4421-4445.

[22] Zhang L , Zhang L , Du B . Deep Learning for Remote Sensing Data: A Technical Tutorial on the State of the Art[J]. Geoscience and Remote Sensing, 2016, 4(2):22-40.

[23] Melgani F , Bruzzone L . Classification of Hyperspectral Remote Sensing Images with Support Vector Machines[J]. IEEE Transactions on Geoscience and Remote Sensing, 2004, 42(8):1778-1790.

[24] Kussul N , Lavreniuk M , Skakun S , et al. Deep Learning Classification of Land Cover and Crop Types Using Remote Sensing Data[J]. IEEE Geoscience and Remote Sensing Letters, 2017, 14(5):778-782.

[25] Rumelhart D E , Hinton G E , Williams R J . Learning Representations by Back Propagating Errors[J]. Nature, 1986, 323(6088):533-536.

[26] Mcculloch W S , Pitts W . A Logical Calculus of the Ideas Immanent in Nervous Activity[J]. biol math biophys, 1943, 5(4): 115-133.

[27] Cortes C , Vapnik V . Support vector networks[J]. Machine Learning, 1995, 20(3): 273-297.

[28] Lecun Y , Bottou L . Gradient-based learning applied to document recognition[J]. Proceedings of the IEEE, 1998, 86(11): 2278-2324.

[29] Hinton G , Salakhutdinov R . Reducing the dimensionality of data with neural networks[J]. Science, 2006, 313(5786): 504-507.

[30] Krizhevsky A , Sutskever I , Hinton G . ImageNet Classification with Deep Convolutional Neural Networks[J]. Communications of the ACM, 2017, 60(6): 84–90.

[31] Simonyan K , Zisserman A . Very Deep Convolutional Networks for Large-Scale Image Recognition[Z/OL]. arXiv, 2015. https://arxiv.org/pdf/1409.1556.pdf.

[32] Szegedy C, Liu W, Jia Y, et al. Going deeper with convolutions[C]. The IEEE Conference on Computer Vision and Pattern Recognition (CVPR), Boston, 2015: 1-9.

[33] He K, Zhang X, Ren S, et al. Deep residual learning for image recognition[C]//Proceedings of the IEEE conference on computer vision and pattern recognition. 2016: 770-778.

[34] Huang G, Liu Z, Laurens V D M, et al. Densely connected convolutional networks[C]//Proceedings of 2017 IEEE Conference on Computer Vision and Pattern Recognition, July 21-26, 2017, Honolulu, HI, USA. New York: IEEE, 2017, 2261-2269.

[35] Girshick R, Donahue J, Darrell T, et al. Rich feature hierarchies for accurate object detection and semantic segmentation[C]//Proceedings of the IEEE conference on computer vision and pattern recognit. 2014: 580-587.

[36] Ren S , He K , Girshick R , et al. Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks[J]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2017, 39(6):1137-1149.

[37] LIPTON ZC,BERKOWITZ J,ELKAN C.A critical review of recurrent neural networks for sequence learning[J].arXiv,2015:1506.00019.

[38] Sabour S,Frosst N,Hinton G E.Dynamic routing between capsules[C] // Advances in Neural Information Processing Systems 30.Long Beach,2017:3856-3866.

[39] Abadi M,Agarwal A,Barham P,et al. Tensor Flow:Large-scale machine learning on heterogeneous distributed systems[J]. ar Xiv preprint ar Xiv:1603. 04467,2016.

[40] Jia Y , Shelhamer E , Donahue J , et al. Caffe: Convolutional Architecture for Fast Feature Embedding[C]//ACM International Conference on Multimedia. Orlando, FL , USA:ACM, 2014:675-678.

[41] AL-RFOU R, ALAIN G, ALMAHAIRI A, et al.Theano:A Python Framework for Fast Computation of Mathematica Expressions[DB/OL]. (2016-05-09) [2019-03-05].https://arxiv.org/pdf/1605.02688.pdf.

[42] 尹宝才, 王文通, 王立春. 深度学习研究综述[J]. 北京工业大学学报, 2015, 000(001):48-59.

[43] MELL P, GRANCE T. The NIST Definition of Cloud Computing[R]. National Institute of Standards and Technology, 2011.

[44] IBM 云计算中心, HiPODS. “智慧的地球”——IBM 云计算 2.0[R/OL]. [2020-06-07]. https://wenku.baidu.com/view/20849f95dd88d0d233d46a74.html.

[45] 陈国良, 孙广中, 徐云,等. 并行计算的一体化研究现状与发展趋势[J]. 科学通报, 2009(08):65-71.

[46] 牛禄青. 阿里云:创新云计算[J]. 新经济导刊, 2013(03):66-68.

[47] 付江. 腾讯云平台与应用实践[J]. 程序员, 2011, 000(006):34-34.

[48] White, Tom. Hadoop : The Definitive Guide[J]. o'reilly media inc gravenstein highway north, 2012, 215(11):1 - 4.

[49] Ghemawat S , Gobioff H , Leung S T . The Google file system[J]. Acm Sigops Operating Systems Review, 2003, 37(5):29.

[50] Chang F , Dean J , Ghemawat S , et al. Bigtable: A Distributed Storage System for Structured Data[J]. ACM transactions on computer systems, 2008, 26(2):1-26.

[51] Dean J , Ghemawat S . MapReduce: Simplified Data Processing on Large Clusters[J]. Communications of the ACM, 2008, 51(1):107-113.

[52] 陈康, 郑纬民. 云计算:系统实例与研究现状[J]. 软件学报, 2009(05):257-268.

[53] 罗军舟, 东方, 东南大学 计算机科学与工程学院, 江苏 南京. 云计算:体系架构与关键技术[J]. 通信学报学术论坛暨 2011 云计算学术会议, 2012.

[54] Rosenblatt, F. The Perceptron: A Probabilistic Model for Information Storage and Organization in The Brain[J]. Psychological Review, 1958, 65:386-408.

[55] 黄嘉俊, 谭泽芳. OD 矩阵反推方法探究[J]. 智能城市, 2020(3):8-11.

[56] 方腾飞, 魏鹏, 程晓明. Java 并发编程的艺术 : The art of Java concurrency programming[M].

机械工业出版社, 2015.

[57] GeorgeCoubouris, JeanDollimore, TimKindberg, et al. 分布式系统概念与设计[M]. 第五版. 北京: 机械工业出版社, 2018: 84-91.

指导教师对学位论文的学术评语

深圳大学
指导教师对研究生学位论文的学术评语

研究生姓名	黄嘉俊	论文题目	基于卷积神经网络的遥感图像分布式分类方法研究		
学号	2170338304	专业（领域）名称	地理信息与智慧城市	学位级别	硕士

对学位论文的学术评语 （评价论文的学术水平和实用价值，论文有无新见解，论据是否充分、可靠，运用基础和专业理论、知识的实际能力，所体现的科研能力，论文写作是否严谨、科学，是否有学术不端行为，论文存在的主要缺点和问题等，是否达到学位论文的要求）：

该论文实现了一个Java语言的分布式计算CNN类库，并以深圳市1月至12月的17景的高分二号卫星影像数据为例，验证该类库的可用性。整体而言，该论文在工程维度的工作量较大，原创性较高。但以学术型硕士学位论文的标准评价，仍有如下问题：

1）研究目标：该论文强调了研究目标来源于现实的工程需求，如“在了解遥感研究人员在进行深度学习研究时出现的运行时间久、现有深度学习库相对难以配置调试分布式计算环境等存在的现实问题后，明确了两个主要目标...”，“本研究是为遥感研究人员设计并实现一个可用于分布式环境下进行并行计算的深度学习库...”，但论文现有中应有相关的证据或相近阐述其迫切的工程需求，而不是过多的主观带入，否则难以令人说服。

同时，该论文主要着重CNN类库（非全类深度学习库）的普通分布式（不具有按需分配、弹性计算等特性的云计算）实现，在论文摘要、1.1、1.3 等章节应该有所界定，否则和本文工作范围并不匹配。

2）研究创新点：现有创新点的归纳较为粗糙，需要从学术角度进一步凝练和归纳。

3）文献综述：现有文献综述过于分散，应聚焦到本文工作的过去科研工作，而不是泛泛的对遥感分类、云计算、深度学习能主题进行综述。

4）研究线路：研究线路图绘制较为凌乱、缺乏逻辑且与文字描述对应程度较差。“设计分布式环境并行计算流架构”出现多次。“计算流架构”和“安全组织架构”在文中哪个章节有对应工作？

5）研究内容：第三章和第四章的论文核心内容阐述不够充分，未能充分响应论文的研究目标和创新点。第三章出现大量的代码截图和自定义类设计截图，应放置于论文的附录之中，而不是放在主论文进一步侵占了本应详尽阐述研究原理的空间。

6）实验：论文现有最大的问题就是实验不充分，未能良好的体现该类库设计的目标，也未能对该类库的可用性和性能有一个较为合理和全面的交代。

7）其他：先版本论文还充斥着英文摘要、论文格式、摘要格式等的非主干性问题，需要进一步完善。

该论文总体上达到了硕士学位论文的要求，后续需认真修改，同意参与硕士学位论文答辩。

指导教师对学位论文的评阅结果：同意答辩

指导教师（签名）：张星　　日期：2020-06-11

学位论文答辩委员会决议书

深圳大学研究生学位（毕业）论文
答辩委员会决议书

研究生姓名	黄嘉俊	论文题目	基于卷积神经网络的遥感图像分布式分类方法研究		
学　号	2170338304	专业名称	0871Z1 地理信息与智慧城市	学位级别	硕士
答辩日期	2020-07-10				

答辩委员会对学位论文及答辩情况的评语及明确的结论意见（850字以内）：

该论文设计、开发了一套Java分布式计算环境下的卷积神经网络库原型，并基于深圳市高分二号卫星图像进行了类库的有效性验证。论文具有一定的理论意义和较强的实际应用价值。

论文主要创新点如下：
1. 实现了Java环境的CNN类库原型研发，通过前向式BP算法（Forward-BP）对深圳市高分二号卫星图像进行梯度下降测试的验证。
2. 以MapReduce编程模型为并行计算架构，借鉴ConcurrentHashMap的分段加锁结构及结合深浅复制的特性设计实现了数据的同步。

论文写作整体较好、逻辑组织和内容表述清晰，具有较强的创新性，体现出作者较好的专业知识和很好的实际动手能力。答辩过程中论述条理，重点突出，能够清楚地回答委员们提出的问题。作者具有较扎实的基础理论和系统的专业知识，具备了从事本学科科学研究工作的能力。
答辩委员会一致认为该论文达到了工学硕士学位论文水平。
经答辩委员会无记名投票表决，通过该论文答辩，建议授予黄嘉俊工学硕士学位。

答辩委员	表决项目		计票	结论
	毕业论文（准予毕业）	通过	5	通过 √
		未通过	0	未通过□
	学位论文（建议授予学位）	同意	5	通过 √
		不同意	0	未通过□
	学位论文答辩不通过，允许其修改论文后重新申请一次答辩0票。			

秘书（签名）：陆昊　　主席（签名）：王艳东

深圳大学研究生学位（毕业）论文
答辩委员会决议书

<table>
<tr><td colspan="2">研究生姓名</td><td colspan="2">黄嘉俊</td><td>论文题目</td><td colspan="3">基于卷积神经网络的遥感图像分布式分类方法研究</td></tr>
<tr><td colspan="2">学号</td><td colspan="2">2170338304</td><td>专业名称</td><td>0871Z1 地理信息与智慧城市</td><td>学位级别</td><td>硕士</td></tr>
<tr><td rowspan="10">论文答辩委员会出席名单</td><td>委员会成员</td><td>姓名</td><td colspan="2">职称</td><td colspan="2">工作单位</td><td>本人签名</td></tr>
<tr><td>主席</td><td>王艳东</td><td colspan="2">教授</td><td colspan="2">武汉大学</td><td>王艳东</td></tr>
<tr><td rowspan="6">委员</td><td>乐阳</td><td colspan="2">教授</td><td colspan="2">建筑与城市规划学院</td><td>乐阳</td></tr>
<tr><td>黄正东</td><td colspan="2">教授</td><td colspan="2">建筑与城市规划学院</td><td>黄正东</td></tr>
<tr><td>高松</td><td colspan="2">助理教授</td><td colspan="2">威斯康星大学麦迪逊分校</td><td>高松</td></tr>
<tr><td>夏吉喆</td><td colspan="2">副研究员</td><td colspan="2">建筑与城市规划学院</td><td>夏吉喆</td></tr>
<tr><td></td><td colspan="2"></td><td colspan="2"></td><td></td></tr>
<tr><td></td><td colspan="2"></td><td colspan="2"></td><td></td></tr>
<tr><td>秘书</td><td>陆旻</td><td colspan="2">副研究员</td><td colspan="2">建筑与城市规划学院</td><td>陆旻</td></tr>
<tr><td colspan="3">答辩日期：</td><td colspan="5">2020-07-10</td></tr>
<tr><td colspan="3">答辩地点：</td><td colspan="5">线上答辩 腾讯会议：450 188 026</td></tr>
</table>

致　　谢

时光荏苒，岁月如梭，转眼之间三年就已过去，这三年有失去也有收获，失去了一些挥斥方遒的自信，但也收获了一分脚踏实地的沉稳，失去了一些天马行空的梦想，但也收获了一分勤能补拙的感悟，失去了那些稚气未脱的想法，结束了廿年的学生生涯，开始了真正的去接触外面的天地，这其中最应该感谢的是我的父亲黄中以及我的母亲刘海珍，是你们用那坚韧不懈的毅力撑起了整个家，独自承受着苦难，不忍让多一点的艰辛加于你们的子女，这让我得以无忧无虑的度过这二十六年，可以有底气的去做自己想做的事，但回首望去，却心怀惭愧，养育之恩无以为报。

再有要感谢这三年来给予我无限帮助的各位老师，夏吉喆老师给予的实验环境与学习环境让我能够专注于自身的学习研究，张星老师与胡忠文老师在论文完成阶段的耐心指导与平时科研时给予的帮助让我有信心完成我的学业，以及在科研路上给予帮助的老师们，是你们的指导让我一步步的明确了科研方向与目标。在此，我要向你们表达最真挚的感谢。

同时也要感谢一同学习前进的同学，与刘镇华共同进行科研项目并攻破技术难关也让我知道了1+1＞2的道理，以及实验室的各位同僚在科研以及生活上的解惑与帮助让我在科研路上不感到孤单，在此，希望你们前途似锦，并表达我的谢意。

最后，也要感谢我的女朋友陈燕珊，是你在我踯躅难行时给予了无限的慰藉，让我能重振旗鼓的继续前行，也让我的科研路因你而繁花盛开，也是你让我知道了什么是身为男人的责任，让我有了成家立业的打算，谢谢你这一路上的陪伴与鼓励。

攻读硕士学位期间的研究成果

[1] 黄嘉俊, 谭泽芳. OD 矩阵反推方法探究[J]. 智能城市, 2020(3):8-11. （第一作者，通讯作者）

[2] 刘镇华, 黄嘉俊.基于图像校准的多源融合的室内定位方法[J]. 测绘地理信息 （通讯作者，已接收）